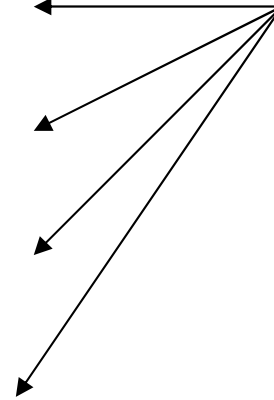


Diagrams

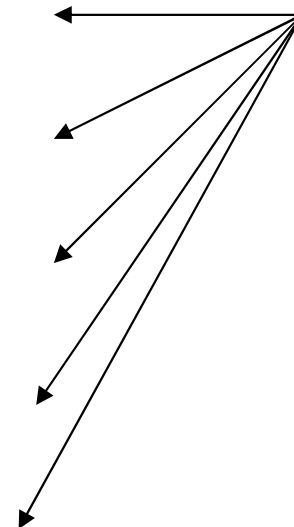
Class Diagram
Object Diagram
Component Diagram
Deployment Diagram

دیاگرام های ساختاری



StateChart Diagram
Activity Diagram
Collaboration Diagram
Sequence Diagram
Use-Case Diagram

دیاگرام های رفتاری



کلیه دیاگرام های ذکر شده در فصول آتی بصورت مفصل شرح خواهیم داد. اما خلاصه ای از برخی دیاگرام های عمده را برای ایجاد ذهنیت ذکر خواهیم نمود .

Class Diagram

در این دیاگرام ما کلاس ها و روابط بین آنها خواهیم داشت .

Object Diagram

این ساختار به مانند عکس گرفتن از اشیا می باشد که برای نمایش دادن اشیا موجود در یک لحظه از سیستم می باشد .

Component Diagram

نمایش بسته ها و اجزایی به مانند کلاس ها در آن می باشد و مزیت آن کاهش پیچیدگی سیستم می باشد.

Deployment Diagram

نمایش دادن وضعیت نهایی عملیاتی نرم افزار و نمایش دادن زیر ساختار هایی که در آینده قرار است این نرم افزار روی آن نصب و اجرا شود و سخت افزار و حافظه در کجا از این منبع اجرا خواهند شد برای ما مدل می کند .

StateChart Diagram

دیاگرام وضعیت برای نمایش دادن وضعیت اشیا موجود در سیستم

Activity Diagram

نمودار فعالیت که رفتار یک شی را مدل می کند و معادل همان فلوچارت است ولی با یکسیری تفاوت جزئی در ساختار و هدف

Use-Case Diagram

لیست تمامی کارهایی که یک کاربر قرار است انجام دهد با این سیستم نمایش می دهد .

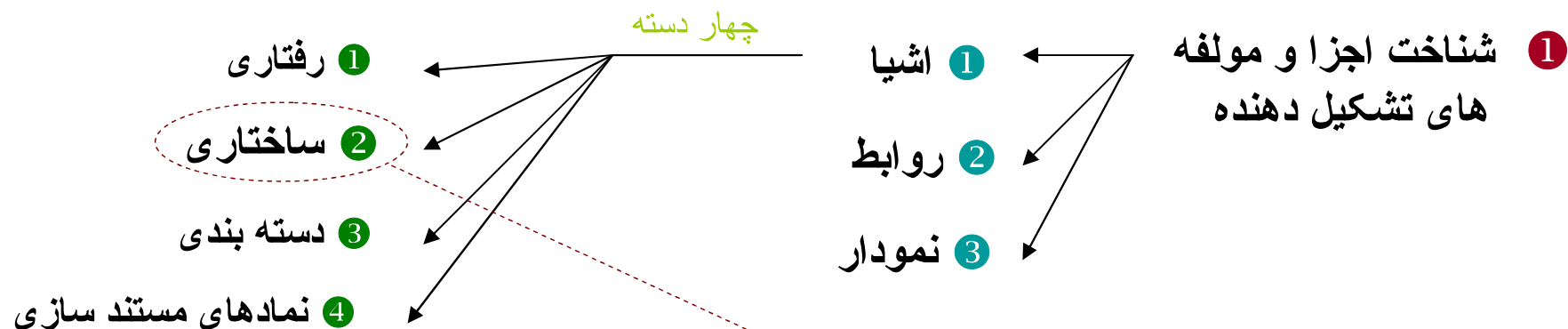
Collaboration Diagram

نمودار همکاری برای شفاف کردن ارتباط اشیا و تاثیر آنها روی هم می باشد .

Sequence Diagram

نمودار مرحله یا فاز برای نمایش دادن زمانبندی نمودار ها که در یک موقعیت با توجه به زمانبندی باید چه کارهایی انجام شود .

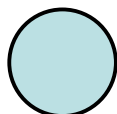
بغیر از ۹ دیاگرام ذکر شده سه دیگر نیز وجود دارد که از مهمترین این دیاگرام ها Class دیاگرام می باشد .



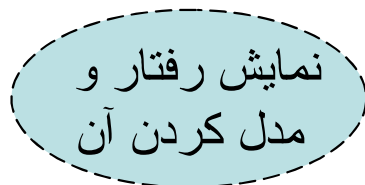
1 کلاس Class:
توصیف يك مجموعه اشیا

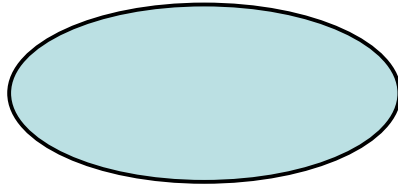
اشیا ساختاری معادل اسم
کلاس در UML

2 رابط ها Interface: مجموعه ای از عملیاتی است که یک کلاس یا یک سرویس ویا یک مولفه و متد ها را مشخص می کند .

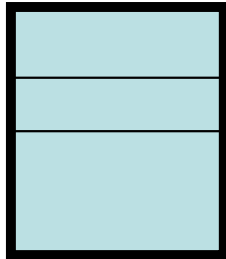


3 همکاری Collaboration: تعریف کننده تعامل است که این تعامل شامل مجموعه ای از قواعد دیگر مولفه هاست که با هم تشکیل دهنده رفتاری است که رفتار بزرگتر از رفتار همه مولفه هاست.

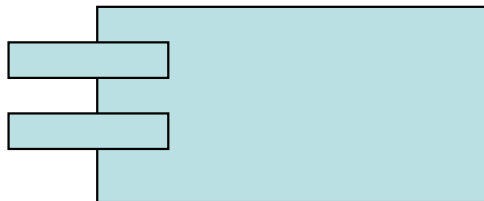




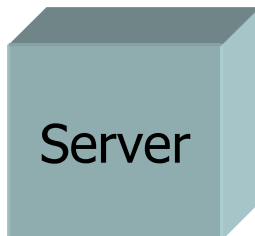
4 **Use-Case** : توصیف مجموعه ای از عملیات که سیستم انجام می دهد برای اینکه نتیجه قابل مشاهده به یک عامل خاص **Actor** برساند .



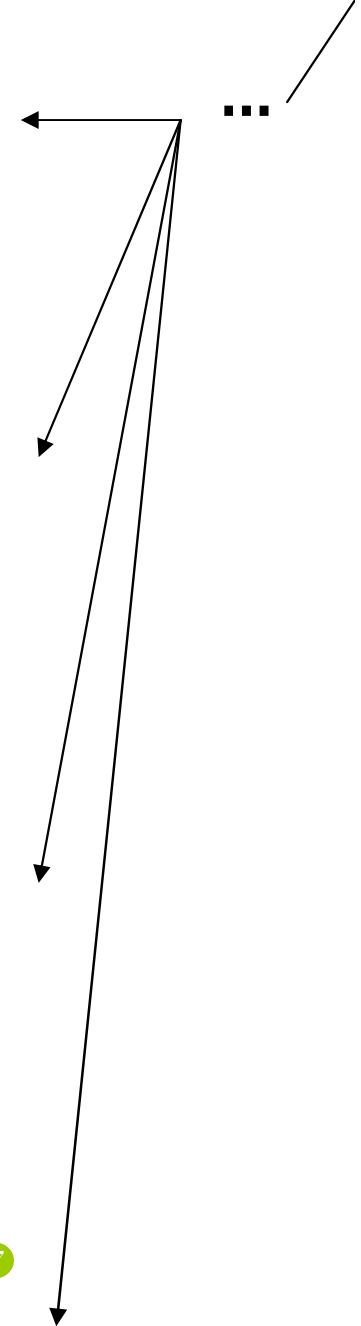
5 **Active Classes** : کلاس هایی هستند که دارای یک یا چند فرآیند یا رشته های اجرایی می باشند و از این رو می توانند شروع کننده یک فعالیت کنترلی باشند .



6 **Component** : مولفه شی زمان اجرا است (**Package** مفهوم زمان طراحی ست ولی مولفه مفهوم زمان اجراست و قابل تغییر می باشد) مولفه ، یعنی یک جز فیزیکی و قابل تعویض از سیستم که تبعیت می کند از یک سری رابط ها و پیاده سازی می کند .



7 **Node** : یک جز فیزیکی از زمان اجرا وجود دارد و نشان دهنده یک منبع محاسباتی و توان پردازش دار است چنین سیستمی مقداری حافظه خواهد داشت .



اشیا ①

ساختاری ②

① رفتاری: (معادل فعل ها در UML) نشان دهنده قسمت های دینامیک (پویا) یعنی حرکت ها مدلهای و پیامهایی که ردوبدل می شود را مدل می کند .

① Interaction : (تعاملات و پیامهایی که بین اشیا ردوبدل می شود)

یک تعامل یک رفتار که تشکیل شده از مجموعه ای از عوامل پیغام میان مجموعه ای از اشیایی که داخل یک مجموعه مشخص قرار دارند و برای رسیدن به هدف معین می باشد .

→
Display

② State Machine : یک ماشین وضعیت یک رفتار که

مشخص کننده یک توالی از وضعیت از یک شی یا

Interaction طی خواهد کرد و در طی حیاتش برای پاسخ

به رویدادها .

Waiting

③ دسته بندی : مهمترین شی دسته بندی package می باشد.

Package : یک مکانیزم همه منظوره است برای سازمان دهی عناصر

در داخل یکسری گروهها این اشیا می توانند اشیا رفتاری یا ساختاری یا ... باشند .

④ نمادهای مستند سازی: قسمت توضیحی UML

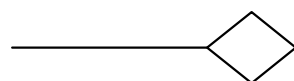
روابط 2

1 وابستگی Dependency : یک رابطه ی مفهومی بین دو شی می باشد که در حقیقت تغییر در یک شی تاثیر می گذارد روی تعریف شی دوم

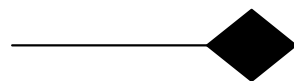
----->

2 همکاری Association : یک رابطه ساختاری که توصیف کننده بین پیوندهاست.

0..1 *
employee employee

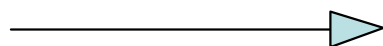


1 تجمع aggregation

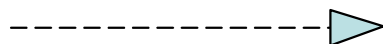


2 ترکیب

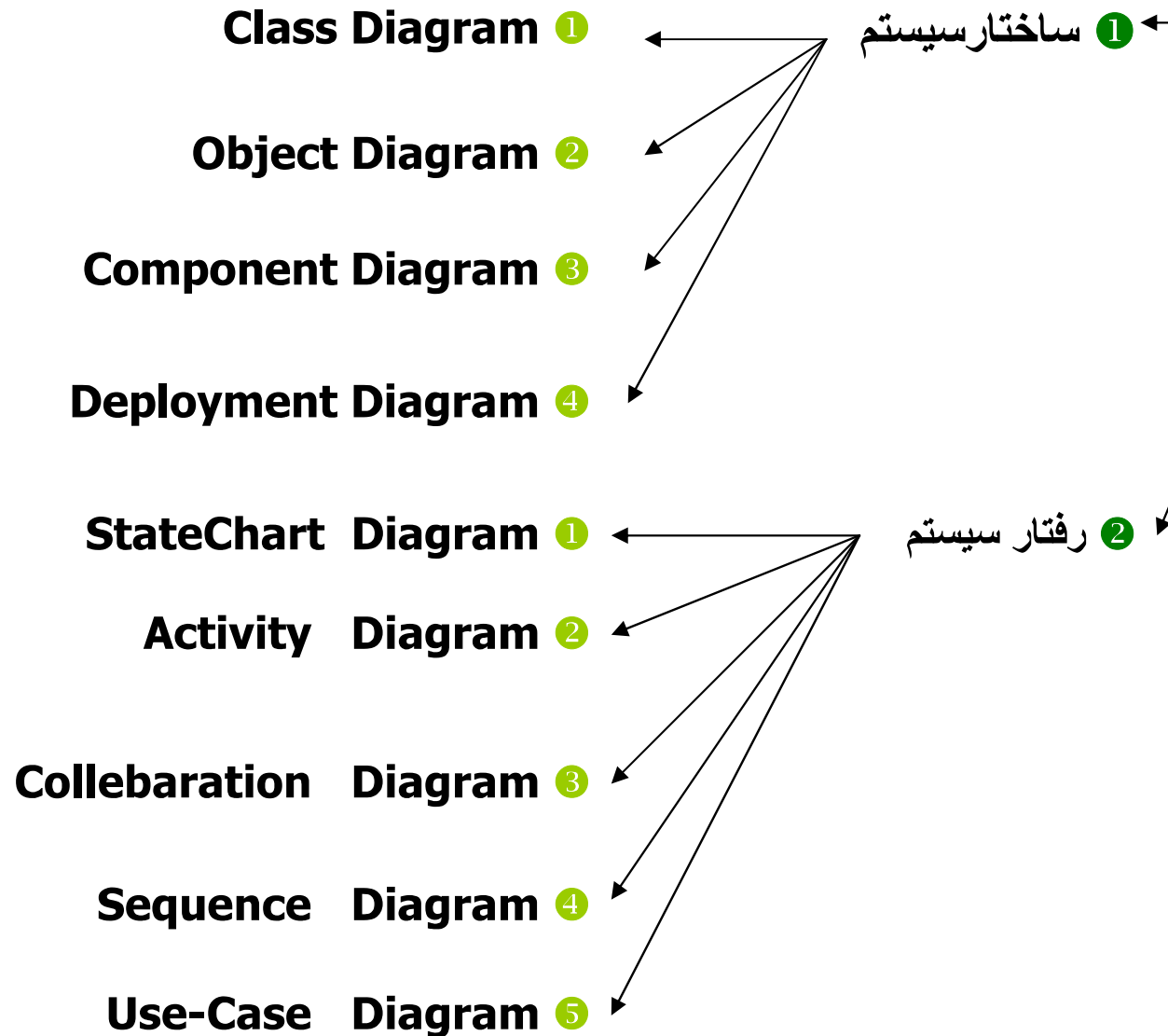
3 عمومیت دادن Genralization : رابطه بین Sub کلاس و Super کلاس ،
کلاس ها نوع خاصی از Super کلاس ها و Super کلاس ها نوع عامی از Sub کلاس هستند . رابطه
تعمیم مثل حالت وراثت میباشد بین شی پدر و شی
فرزند



4 به واقعیت در آوردن Realization : یک رابطه مفهومی میان تعریف کننده های کلاس
یک کلاس یک Interface را پیاده سازی و به واقعیت در می آورد .



3 نمودار : منظور از نمودار نمایش تصویر مجموعه ای از اجزا معمولاً بصورت یک گراف نمایش داده می شود لبه یا **Node** آن اشیا می باشند و خطوط ارتباطی روابط هستند .



② قواعد بین مولفه ها در UML

ما در مرحله قبل اجزا و مولفه ها را معرفی کردیم در این بین ما نیاز به قواعدی بین این اجزا داریم .

این قواعد در واقع مانند هر زبان دیگر **UML** نیز دارای یکسری قواعد بوده که اگر رعایت شوند مدل ما **خوش ترکیب** خواهد بود .

مدل خوش ترکیب : مدلی است که به لحاظ مفهومی خود سازگار (بین اجزا داخلی با هم سازگارند) یعنی دارای تناقض داخلی نباشد و هماهنگ است با بقیه مدل‌های مرتبط



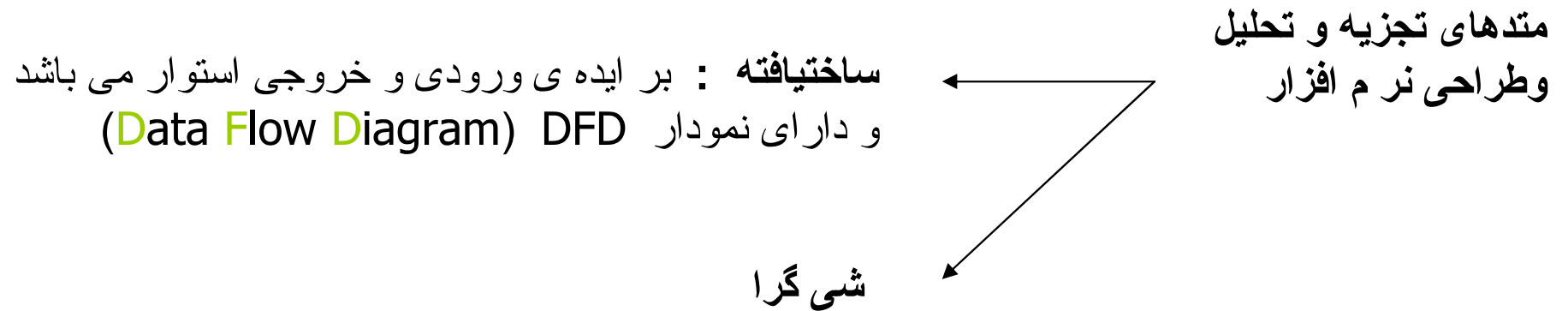
- Name:** آنهایی است که شما می توانید برای اشیا روابط و دیاگرام ها بنامید.
- Visibility:** قابلیت دسترسی يك سري قاعده می باشد.
- Integery:** اگر يك شي از کلاس دیگر می سازیم باید آن کلاس وجود داشته باشد.
- Execution:** ناظر بر این که يك مدل با چه نحوي ما تواند باشد تا اجرا شود.

آیا همیشه نمودارهایمان یک مدل خوش ترکیب اند یا خیر ؟

سه حالت امکان دارد که این مدل خوش ترکیب نباشد :

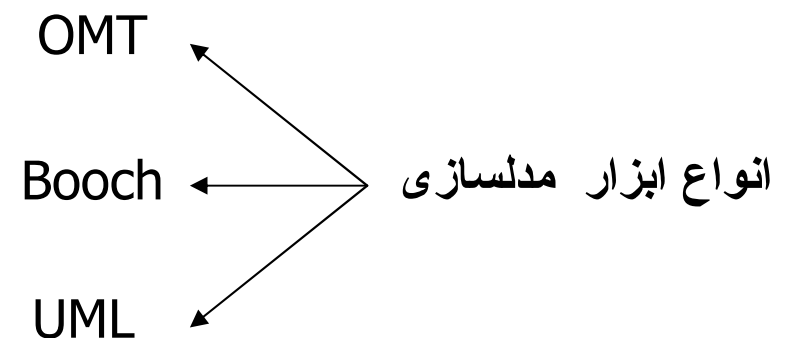
- ۱- Elided (مخفی)
- ۲- Imcomplete (ناقص)
- ۳- Inconsistent (اشکال)

نکته : برای این که از صورت مسئله تا اتمام پروژه مراحل را به خوبی طی کنیم می بایست طبق **حیات نرم افزار** عمل کنیم .



از دو متد فوق ما از **شی گرا** استفاده خواهیم کرد .

اکنون که می واهیم از شی گرا استفاده کنیم به یکسری نیازمندیها از قبیل یک زبان برنامه نویسی و یک ابزار مدلسازی **Modeling** نیاز داریم .



که از سه ابزار ذکر شده ما **UML** که به یک استاندارد تبدیل شده برای مدلسازی پروژه انتخاب می کنیم.

نکته : در مدلسازی ما بیشتر روی دو قسمت اول چرخه حیات نرم افزار کار خواهیم کرد .

دو روش استفاده از **UML** :
۱- ابزار کاغذ و قلم
۲- نرم افزاری

ابزار نرم افزاری که برای **UML** وجود دارد : **CASE**
(**Computer Adder Software Engeeniring**)

CASE : قادر به طراحی دیاگرامها تبدیل به زبان برنامه نویسی و یا حتی از یک ابزار به ابزار دیگر می باشد و اشکالات موجود را به ما نشان خواهد داد.

انواع نرم افزار CASE ← Rational Rose : نرم افزاری که با هر سه ابزار مدلسازی سازگاری دارد.

Together : نرم افزاری که با مدل UML و زبان Java سازگاری دارد .

UML

Unified Modeling Language

یک زبان گرافیکی برای طراحان نرم افزار به جهت طراحی و یا توسعه سریع نرم افزار می باشد .

شرکت OMG سازنده UML برای جزوات و مقالات آموزشی دارای سایت :

WWW.OMG.ORG

ویژگی های UML :

- ۱- نمای گرافیکی
- ۲- قابل بسط و توسعه
- ۳- امکان مدل کردن سیستم های شی گرا

Design Patterns : یک سری الگو ها است که از قبل ساخته شده که برای طراحی بهتر است بر حسب نیاز استفاده نمائیم مانند نقشه ی خانه برای معمار و دوم اینکه مستقل از زبان های برنامه نویسی طراحی را انجام می دهد .

ویژگی الگو ها :

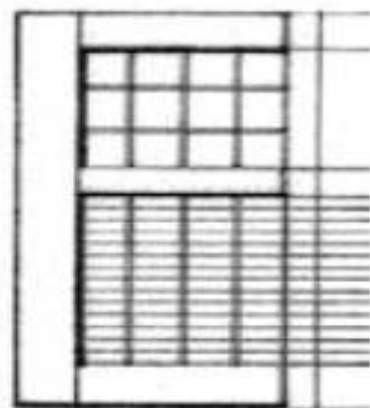
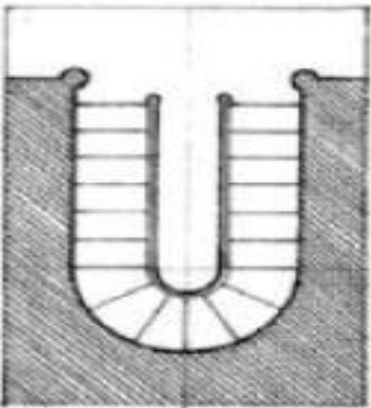
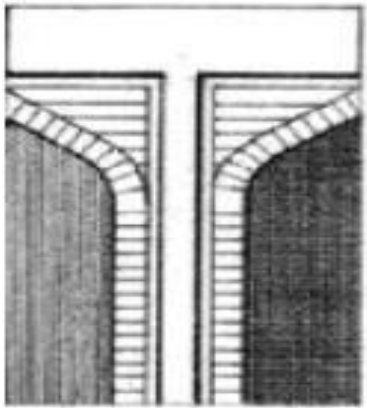
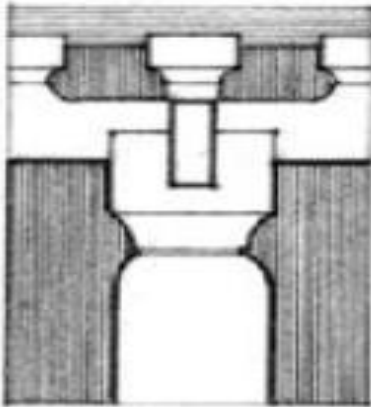
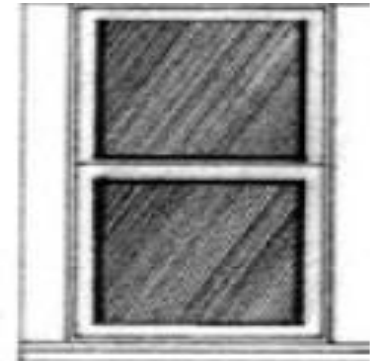
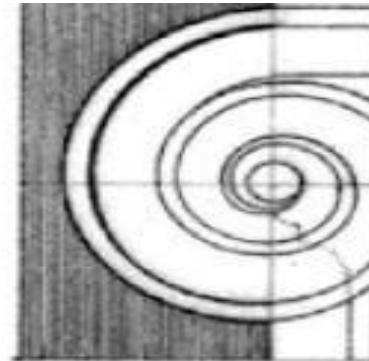
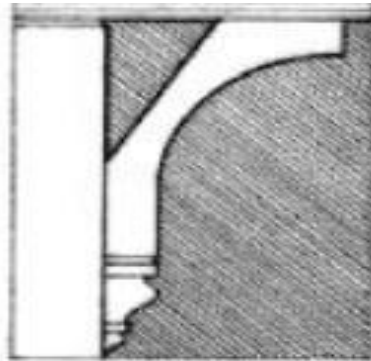
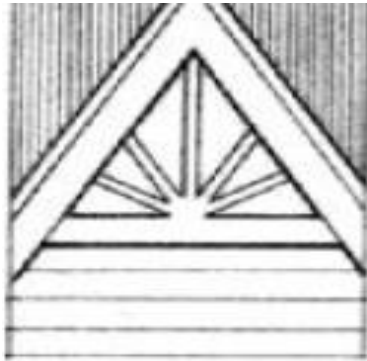
- معماریهای اثبات شده برای طراحی نرم افزار
- کاهش فرآیند پیچیدگی
- نشان دادن اشکالات موجود در طرح
- لغت نامه مشترک در طراحی
- ...

انواع نمودار (دیاگرام ها) موجود در UML :

به دو دسته کلی تقسیم بندی می شوند

- ۱- ساختار سیستم (ساختاری)
- ۲- رفتار سیستم (رفتاری)

Part I: Getting Started



فصل اول

مدل

Why We Model ?

تاریخچه UML

از دهه ی ۷۰ تا ۸۰ مدل‌سازی بوجود آمد و در ۸۰ تا ۹۴ حدود ۴۰ مدل‌سازی مختلف در دنیا ایجاد شد که این مدل‌ها متناسب با نیاز ها می بود سه تا از معروف ترین این مدل‌ها ، مدل‌سازی Booch و OOSE و OMT که هر کدام در یک قسمت قوی بودند که در نهایت با پیوستن هر سه بهم مدل‌سازی UML ارائه شد و چیزی که مد نظر ماست درمدل‌سازی UML نقطه قوت Booch می باشد .

هر سه نفر برای ایجاد UML اهدافی از قبیل زیر دنبال می کردند :

- (۱) ایجاد ابزار مدلسازی برنامه نویسی شی گرا
- (۲) قابل توسعه بودن
- (۳) ایجاد مدلی ساده به نحوه ای که قابل فهم انسانها و ماشین ها باشد .

اولین نسخه ی UML که به نام UM بود در سال ۱۹۹۵ بوجود آمد که خود UML در نسخه های بعدی ایجاد شد .

شرکت OMG با دادن پیشنهادی به این گروه سه نفره و با پذیرفتن آنها در سال ۱۹۹۷ نسخه ی UML 1.0 ایجاد شد و این کتاب در مورد UML نسخه ۱,۳ می باشد .

چرا ما مدلسازی نیاز داریم ؟

با یک مثال ساده این مسئله را توضیح خواهیم داد به فرض اگر بخواهیم برای سگ مان یک کلبه بسازیم و در نهایت روز به روز بدون طرح و برنامه ی اساسی بخواهیم این کلبه را توسعه دهیم تا اینکه به یک آپارتمان تبدیل شود مسلما در نهایت این بنا ساخته شده خراب خواهد شد .

در مورد نرم افزار نیز به همین شکل است اگر بدون مدل و طرح اولیه بخواهیم نرم افزار ساده ی تولید شده را توسعه دهیم و در نهایت این نرم افزار به جایی خواهید رسید که درون خود خراب خواهد شد .

بنابراین باید یک مدل از پیش تعیین شده ای برای آن وجود داشته باشد که بتوانیم باصرف کمتر اتلاف وقت مدل خود را توسعه داده که در آینده برای آن مشکلی پیش نیاید .

مدل چیست؟

A model is a simplification of reality.

مدل يك ساده سازي از واقعيت است.
جدا سازي يك جزييات بي تاثير مسئله و تاكيد زياد روي مسائلي كه تاثير زيادي روي برنامه ما دارند.

✓ هدف از مدل سازي :
مدل ها را انجام مي دهيم تا به يك درك و فهم بهتر از سيستمي كه با ان كار مي كنيم برسيم.

اهداف مدل سازي:

- ۱- به ما كمك مي كند تا ان سيستمي را كه مي خواهيم را به تصوير در آوريم.
- ۲- به ما اجازه مي دهد تا سيستم را به لحاظ رفتاري و ساختاري مشخص كنيم.
- ۳- يك الگوي است براي طراحي ساختمان سيستممان.
- ۴- تصميماتي كه در زمان طراحي و تجزيه و ساخت سيستم انجام مي گيرد را مستند مي كند.

اصول مناسب در مدل سازي:

- اصل ۱- مدلي که براي سيستم انتخاب مي شود بايد تاثير مستقيمي روي نتيجه جوابي که بدست مي ايد داشته باشد.
- اصل ۲- هر مدل مي تواند در سطوح مختلفي براي بيان جزييات باشد.
- اصل ۳- بهترين مدل ان مدلي است که منطبق و متصل با واقعيت باشد.
- اصل ۴- معمولاً با يك مدل به جواب نمي رسيم و بايد با همکاري و کمک چندين مدل به نتيجه رسيد .

✓ در سيستم هاي شي گرا به ۵ ديد نياز داريم:

- ۱- ديدني است مبتني بر نيازمندي هاي سيستم .
- ۲- فضاي طراحي و قابل حل را براي ما توصيف مي کند .
- ۳- فرايند ها را مدل سازي مي کند .
- ۴- در پياده سازي قطعات فزيکي به ما کمک و انها را مدل سازي مي کند .
- ۵- محيط ساخت و عملياتي را مدل سازي مي کند.

در تجزیه و تحلیل دو دیدگاه وجود دارد :

- ۱ - شی گرایي
- ۲ - ساخت یافته

فصل دوم

مقدمه UML

Introduction the UML

✓ UML زباني است براي:

۱ - به تصویر در آوردن Visualizing

۲ - مشخص کردن Specifying

۳ - ساختن Constructing

۴ - مستند کردن Documenting

UML زبان مدلسازی نرم افزاری برای نمایش این لغت نامه و قواعدی که برای در کنار هم قرار دادن این اجزا می باشد .

۱- به تصویر در آوردن (visualizing)

ایراد های موجود در این بحث در رابطه با عدم استفاده از UML شامل:

- ۱ - در ذهن خود مفاهیم و کدها را در نظر گرفته ایم. پس انتقال مفاهیم با استفاده از کد به ذهن دیگران نکته منفی و ایراد بدی است.
- ۲ - مفاهیم را باید غیر مستقیم استنتاج کرد و این ایراد دیگری است.
- ۳ - راه حلی که در ذهن می باشد به صورت کد تبدیل شده که بعد از مدتی همه داده ها به خاطر خلاصه شدن در قالب کد از ذهن فراموش شده و از یاد می رود.

در نتیجه ما به زبانی نیاز داریم که برای قابل درک و فهم کردن کد و به تصویر کشیدن آن کارما را راحتتر کند .

۲ - مشخص کردن:

سیستمی که با UML مشخص می کنیم باید :

۱- دقیق ۲- غیر مبهم ۳- کامل باشد.

این قسمت به دو دسته تقسیم می شود:

۱ - اشیا ساختاری

۲ - اشیا رفتاری

۳ - ساختن :

اگر UML ما در ساخت سیستم به ما کمک نکند دیگر UML به حساب نمی آید .
پس در اینجا ما یک Mapping داریم که به ما دو امکان را می دهد :

(۱) مهندسی رو به جلو

forward engineering : یعنی تولید کد از مدل ایجاد شده UML

(۲) مهندسی معکوس

Reverse engineering : یعنی از مدل به کد برسیم .

✓ UML آنقدر سریع و غیر مبهم است که سه امکان را به ما می دهد :

(۱) اجرای سیستم در مراحل طراحی و RUN

(۲) شبیه سازی سیستم

(۳) سیستم ساخته شده را از نظر کیفی اندازه گیری می کند.

۴ - مستند کردن :

باید دستور العمل ها و کد نویسی های خود را به صورت مکتوب نوشت و این کار UML می باشد.

✓ مسائل مکتوب (ویژگی مستند سازی) در UML شامل ۸ قسمت می باشد :

۱ - **Requirment** : نیاز ها را در آورده و در برنامه ضمیمه می کند .

۲ - **Architecture** : باید دیدی از معماری و طرح کلی سیستم را داشته باشیم .

۳ - **Design** : روابط و نیازها را در بعد طراحی مکتوب می کند .

۴ - **Source Code** : در کنار هر کد دستور العمل ها و **Help** ها را باید داشته باشیم .

۵ - **Project Planes** : دسته بندی کارها و اجرایی پروژه را انجام می دهیم .

۶ - **Tests** : سیستم از این تست ها کمک گرفته و انجام شده .

۷ - **Prototypes** : یک مقدمه و امکانات نیمه آماده روی سیستم و برنامه خود گذاشته و مکتوب می کند .

۸ - **Released** : نسخه هایی که از نرم افزار بیرون می آید را مکتوب می کند .

برای شناخت UML نیاز به شناخت سه مورد زیر داریم :

1 شناخت اجزا و مولفه های تشکیل دهنده Building Blocks

2 قواعد بین اجزا

3 مکانیزم کنترل زمان

توجه : تمامی زیر ساختار UML به صورت درختی سطح بندی شده توضیح داده شده است که به ترتیب سطح رنگ شماره گذاری تغییر می کند به این شکل که :

1 سطح اول

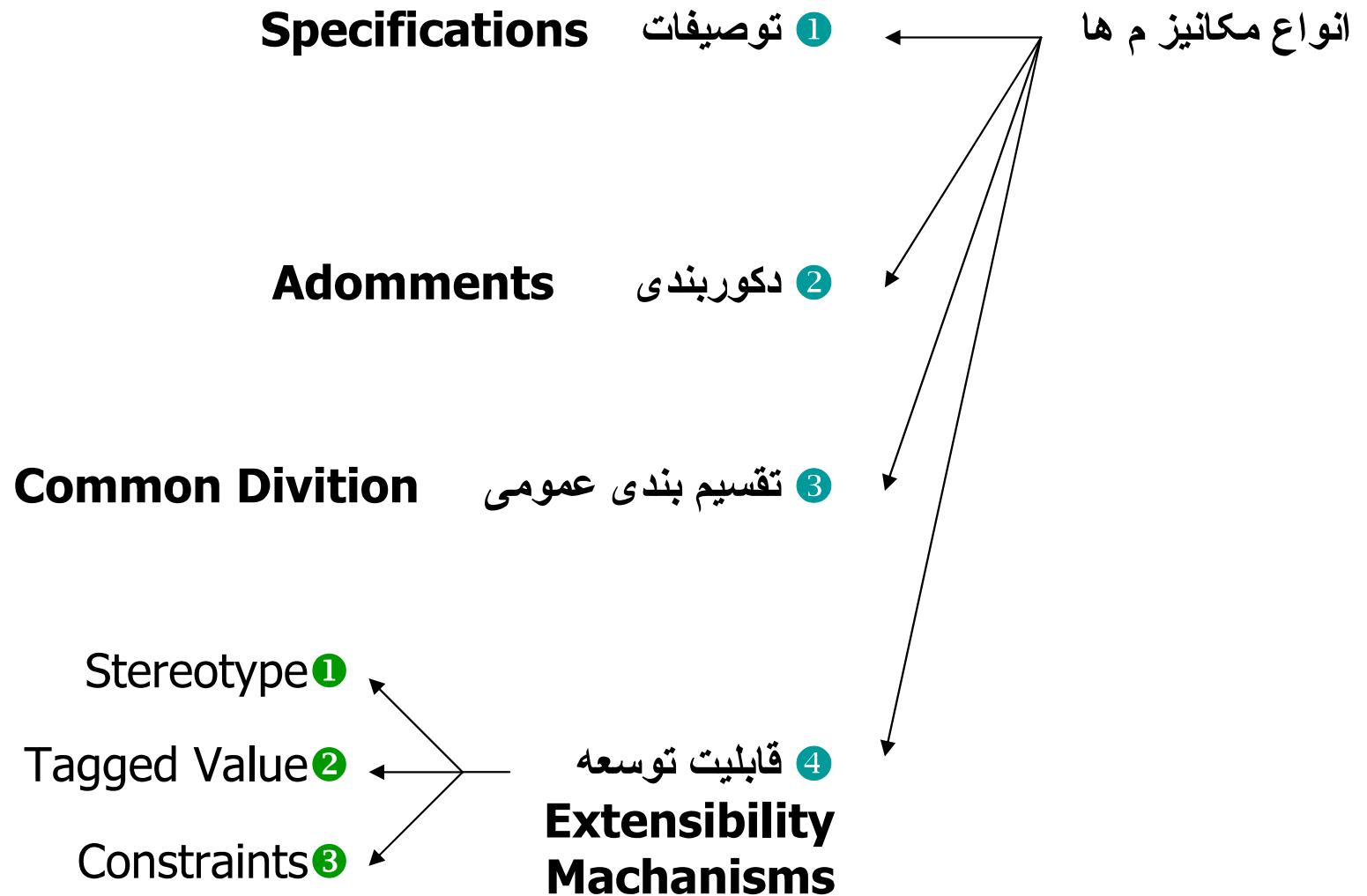
2 سطح دوم

3 سطح سوم

4 سطح چهارم

5 سطح پنجم

3 مکانیزم عمومی در UML



توصیفات : ممکن است در یک مدل رسم شده بعضی از اجزای مدل از دید ما مخفی باشند ولی در پس زمینه وجود دارند پس نیاز به جای داریم که این توصیفات و توضیحات از اینگونه قسمت های مخفی و غیر قابل دید ذکر شود .

موارد استفاده توصیفات :

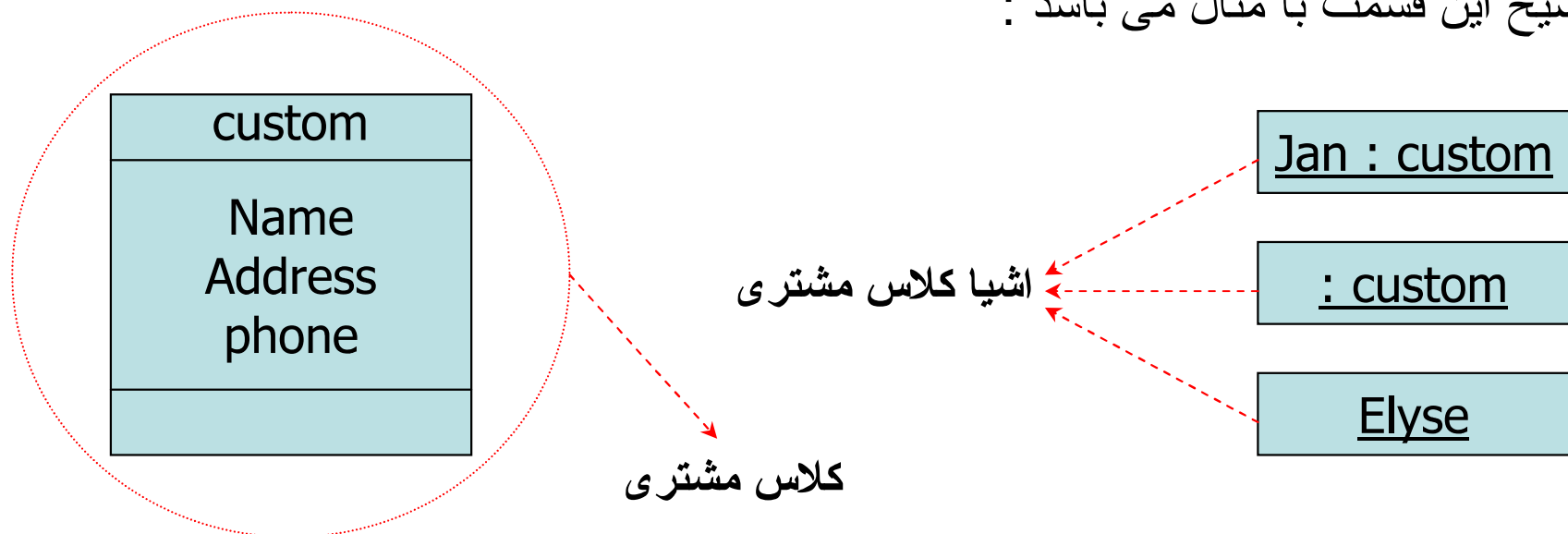
- تست خوش ترکیب بودن مدل

- تولید خود کار کد

دکور بندی : نمایش جزییات در مدل و اضافه کردن توصیفات گرافیکی و مستند سازی گرافیکی بر عهده دارد .

تقسیم بندی عمومی : ما می توانیم به وسیله ان فضاي خودمان را مدل سازي کنیم و اشيا را با توجه به اینکه جزچه کلاسي هستند دسته بندي کرد.

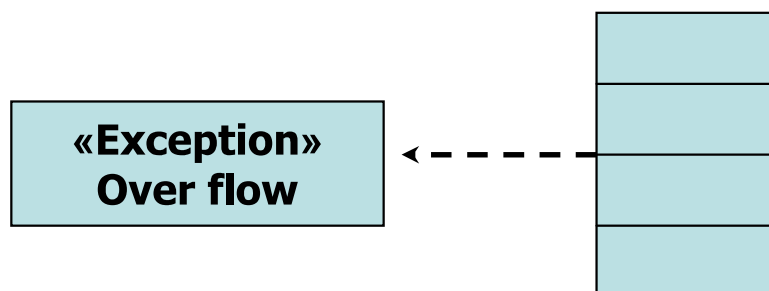
توضیح این قسمت با مثال می باشد :



قابلیت توسعه (مکانیزم توسعه) : مثال : خود UML قابل توسعه و بسط می باشد .

① Stereotype : توسعه دهنده ی لغت نامه UML

مثال شی Exception در جاوا تعریف نشده می توانیم از این طریق به مدل اضافه کنیم .



② Tagged Value : می توان خاصیت اجزا UML را

توسعه داد . مثال : ما در مولفه کلاس سه قسمت نام مشخصه و رفتار داریم از این طریق می توانیم خاصیت ورژن (version) به آن اضافه کنیم .

③ Constraints : محدودیت برای توسعه قواعد UML

انواع مکانیزم توسعه

معماری نرم افزار :

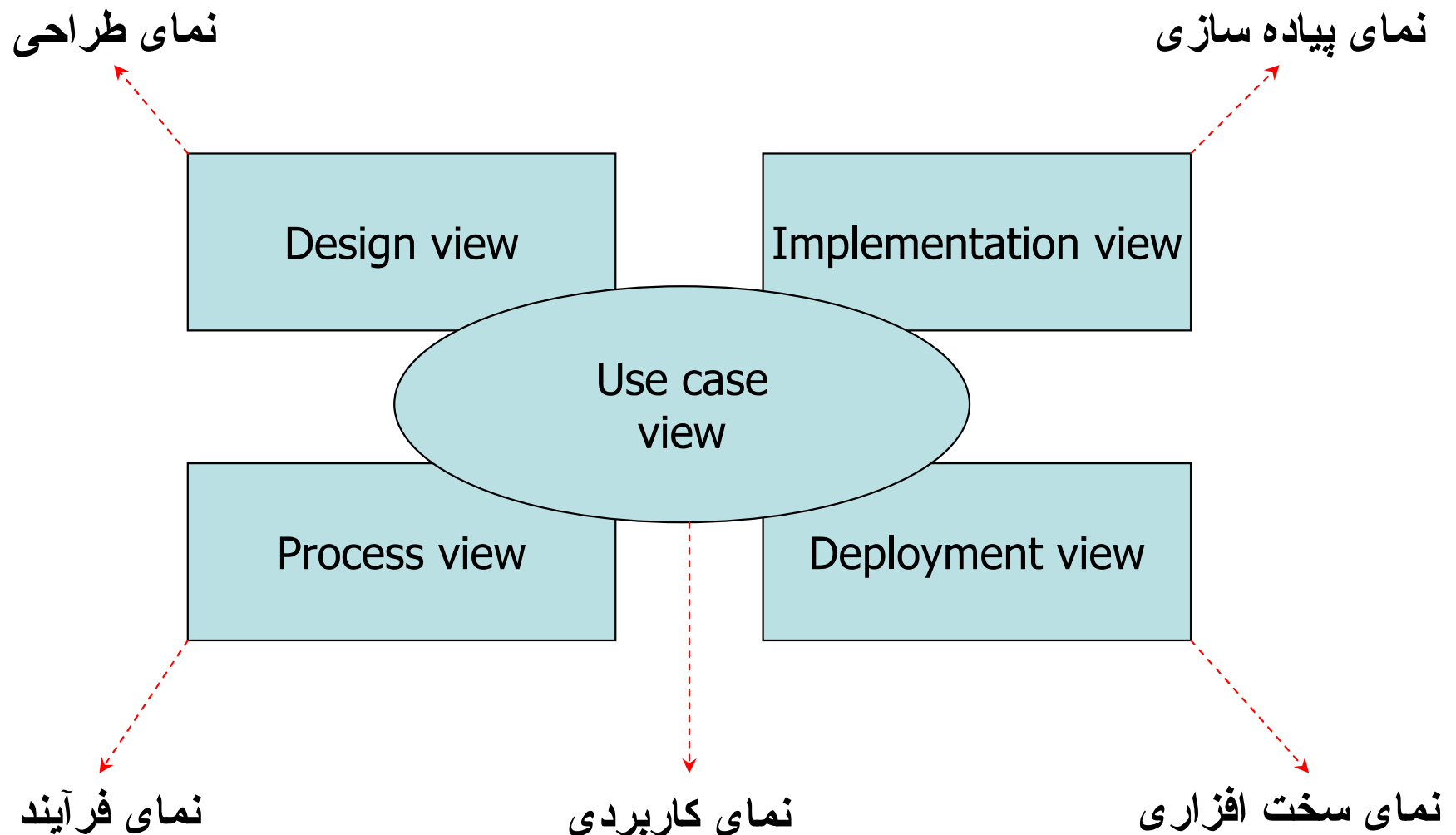
معماری جمع کردن مجموعه ای از دیدها برای يك طرح كلي از سیستم می باشد.

یک مجموعه ی تقسیمات موثر در این اقسام است :

- سازماندهی سیستم نرم افزار (چیدن اجزا کنار هم)
 - انتخاب اجزا ساختاری و روابطی که اینها بهم متصل میکند
 - رفتار اجزا که مشخص کننده ی همکاری بین اجزا است
 - ترکیب این عناصر رفتاری و ساختاری در زیر سیستم های بزرگتر
 - (دسته بندی و سازماندهی زیر سیستم ها در یک سیستم)
- الگویی که برای سازماندهی انتخاب شده است .

انواع دیدگاه نسبت به نرم افزار :

✓ ما پنج دید نسبت به سیستم داریم :



1 Use Case view : این دید کاربردی از سیستم است که این کاربردها توصیف کننده ی رفتارهای سیستم از نظر کاربر نهایی می باشد علاوه ر کابر نهایی تحلیل گر سیستم خود را باید بجای کاربران دیگر بگذارد .

2 Design view : در این دید که شامل مولفه های تشکیل دهنده ی لغت نامه ی صورت مسئله پروژه از قبیل کلاس ها رابط ها و همکاری ها و ... و راه حل های مسئله

3 Process view : این دید تشکیل دهنده ی فرآیندها و روش های سیستم می باشد و همزمان کردن Thread در این دید نمایش داده می شود (بحث هماهنگی یا Synchronization)

4 Implementation view : تشکیل دهنده ای اجزا و فایل های است که از آنها برای ترکیب استفاده می شود .

5 Deployment view : دید سخت افزاری از سیستم می باشد . که مشخص می کند ساختار سخت افزاری که این سیستم روی ان قرار می گیرد به چه صورت می باشد و از چه قطعاتی تشکیل شده است .

چرخه حیات توسعه نرم افزار

فرآیند طراحی از **UML** جداست .

پرکاربرد ترین نرم افزاری که برای توسعه وجود دارد :

RUP (Retional Unified Process)

UML مستقل از تولید نرم افزار بیشترین فرآیندهایی که می توانند از **UML** استفاده کنند :

Use Case - گرایش به کاربرد

Architecture-Centvic - معماری محور

Iterative And ineremental - چرخشی و افزایشی

منظور از گرایش به کاربرد : استفاده از **Use Case** بعنوان ابزار اساسی از بنا کردن رفتار سیستم خواسته شده برای تست و اعتبار سنجی سیستم و ایجاد ارتباط بین اجزای (اعضا) پروژه

منظور از معماری محور بودن : یعنی از معماری سیستم استفاده شود برای ایجاد مفاهیم ساخت و مدیریت و توسعه سیستم در دست ساخت .

منظور از فرآیند تکراری و چرخشی و افزایشی : اینکه بتوان استفاده دوباره از سیستم و توسعه آن .

يعني فرايند بتواند يك جرياني از نسخه هاي قابل اجراي برنامه را توليد كند كه هر نسخه نسبت به نسخه قبلي يك سري امكانات افزوده داشته باشد و نيازمندي هاي بيشتري را پوشش دهد.

مزيت تكراري و چرخشي و افزايشي ريسك گرا بودن نسبت آبخاري .

چنين فرايندي يك فرايند **فازبندي** شده است .

فاز : محدوده ي زماني ميان دو قسمت اصلي از فرايند مي باشد وقتي يك مجموعه خوش تعريف از اهداف بدست مي آيد و مجموعه اي از كار ها تكميل مي شود و يكسري تصميمات اتخاذ مي شود كه آيا امكان انتقال به فاز بعدي هستيم يا خير .يا يك مجموعه خوش تركيب مي باشد .

در RUP فرايند را به ۴ فاز تقسيم بندي مي كنند :

۱- آغازی Inception

۲- شناخت Elaboration

۳- ساخت Constraction

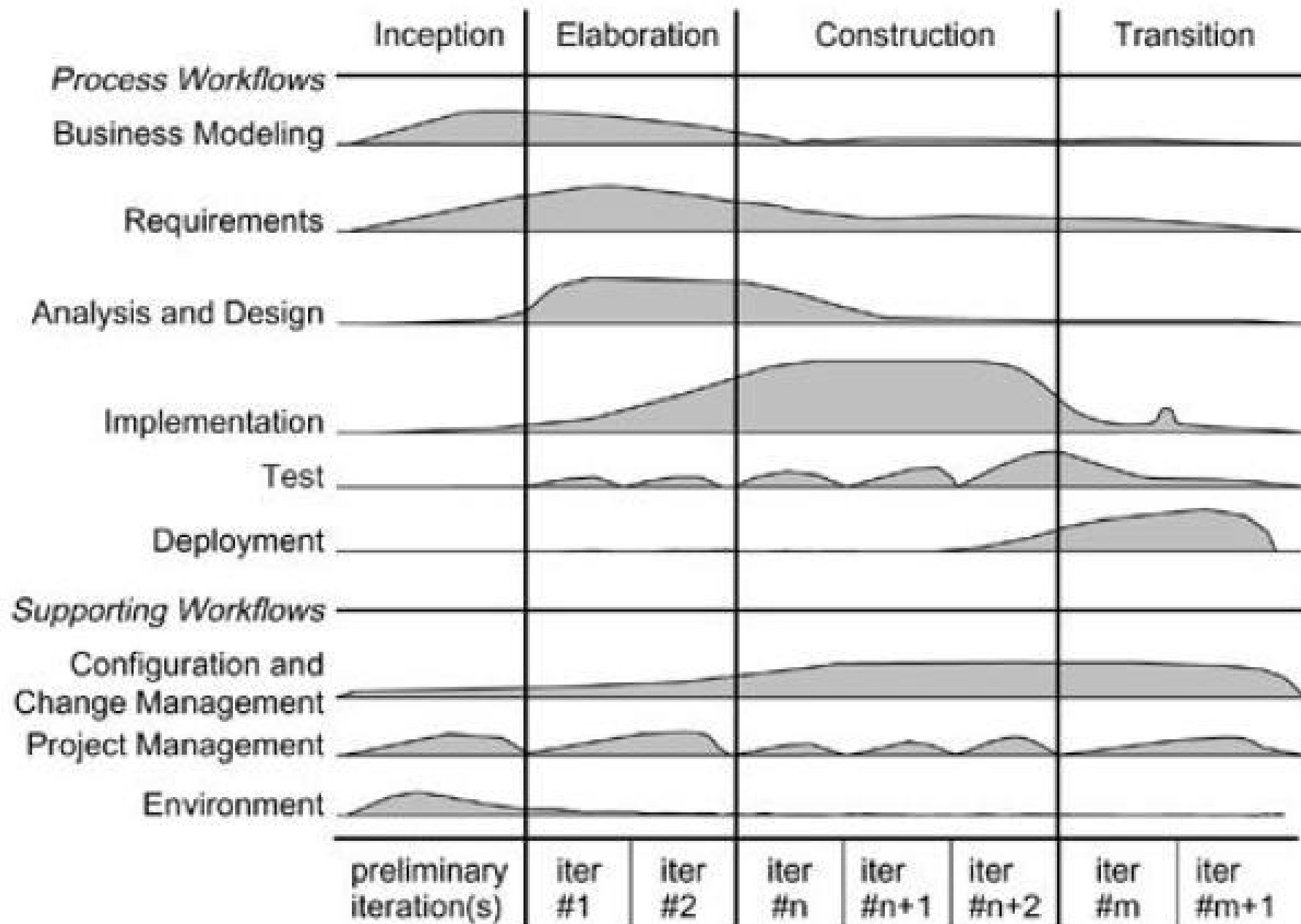
۴- انتقال Transition

آغازی

شناخت

ساخت

انتقال



● اولین فاز فرآیند (آغازین) Inception

اولین جرقه ی پروژه زده می شود و پروژه می رود که شکل بگیرد بررسی می شود که این پروژه شدنی است یا امکان آن وجود ندارد .

● دومین فاز فرآیند (شناخت) Elaboration

در این فاز دید از محصول و دید از نرم افزار کامل شده و بر اساس آن معماری شکل می گیرید ولیست نیارمندیها در می آید و اولویت بندی و برنامه ریزی می شود (نیازمندی کلی یا جزئی) هر کدام از این نیازمندیها مشخص کننده یک رفتار کار کردی یا یر کار کردی سیستم می باشد .

● سومین فرآیند (ساخت) Constraction

در این مرحله معماری بوجود آمده به اجرا مرحله ساخت برسد . در این قسمت مدیریت پروژه بسیار مهم است .

● چهارمین فاز فرآیند (انتقال) Transition

فازی که نرم افزار به دست کاربر نهایی می رسد . در این قسمت تست انجام می شود و همچنین Deployment نیز صورت می گیرد .

یک چرخشی : یک مجموعه مجزا از فعالیتهاست که با یک برنامه از پیش تعیین شده و یکسری معماری های ارزش یابی است که نتیجه آن نسخه ی جدید از نرم افزار می باشد .

Interaction : مجموعه اي از فعاليت ها در يك برنامه مدون و يك سري معيار هاي ارزيابي كه
نتيجه اش به صورت يك سري نسخه هاي نرم افزاري نشان داده مي شود كه اين نسخه ها مي توانند
براي يك شركت كامپيوتري و يا براي يك كاربر كامپيوتري باشد.

فصل سوم

مثال

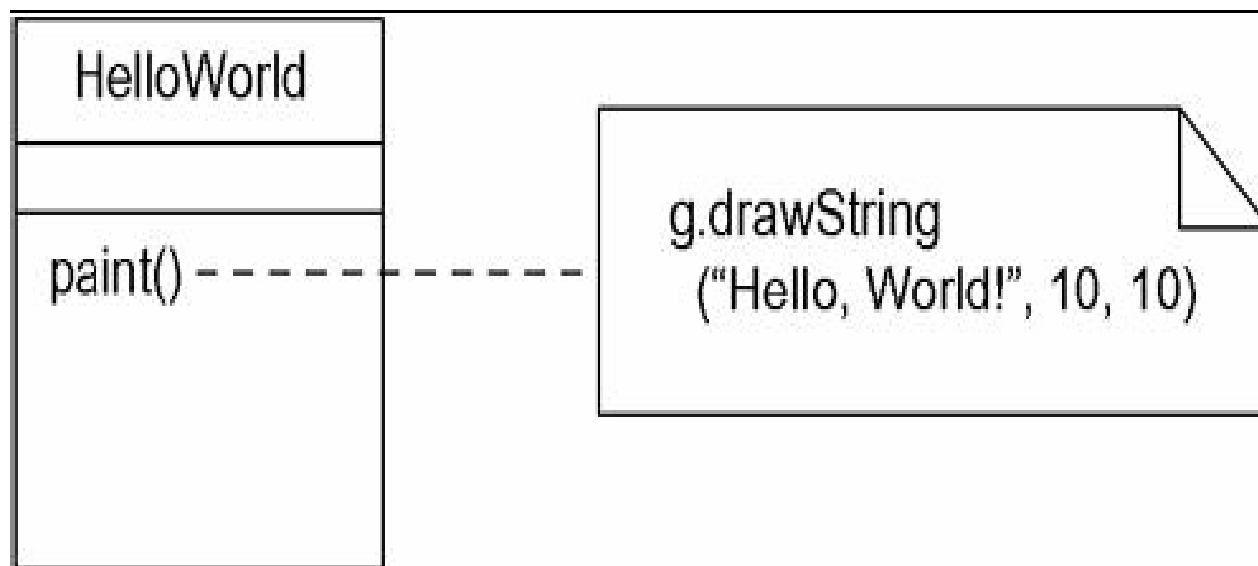
Hello, World!

چکیده های کلیدی

در این مثال می خواهیم قطعه کدی به جاوا بنویسیم که:
کد اپلتی بنویسد که در مرورگر وب ای جمله را چاپ کند :

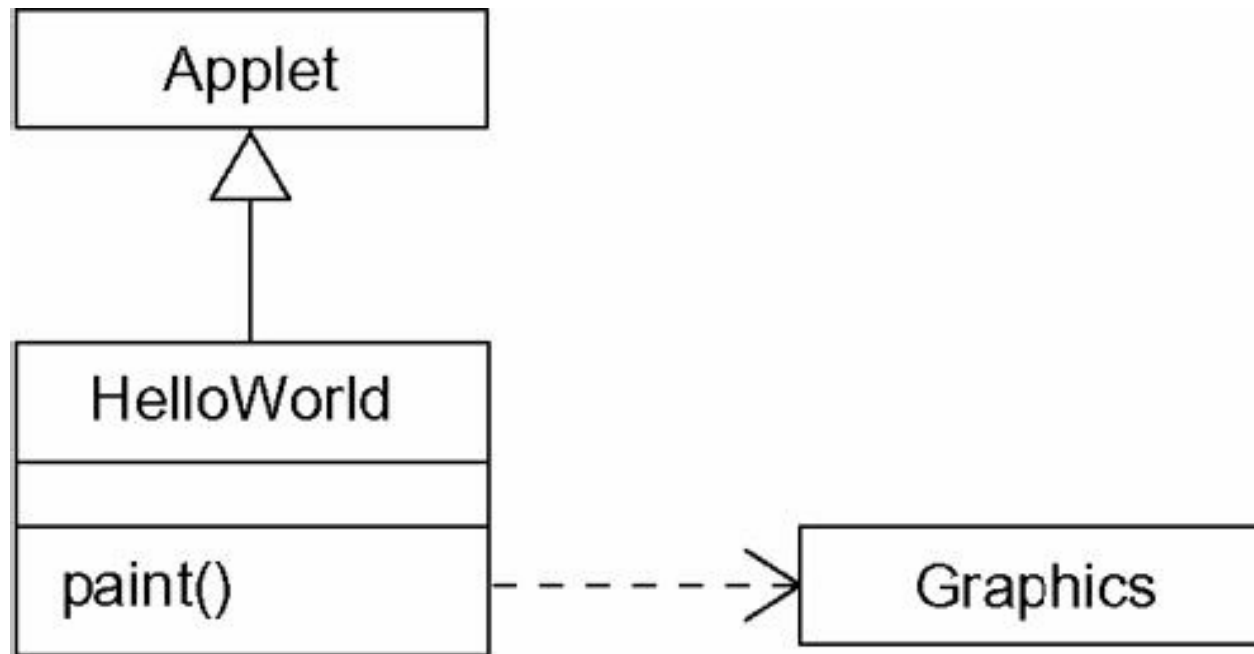
```
import java.awt.Graphics;  
class HelloWorld extends java.applet.Applet {  
    public void paint (Graphics g) {  
        g.drawString("Hello, World!", 10, 10);  
    }  
}
```

اکنون برنامه ای که نوشته ایم می خواهیم در **UML** مدل کنیم :



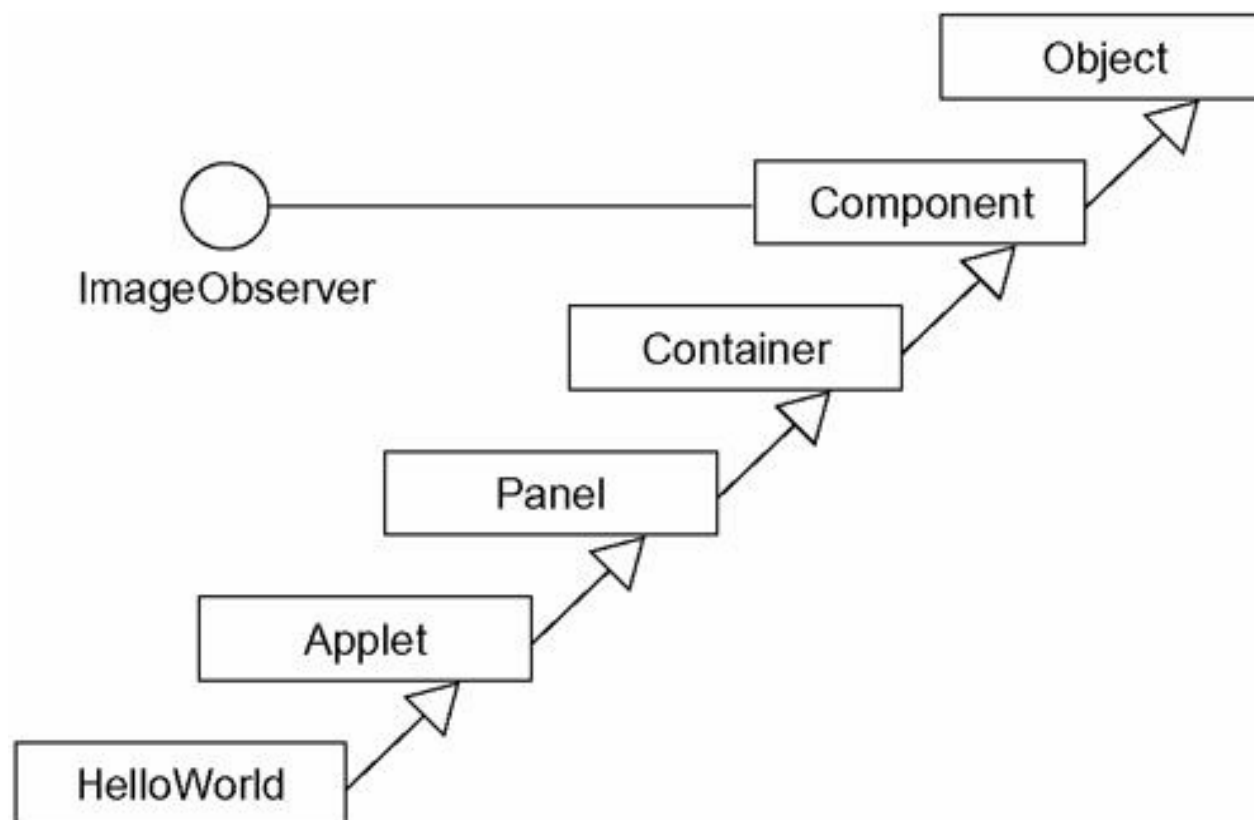
در این مدل ما یک کلاس داریم به نام **HelloWorld** و دارای یک متد به نام **Paint()** و یک مولفه توضیح داریم که به متد کلاس مرتبط است و نشان می دهد که این متد متن مورد نظر را چاپ می کند .

اکنون ارتباط ها را مدل می کنیم :



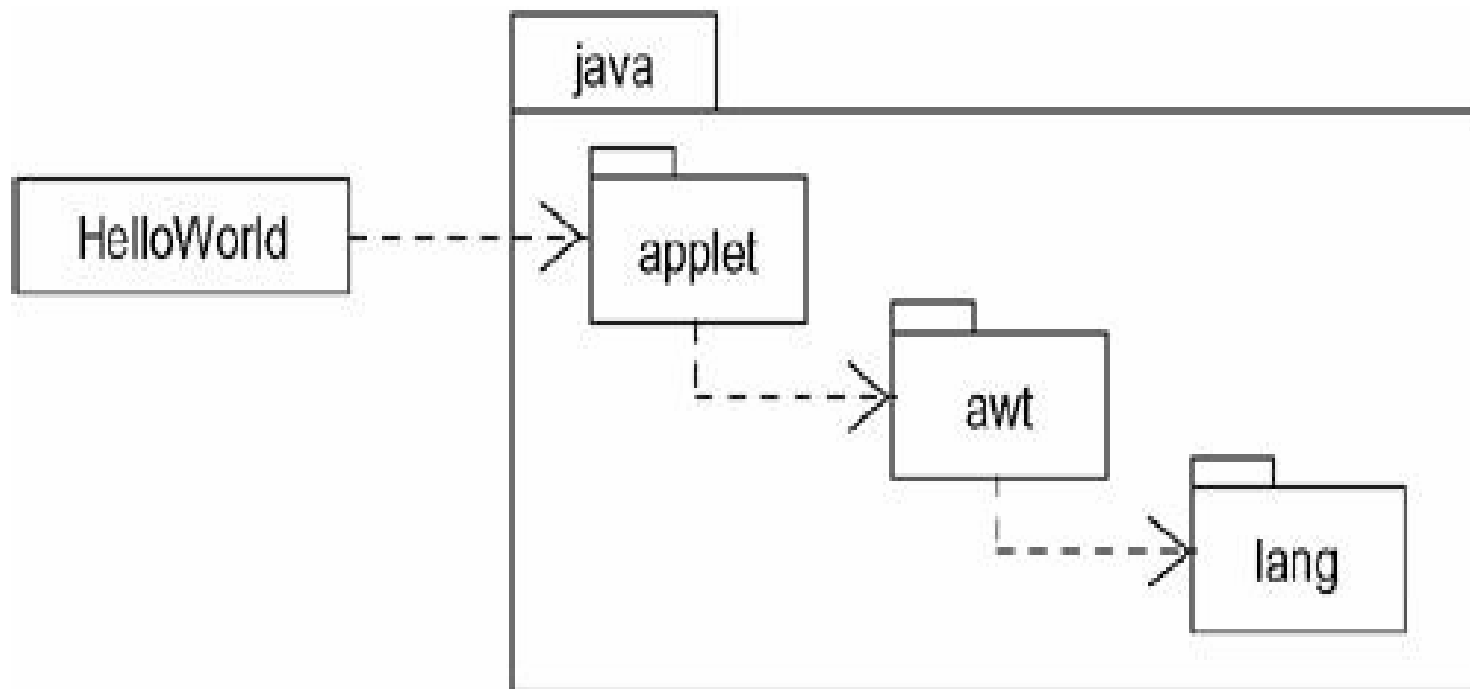
همان طور که در شکل فوق می بینید که کلاس HelloWorld به دو کلاس Applet و Graphics ارتباط دارد ولی نوع ارتباط آنها فرق دارد از یک طرف کلاس HelloWorld فرزند کلاس Applet می باشد و از طرفی دیگر کلاس ما با کلاس Graphics رابطه ی وابستگی وجود دارد. به این معنی که کلاس HelloWorld از کلاس Graphics استفاده می کند .

همان طور که در جاوا خوانده ایم در رابطه کتابخانه کلاسها و وراثت بین آنها می خواهیم این وراثت را مدل کنیم در حینی که ببینیم کلاس HelloWorld چه جایگاهی دارد به شکل توجه کنید :



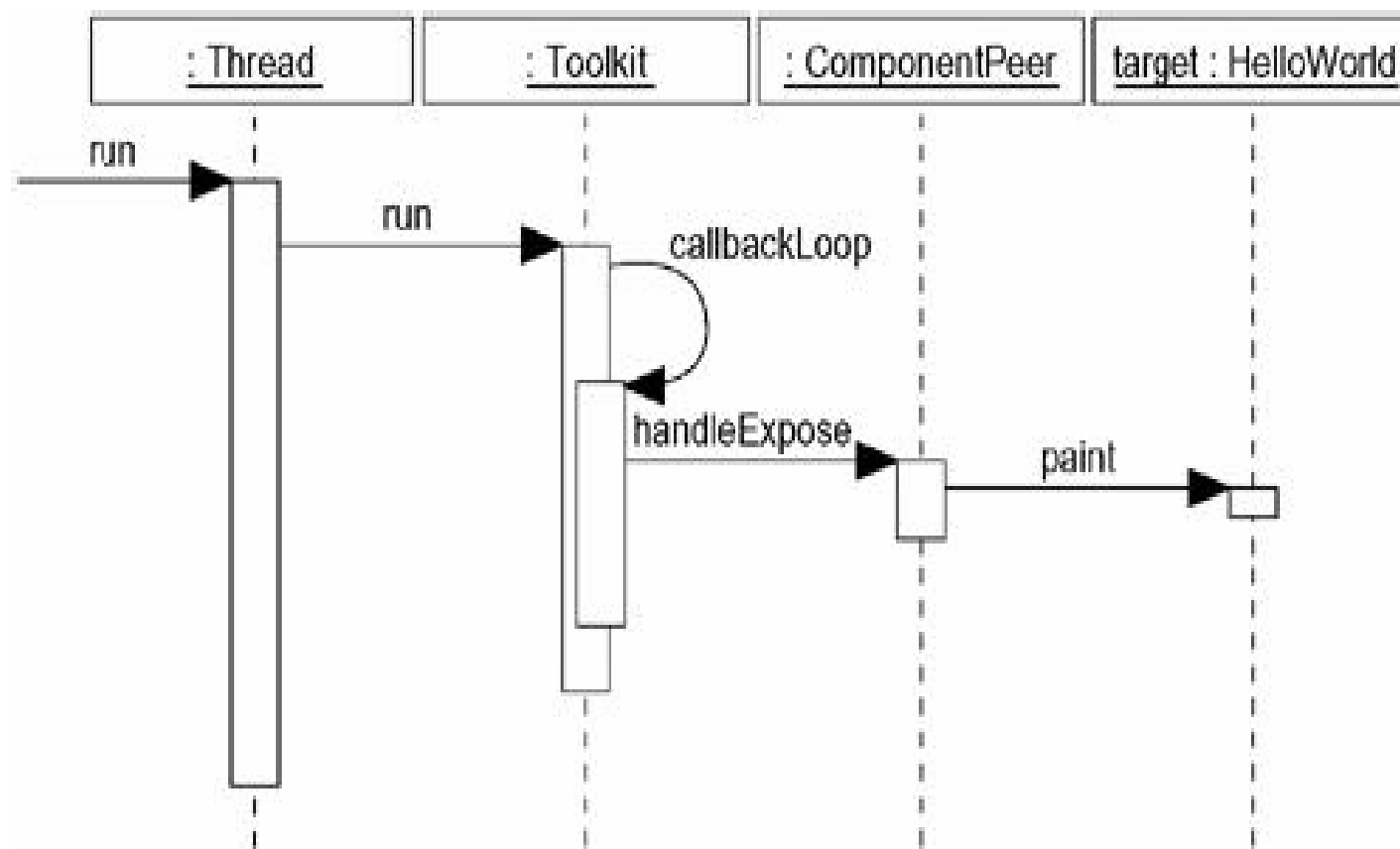
همانطور که مشاهده می کنیم کلاس HelloWorld فرزند Applet و Applet فرزند Panel و panel فرزند Container و Container فرزند Component که خود پیاده سازی Interface ImageObserver می باشد و از طرفی Component فرزند کلاس پدر Object می باشد .

شما قادر هستید بسته بندی **Packaging** به تصویر بکشید در کلاس دیاگرام :

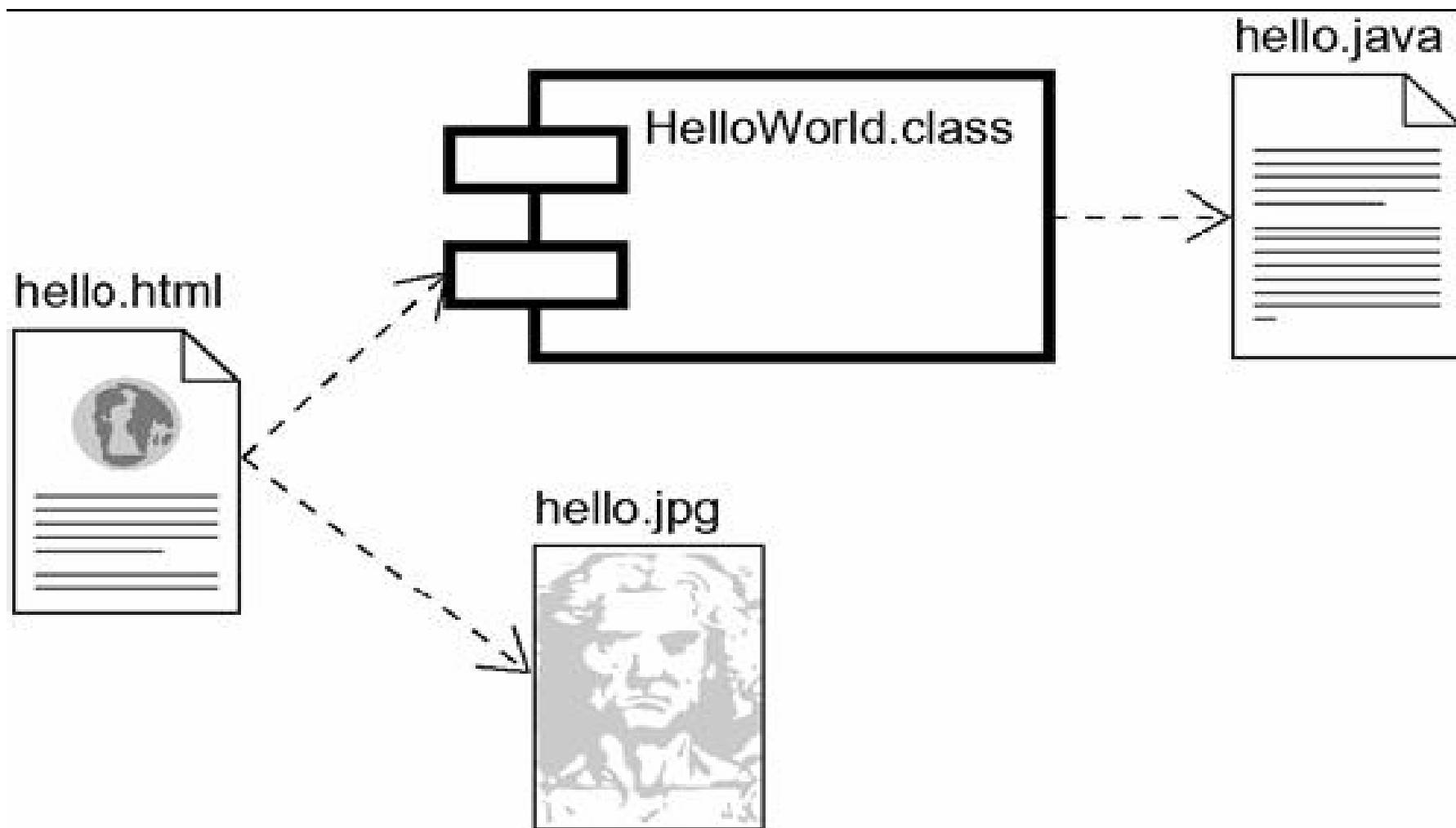


آنچه را که می بینیم کلاس `HelloWorld` به `package Applet` وابستگی دارد و از طرفی `Package Applet` خود نیز بعنوان زیر مجموعه ای از `Package Java` می باشد و از طرفی دیگر خود `Package Applet` به `package` های `awt` و `lang` وابسته است.

مکانیزم ها



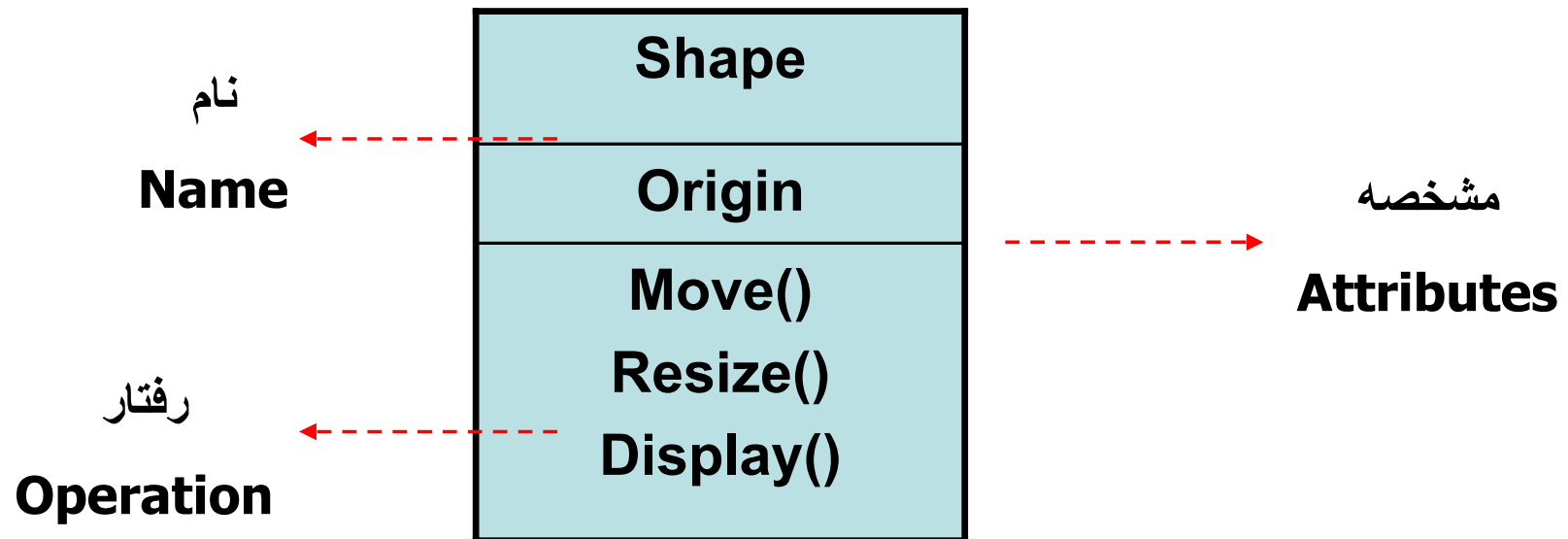
این یک Sequence دیاگرام را نشان می دهد وارد جزئیات نمی شویم فقط بطور کلی یک مکانیزم چاپ را شرح خواهیم داد در این دیاگرام ما چهار شی داریم که یکی پس از دیگری اجرا شده و این مستطیل های ایستاده نشان دهنده بازه زمانی که یک شی فعالیت می کند .



کلاس چیست؟

مولفه اصلي سازنده سيستم هاست. نشان دهنده مجموعه اي از **object** ها که از نظر **Operation** رفتار مفهوم و غيره مشابه هستند که به جاي آن يك کلاس توليد مي شود.

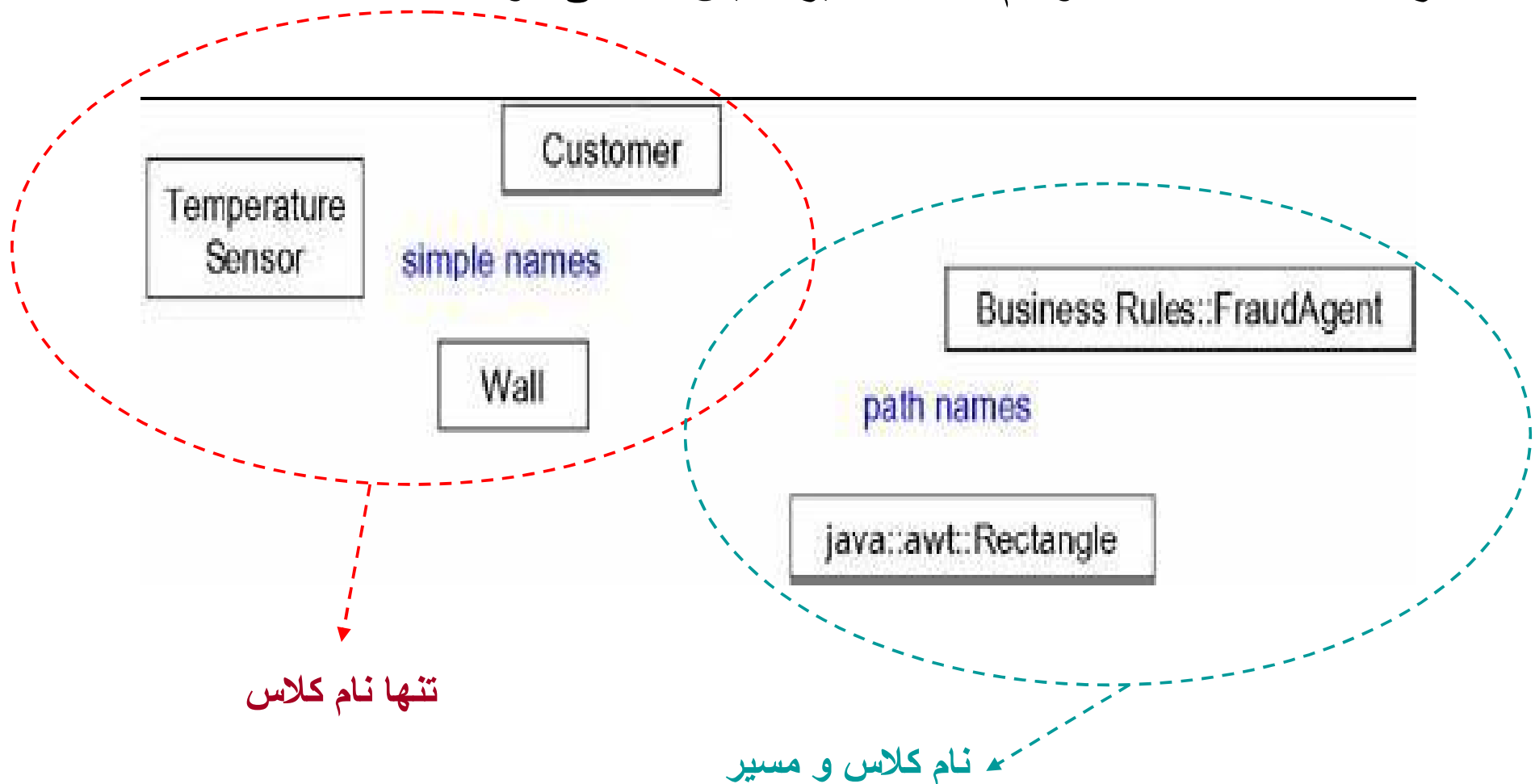
مهمترين اجزاي سيستم هاي شي گراست. اين کلاس توصيف مجموعه اي از اشيا است که داراي نام ، انجام عمليات و مشخصه ها ارتباطات و مفهوم مي باشد.



کلاس شامل اجزا زیر می باشد :

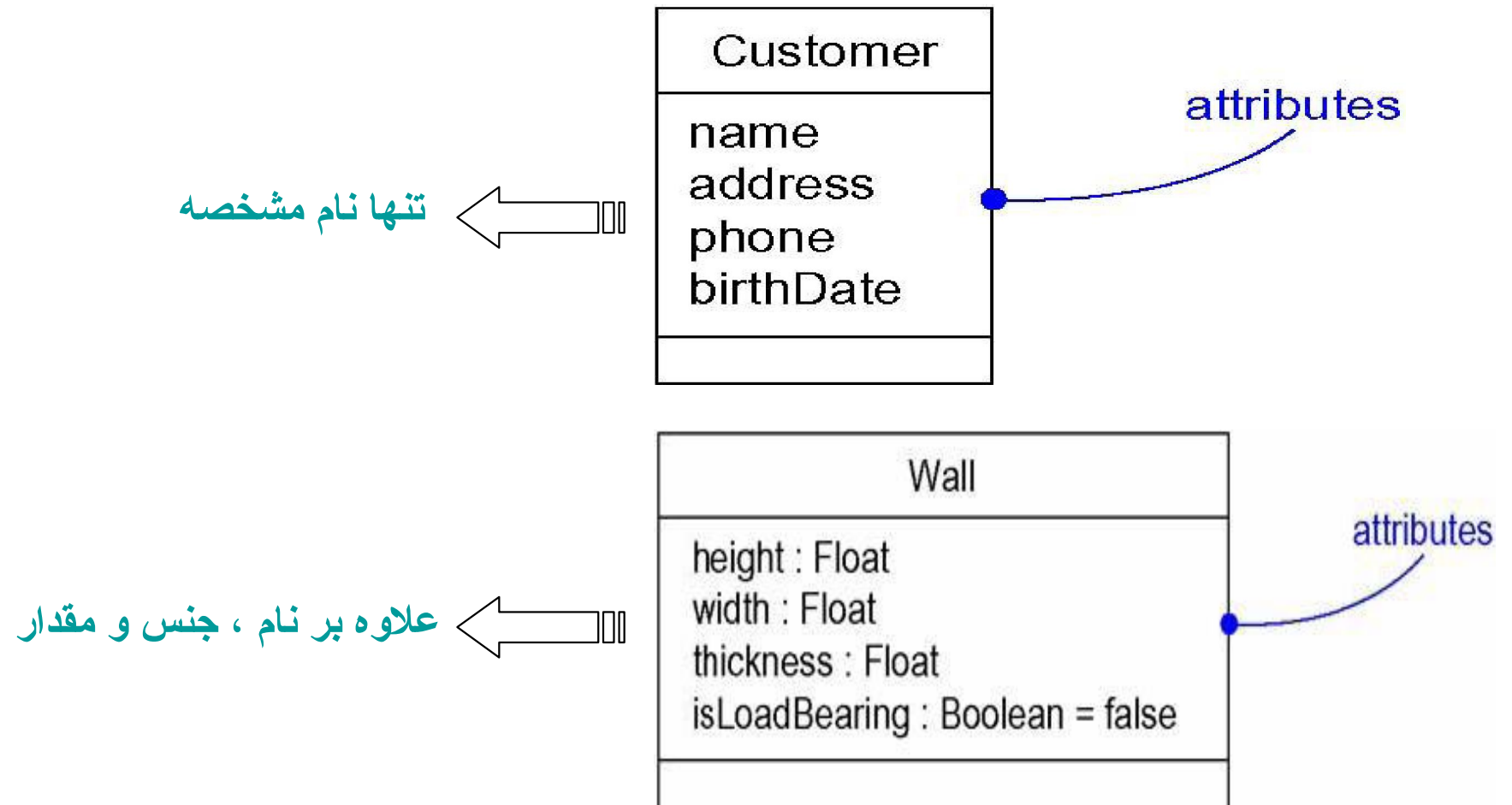
- 1 **Name نام :** نام کلاس از یکسری حروف و ارقام و علائم تشکیل شده که به حروف شروع شده و این حروف قاعدتا با حرف بزرگ و بقیه کوچک می باشد .
نام کلاس که در Package تعریف می شود باید منحصر به فرد باشد .

که در UML به شکل ساده و نام های با مسیر نمایش داده می شوند :



Attribute 2 مشخصه ها

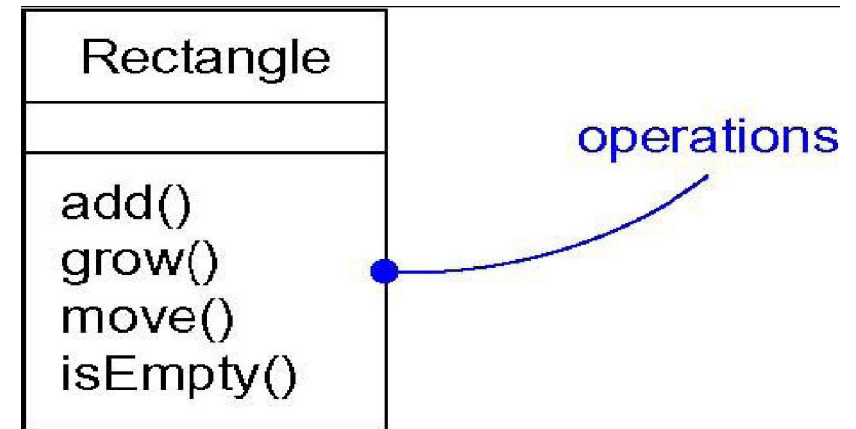
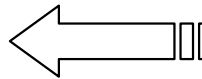
مشخصه های یک کلاس می تواند علاوه بر خود مشخصه جنس یا نوع و مقدار نیز در آن مشخص شود.



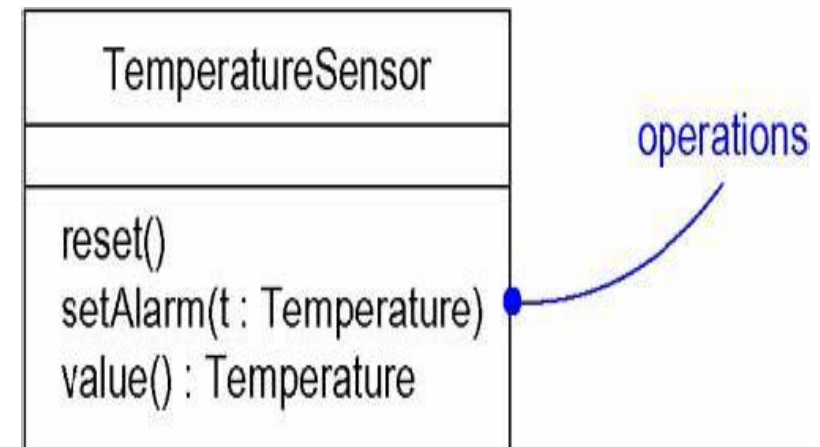
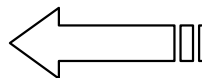
3 Operation عملیات

(رفتارهای کلاس) در این جا می توان از اسم متد به اضافه لیست پارامتر ها یا مقدار برگشتی استفاده کنیم.

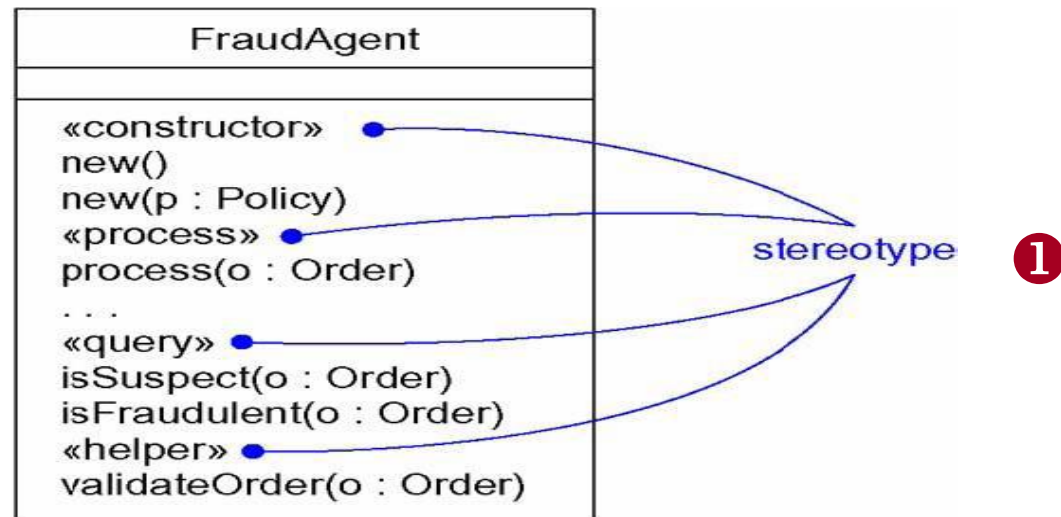
بدون پارامتر



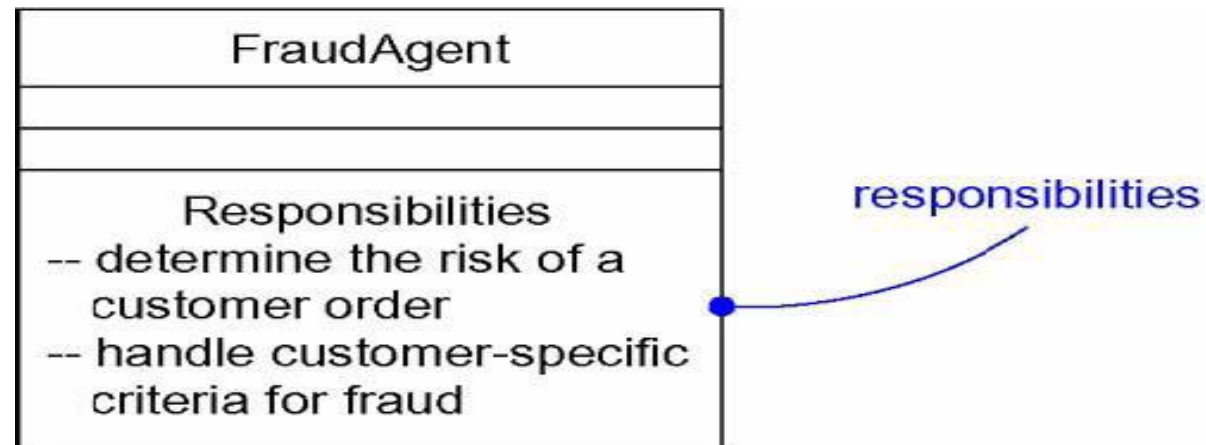
با پارامتر دریافتی و مقدار برگشتی



دو روشی که می توان یک کلاس را از لحاظ دسته بندی توسعه داد :



2 می توان یک کلاس را به چهار قسمت تقسیم کرد که **مسئولیت** در قسمت چهارم است. **مسئولیت** یا **Responsibility** هیچ تاثیر مفهومی در خود کلاس ندارد و مثل يك **Comment** می باشد و فقط يك یادداشت و **Node** در کلاس می نویسد.



نکته: يك كلاس حداقل يك مسئوليت و حداكثر ده مسئوليت دارد. وقتي مسئوليت مشخص شود بعد خصوصيات و عمليات مشخص مي شود.

قدم هاي مدل کردن سیستم :

براي مدل کردن لغت نامه سیستم مي بايست قدم هاي زیر را اعمال کرد :

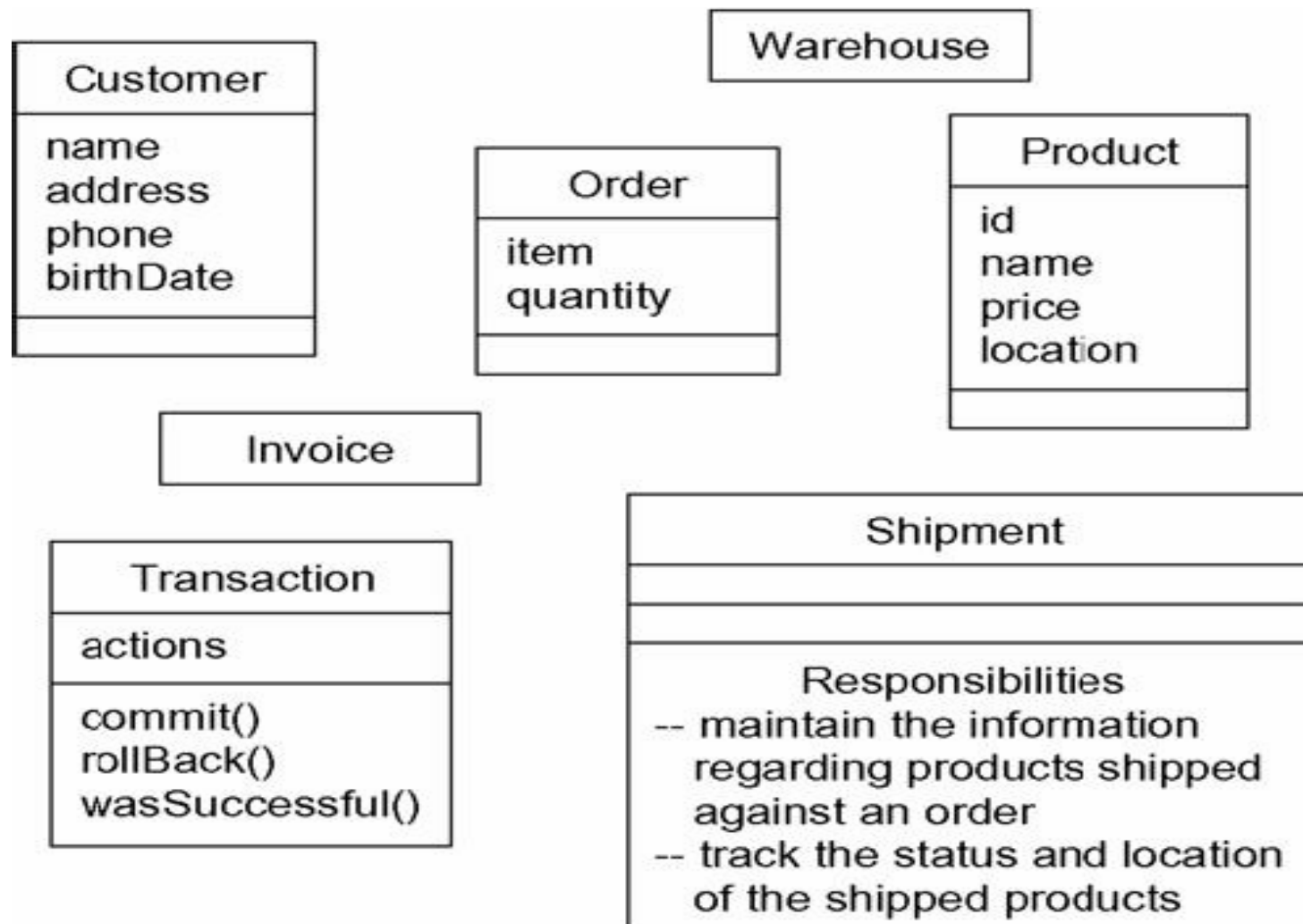
۱- در صورت مسئله به دنبال کلماتي بگردیم که توصیف کننده خود مشکل و مسئله و راه حل هاي آن باشد. آن ها را پیدا کرده و بعد جدا مي کنیم. با استفاده از **Abstraction** آنها يي که لازم است را جدا مي کنیم. (استخراج اسامی کلاس ها و اشيا)

۲- براي هر دید و **Abstraction** يك مسئوليت تعريف مي کنیم. در اين جا بايد براي براي يك دید و کار مجموعه اي از مسئوليت ها را برایش تعريف کرد. بايد مشخص شود مسئوليت هر کلاس چه مي باشد؟

۳- **Operation** ها و **Attribute** ها را که کلاس براي انجام آن مسئوليت به آن نیاز دارد را به آن اضافه کرد.

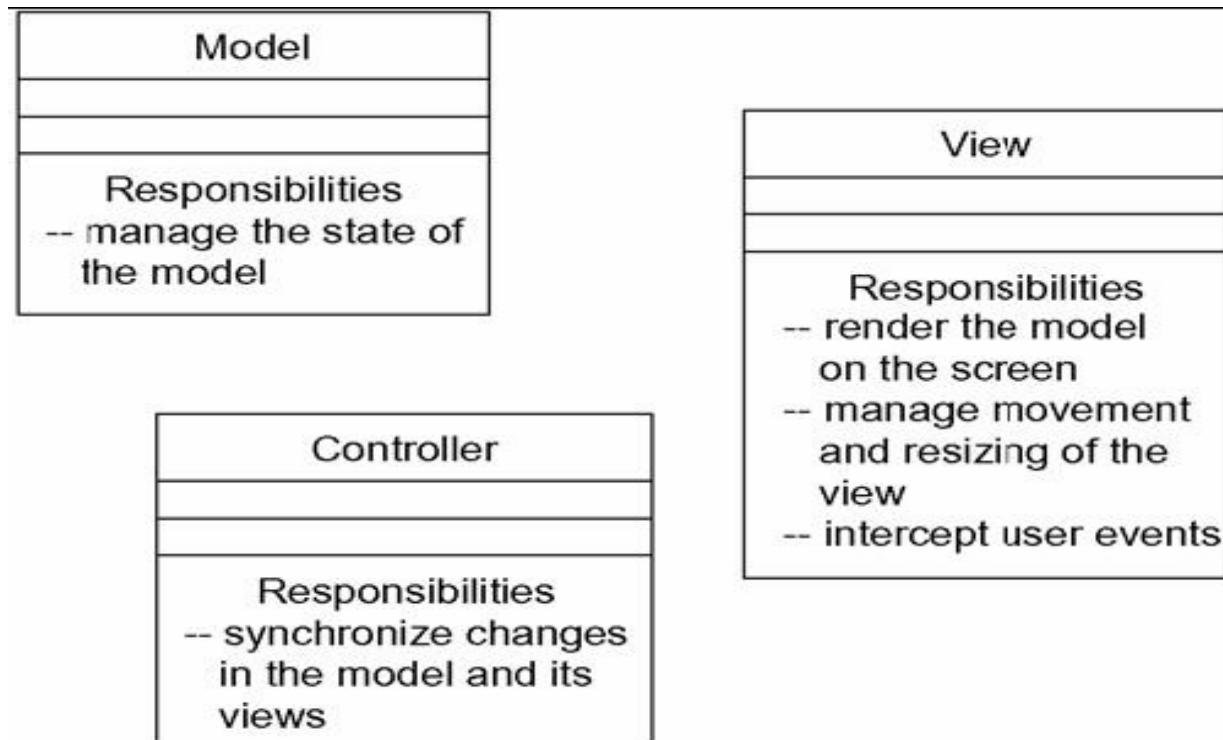
۴- در آخر کلاس هايي که ارتباط معنایی و مفهومي زیادی با هم دارند را از هم جدا کرده و در يك **Package** و دسته قرار دهیم.

مثال : سیستم انبارداری و فروش



توزیع مسئولیت های مدل سازی يك کلاس در سیستم :

- ۱- يك مجموعه اي از کلاس ها را که با هم کار مي کنند تا يك رفتار مشخص نشان دهند را مشخص کنیم.
- ۲- مشخص کنیم مسئولیت هر کلاس در ان رفتار چه چیز مي باشد؟
- ۳- اگر مسئولیت روي يك کلاس شدید باشد به نحوي که يك مسئولیت مرکزي دارد باید آن کلاس شکسته شود تا مسئولیت تقسیم شود. اگر مسئولیت کن باشد باید تقسیم را به اعتدال رساند تا به هر کلاس مسئولیت متعادل و دقیقي برسد.



Modeling Nonesoftware things

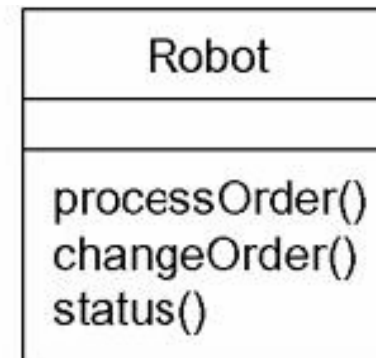
تعریف اشیا غیر نرم افزاری : اشیایی که در آینده مدل نرم افزاری نخواهند داشت .

برای مدل کردن اشیا غیر نرم افزاری چه اعمالی باید انجام داد؟

- ۱- می توان آن ها را به صورت يك كلاس استفاده کرد.
- ۲- می توان با استفاده از Stereotype يك بلاك يا مولفه جديد براي نمايش آن شي استفاده کرد (به وسیله دادن icon تصویری آن را در برنامه مشخص کرد).
- ۳- اگر اشیا ي سخت افزاری می باشد می توان به وسیله Node آن ها را مدل کرد.

مثال : شخصی که مسئول بر چسب زدن آدرس پرینت شده از پرینتر بر روی اجناس سفارش شده از طریق اینترنت می باشد .

Accounts Receivable Agent

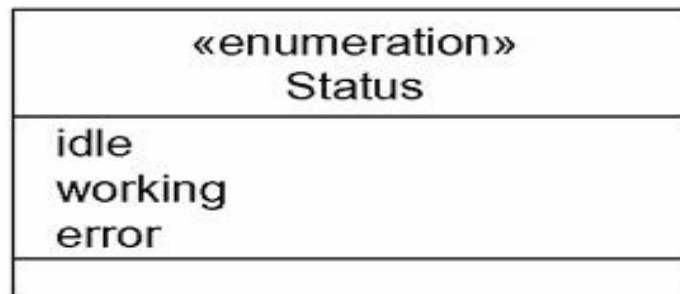
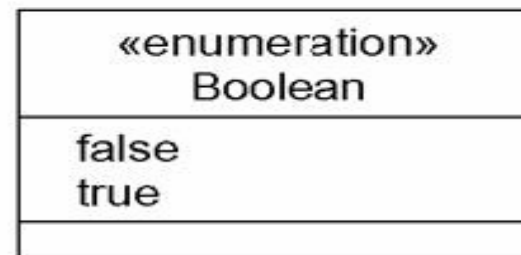
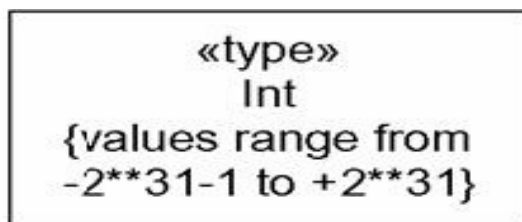
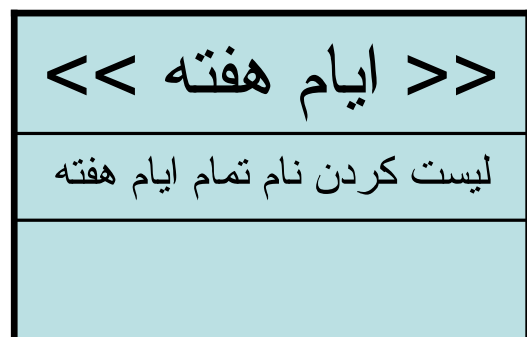


Modeling Primitive types

در مدل کردن نوع های اولیه بوسیله دو روش کار را انجام می دهیم:

۱- با استفاده از Enumerat types که به سوي Stereotype تغییر می کند.

۲- با استفاده از Stereotype که به سوي Enumerat types تغییر می کند.



◎ مدل کردن برای انواع شمارشی در جاوا ۱,۵ به نام enum داریم :

(در منبع CookCook)

به شکل زیر داریم :

```
Public enum Media {  
    book, music,cd,music-viny1, movie  
}
```

نتایج و نکات کلی:

*یک کلاس خوی ساخت: کلاسی است که باید یک **Abstraction** یا تجرید باشد از یک متن دقیق یا در یک سوال یا در یک جواب.

*یک کلاس خوش ساخت باید یک مجموعه کوچک و خوش تعریف از مسئولیت ها را داشته باشد که بتواند آن ها را به خوبی جواب دهد.

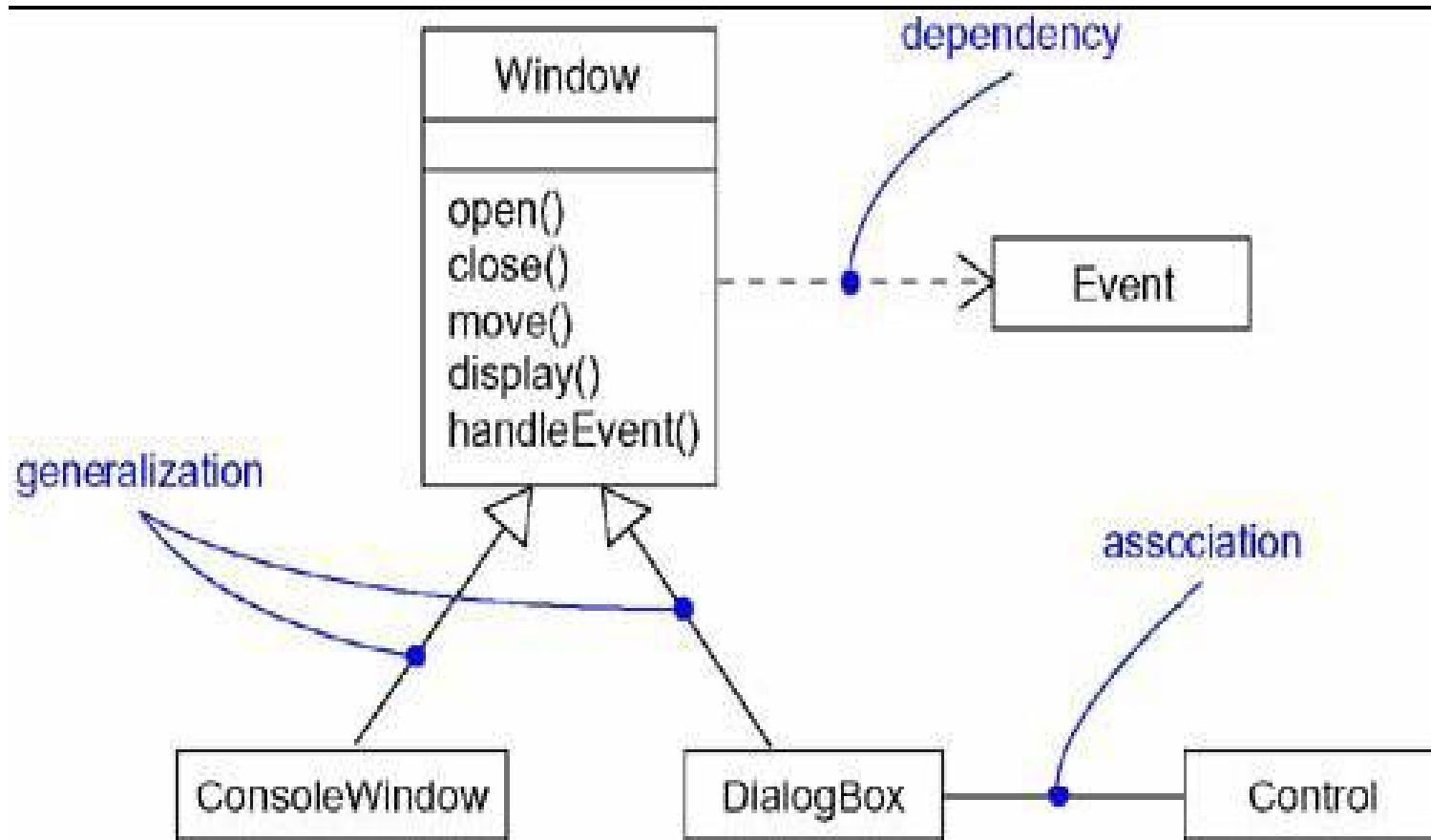
*در یک کلاس خوش ساخت یک خط مشخص برای کلاس بین مشخصات و پیاده سازی وجود دارد.

*یک کلاس خوش ساخت باید قابل فهم و ساده و قابل توسعه و همچنین قابل تلفیق باشد.

براي رسم کلاس در يك نمودار: UML

- ۱- آن **Property** ها **Operation** , **Attribute** ها را نمایش دهد. آن **Property** هایی که لازم و ضروري مي باشد را ذکر نموده جوري که آن هایی که لازم است نیاز برنامه باشد را نگه دارد.
- ۲- لیست طولانی از **Attribute** ها **Operation** , ها را باید به نوع مدیریت در آورد. یا اینکه همه را ذکر نکنیم یا اینکه بیاییم آن ها را دسته بندی کنیم.
- ۳- کلاس هایی که به هم مربوط هستند را در يك **Diagram** نمایش دهیم.

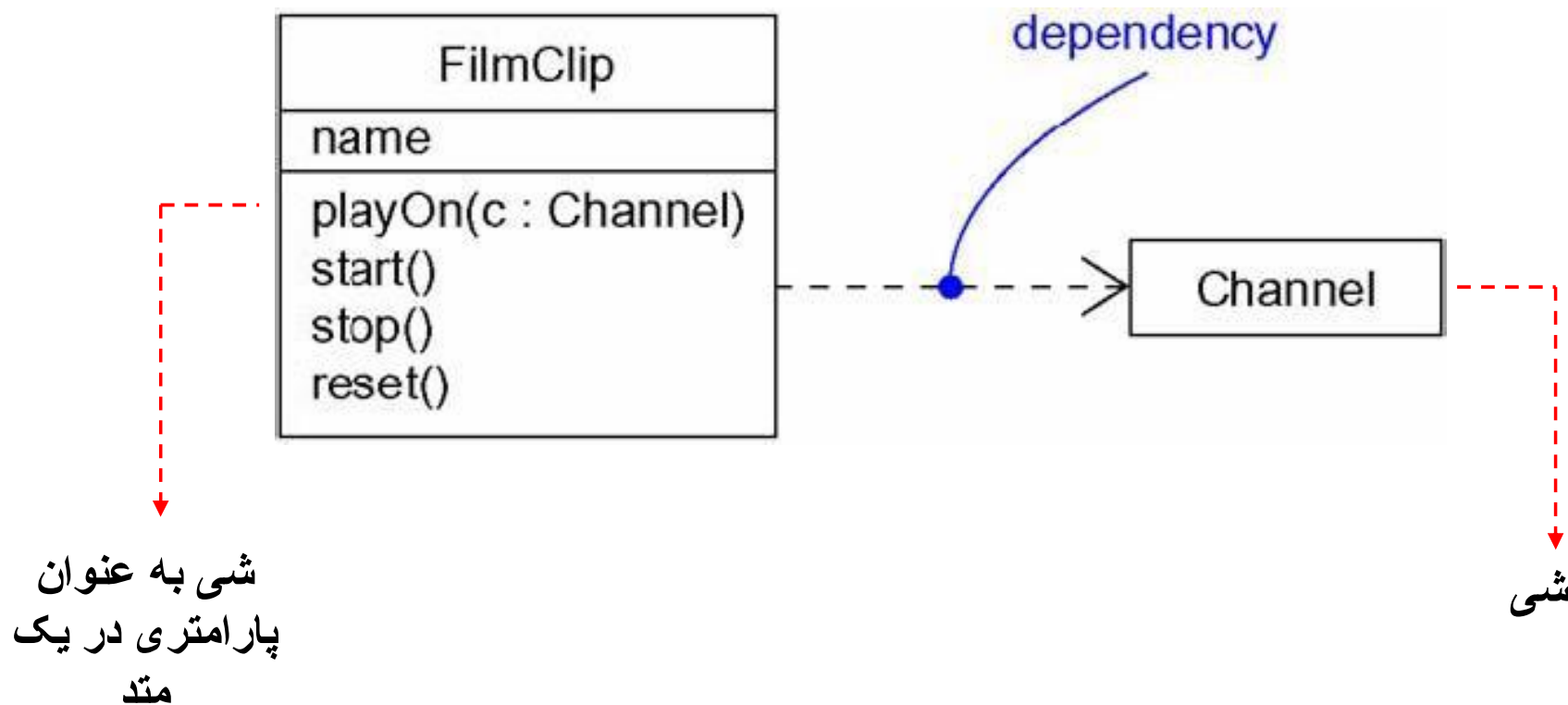
Relationship ارتباط بین اشیا است . در مدلسازی شی گرا سه ارتباط بسیار مهم داریم که شامل ارتباطهای وابستگی **dependency** ، همکاری **association** و تعمیم **generalization** می باشد . که بصورت گرافیکی یکسری خطوط ارتباط هستند که از مسیرهای مختلفی می گذرند و شکل های متفاوتی دارند .



وابستگی (Dependency) چیست؟

وقتی که يك شي از لحاظ معنایي وابستگی به شي دیگری داشته باشد . یعنی تغییر در يك شي مستقل باعث تغییر در شي مرتبط به همان شي در شي وابسته دیگر گردد.

اسم شي (وابسته) در پارامتری از متد شي (مستقل) آمده که این وابستگی شي را بیان می کند .

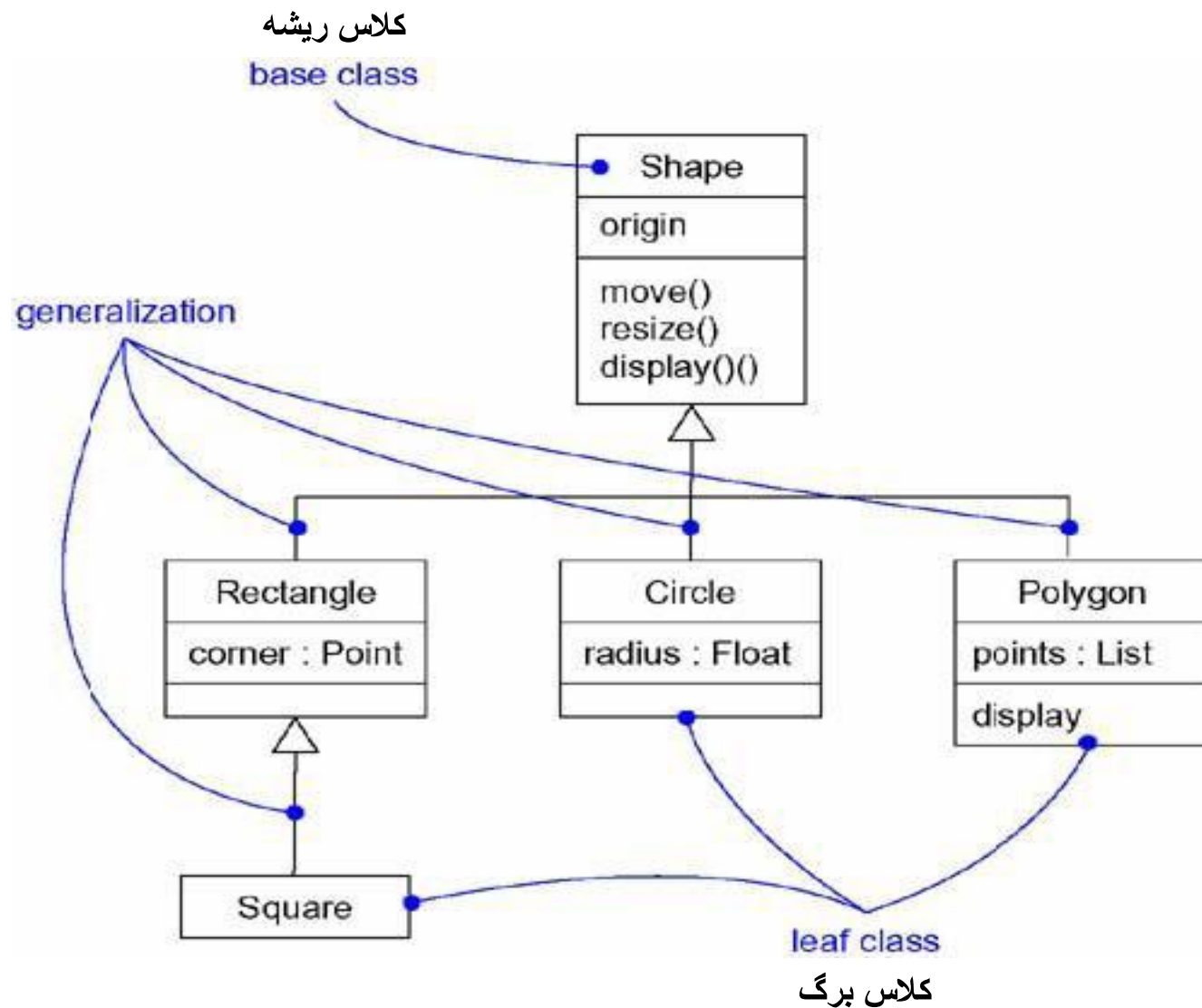


◎ بیشترین حالت وابستگی مربوط به حالتی است که شی ای از یک کلاس به صورت پارامتر در متد کلاس دیگر باشد.

◎ معمولاً اسمی برای وابستگی ها ذکر نمی شود . تنها حالت اسم گذاری موقعی است که بین دو کلاس چندین وابستگی وجود داشته باشد.

تعميم (Generalization) چیست؟

رابطه اي كه بين يك **Sub class** و **Super class** مي باشد.
آن را اصطلاحاً با **Is-a** مشخص مي كنند.



◎ این رابطه نیز می توان علاوه بر بین کلاس ها بین Package ها نیز اعمال کرد .

نکته : Sub class از جنس Super class است ولي Super class از جنس Sub class نمي باشد.

يك كلاس مي تواند صفر يا يك حالت يا والد داشته باشد كلاس Object فقط صفر حالت يا والد دارد.

كلاس پايه كلوسي است كه هيچ والدي ندارد. در جاوا كلاس برگ را با كلمه Finall نشان مي دهند.

Final class كلاس هايي هستند كه در جاوا نشان دهنده والدهايي است كه هيچ فرزندي ندارند.

همكاري (Association) چيست؟

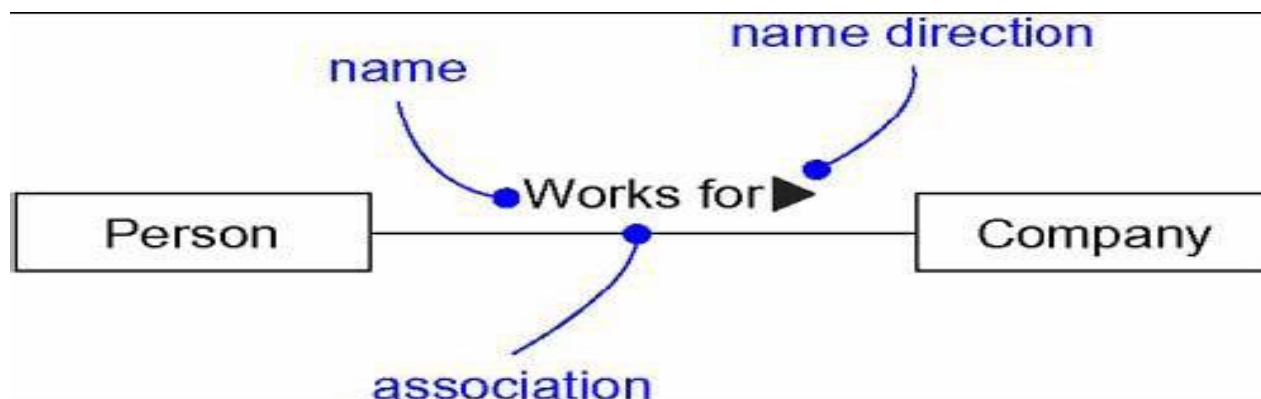
همكاري و وابستگي مي توانند انعكاسي باشند.
يك شي مي تواند همكاري و وابستگي به خودش داشته باشد.
رابطه همكاري و وابستگي Reflective نيست يعني نمي تواند
از خودش به ارث ببرد.

تعريف همكاري:

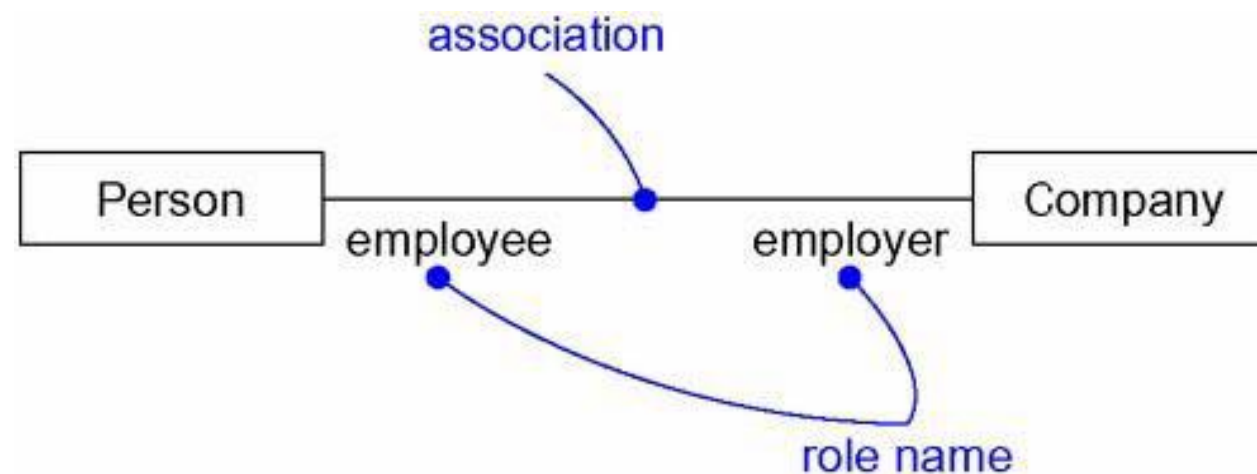
يك رابطه ساختاري است كه مشخص مي كند با داشتن اشيا يك كلاس ما مي توانيم به
شي يك كلاس ديگر برسيم (كه رابطه اشيا موجود كه نشان دهنده تعداد اشيا درگير در
دو طرف با كليۀ مشخصات عمومي) .

رابطه همكاري به چهار شكل زير نشان داده مي شود :

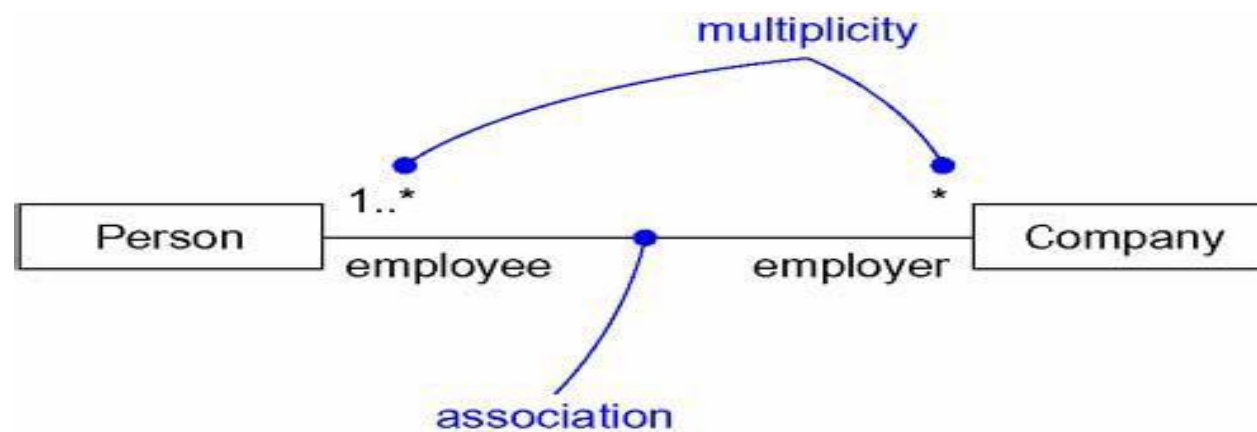
نمونه ۱: رابطه همكاري با نام و جهت ارتباط



نمونه ۲ : رابطه همکاری با نقش دو طرف



نمونه ۲ : رابطه همکاری با نقش دو طرف و تعدد در همکاری



Super class از Sub class خبري ندارد ولي Super class از Sub class خبر دارد.

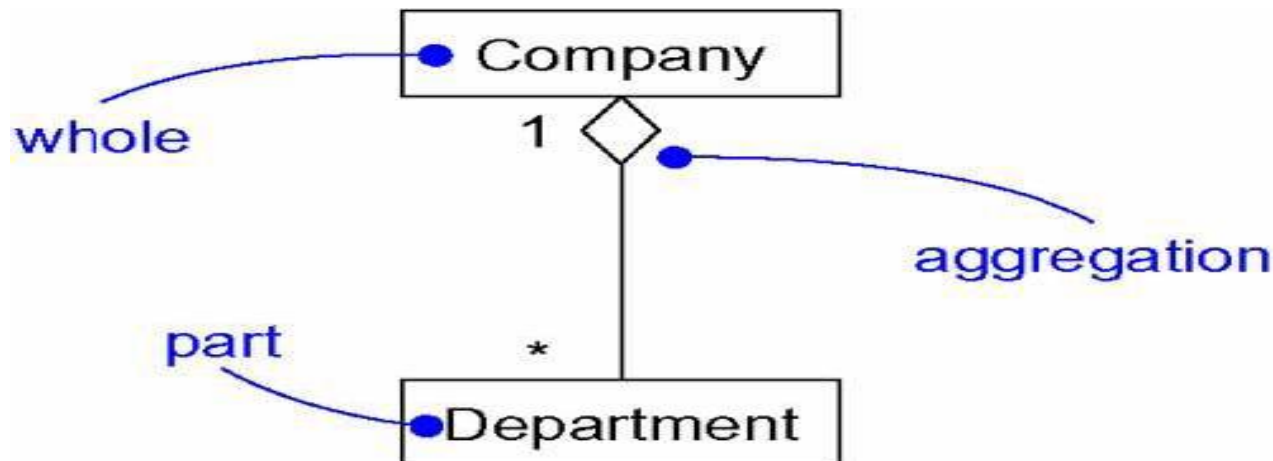
● حرکت کردن از يك شي شروع مي شود و به يك شي ديگر ختم مي شود كه اين گفته را **Navigation** مي نامند . پس بايد تفاوتی بين navigation و association وجود داشته باشد.

تجمع (Aggregation) چیست؟

نوع خاصی از همکاری است.

Association مي تواند حالت ساده باشد يعني در يك سطح باشد.

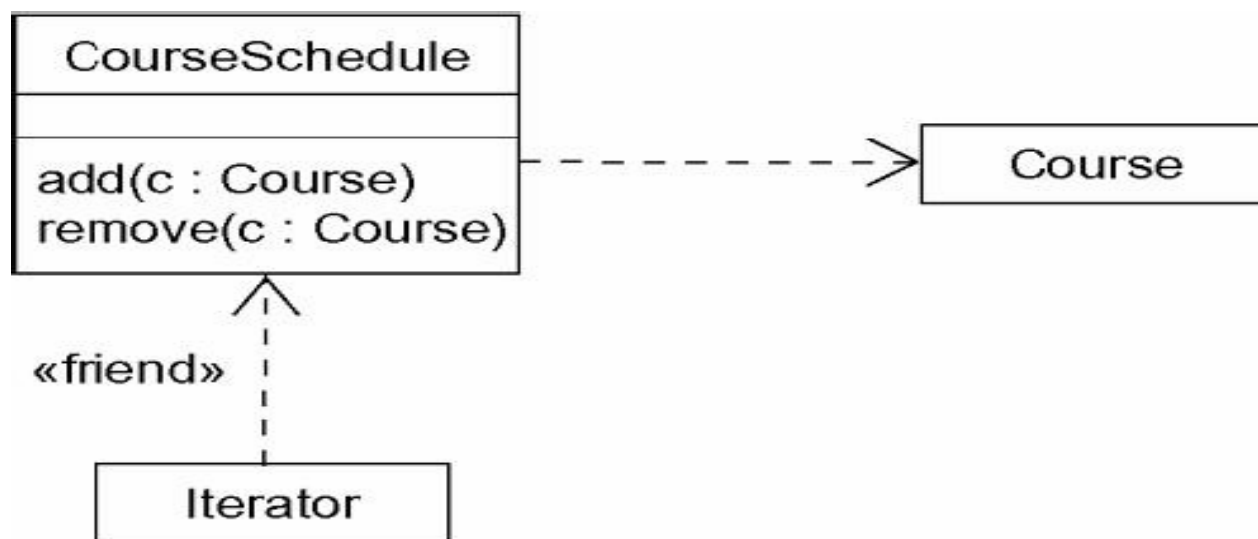
اما حالت ديگر مي تواند در سطح بالاتري باشد يعني: حالت جز به كل.



◎ برای نمایش آن از یک لوزی در طرف کل استفاده شود.
اگر این لوزی تو پر باشد **Composition aggregation** می باشد. به این معنی که این رابطه جز در رابطه کل ما ضروری می باشد.

نکات و تکنیک هایی که در روابط باید رعایت شوند

مدل کردن وابستگی **Dependency**:



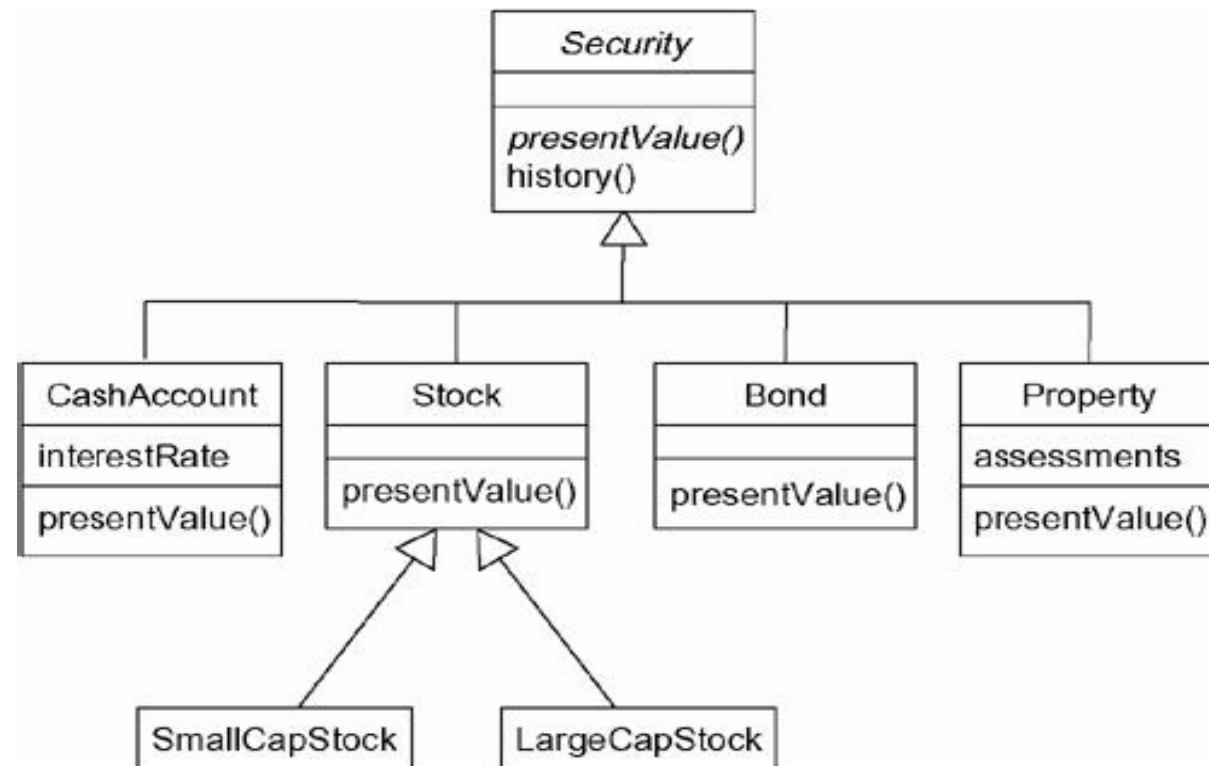
در این قسمت موضوع وراثت یگانه نیز مطرح می باشد.

برای مدل کردن وراثت یگانه باید سه قدم را انجام دهیم:

- 1 باید به کلاس ها نگاه کنیم . به اشتراك در **Attribute** و **Operation** و **Responsibility** نگاه می کنیم . آیا اشتراکی بین آن ها می باشد؟
این کلاس را مشخص می کنیم . اگر کلاسی وجود دارد مطالب مهم را از دل آن ها بیرون می کشیم و آن را به عنوان پایه و مرکز می گیریم.
- 2 يك کلاس را بیرون می کشیم و آن را به عنوان **Parent** انتخاب می کنیم. آن گاه دیگر کلاس های موجود را به عنوان فرزند های آن والد انتخاب می کنیم.
✓ نباید خیلی عمیق و گسترده باشد (هر کلاس نباید مسئولیت های زیادی انجام دهد)
✓ درخت وراثت نباید ارتفاع آن بیشتر از 5 Level باشد .
- 3 ارتباط **Sub class** و **Super class** را مشخص می کنیم و آنها را به کلاس جدیدی که ما ساخته ایم مدل می کنیم .

مدل کردن ارتباطات ساختاری:

Generealization و Relationship يك ارتباط يك طرفه است اما Dependency يك رابطه دو طرفه است.



تفاوت عمده ی بین وابستگی و عمومیت دادن با همکاری یکطرفه بودن این ارتباط است که در همکاری این ارتباط دوطرفه ولی در وابستگی و عمومیت دادن یکطرفه می باشد .

برای مدل کردن ارتباطات ساختاری مراحل چهارگانه زیر را باید انجام داد :

1 دو کلاس را در ابتدا انتخاب می کنیم بررسی می کنیم که آیا لازم است از یک شی از یک کلاس به یک شی از کلاس دیگر حرکت کنیم یا نه؟

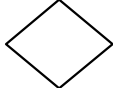
✓ اگر نیاز وجود داشت یک **Association** بین آن ها رسم می کنیم یک دید **داده گرا** بین آن ها تولید می کنیم.

2 برای هر جفت از اشیا ی سیستم بررسی می کنیم که آیا بین اشیا دو کلاس ارتباط یا تعاملی وجود دارد غیر از آن تعاملی که در **Dependency** است یا نه؟
✓ اگر غیر از این تعامل ارتباط دیگری باشد آن می شود **Association** و کلا دید **رفتار گرا** را در آن تعریف می شود.

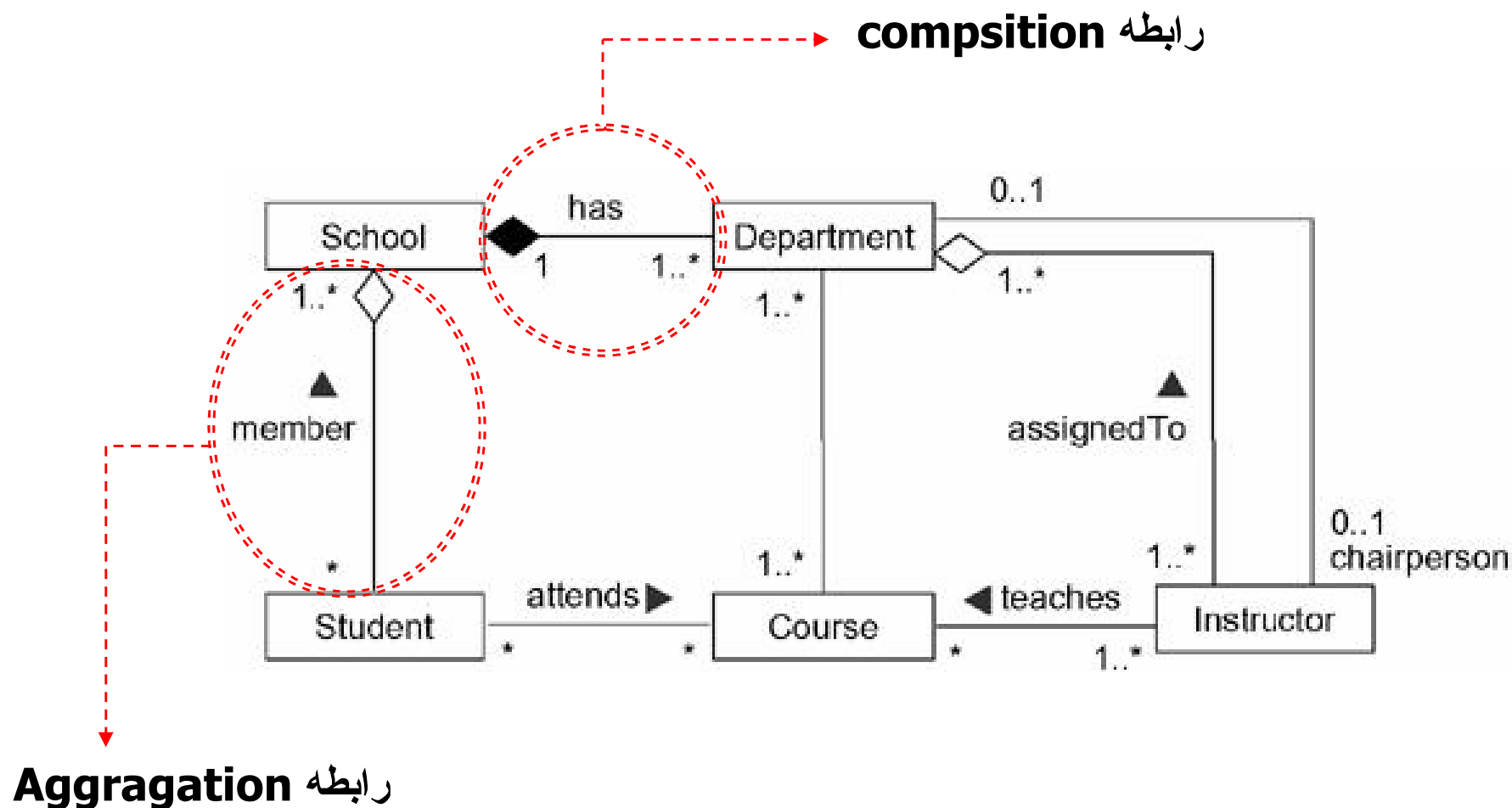
3 باید مشخصه هر **Association** را بیان کنیم .

4 در قدم آخر باید بیاییم نگاه کنیم آیا این **Association** نشان دهنده **Aggregation** است یا نه؟

✓ اگر بود **Association** را با دادن یک لوزی شکل و ساختار **Aggregation** در می آوریم.

و همچنین در گام چهار باید تعدد را مشخص کرد که در ارتباط همکاری این رابطه ی **Aggragation** که با لوزی تو خالی  نمایش داده و در کنار کل قرار می دهیم نوع دیگر ارتباط همکاری که شدیدتر می باشد رابطه ی ترکیب **Compsition** که با لوزی تو پر  نمایش داده می شود .

مثال : یک مدرسه حتما باید کلاس ، دفتر داشته باشد به این رابطه composition گفته می شود ولی هر مدرسه میتواند دانش آموز نداشته باشد که به این رابطه Aggragation گفته می شود .



☑ نکات :

🎯 از وابستگی زمانی استفاده می کنیم که ارتباطی که ما داریم مدل می کنیم ساختاری باشد.

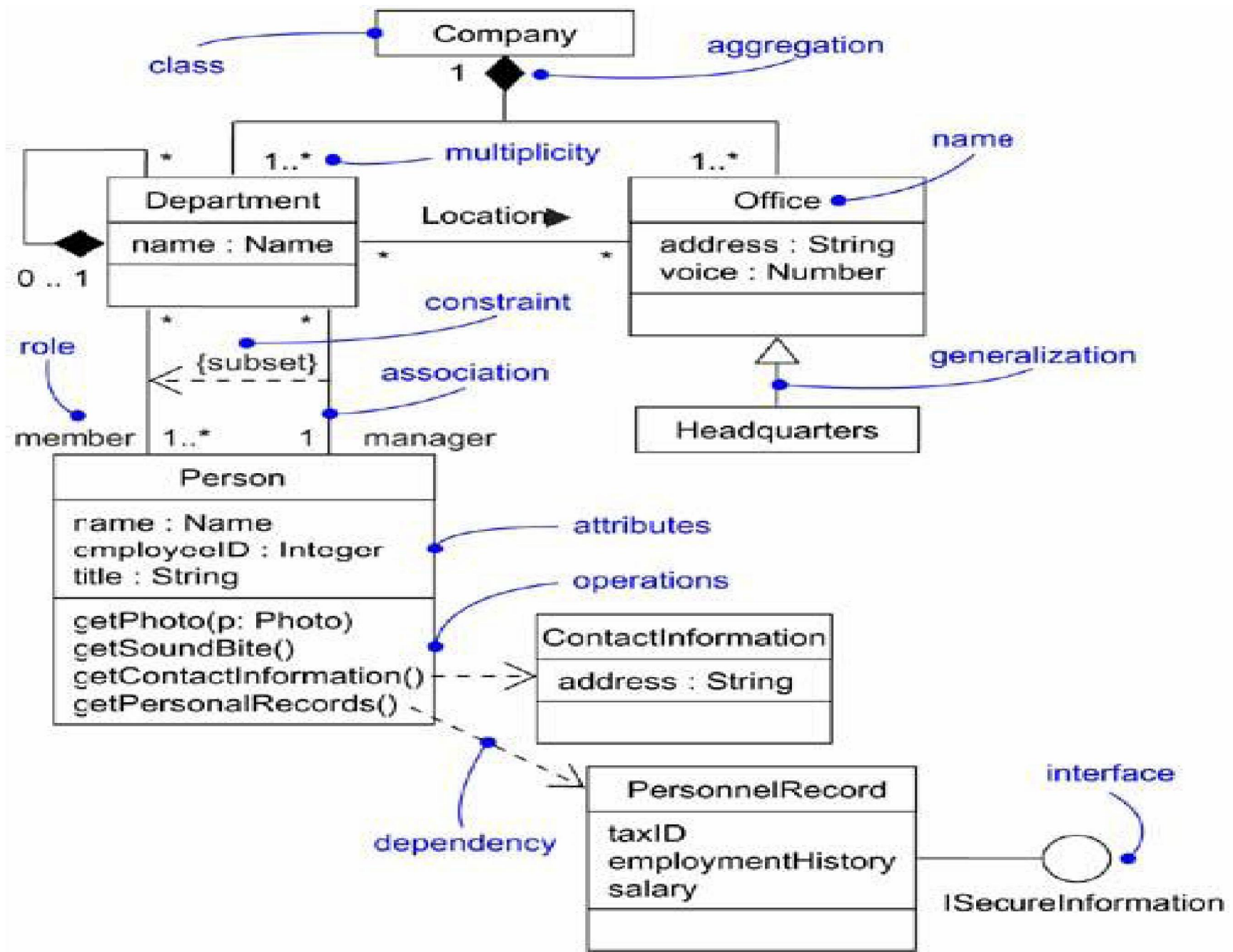
🎯 زمانی از وراثت استفاده می کنیم که یک رابطه **is-a** در سیستممان باشد.

🎯 وراثت چند گانه را می توان از رابطه **Aggregation** استفاده کرد.

🎯 از وراثت چرخشی بپرهیزید.

🎯 متعادل نگه داشتن درخت وراثت می باشد.

کلاس دیاگرام



مفهوم کلاس دیاگرام :

کلاس دیاگرام از مجموعه ای از کلاس ها ، رابط ها و همکاری و ارتباط ها بصورت گرافیکی می باشد .

کلاس دیاگرام معمولا شامل اشیا زیر می باشد :

- ⊙ کلاس ها
- ⊙ رابط ها
- ⊙ همکاریها
- ⊙ ارتباط های وابستگی ، همکاری و تعمیم

کلاس دیاگرام مانند دیگر دیاگرام ها ممکن است شامل یادداشت های توضیحی و محدودیت باشد .

کلاس دیاگرام ممکن است که شامل package یا subSystem یا هر دو باشد .
عمده ترین جایی که از کلاس دیاگرام استفاده می شود دید طراحی است .

وقتی که یک دید طراحی استاتیک از سیستم را مدل می کنید شما از کلاس دیاگرام ممکن است بعنوان یکی از سه مورد زیر استفاده کنید .

از کلاس دیاگرام در چه مواردی استفاده می کنند؟

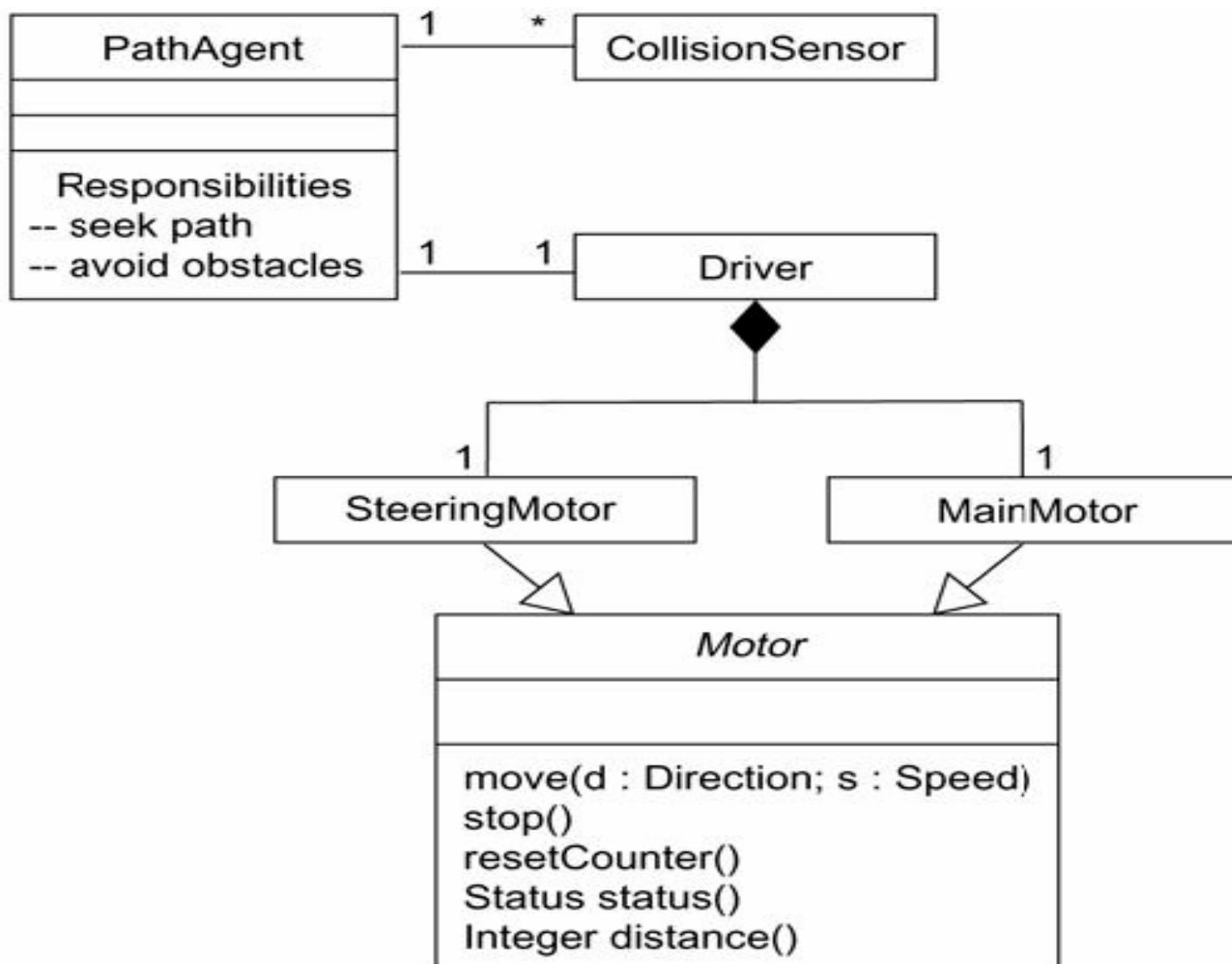
- ① در مدل کردن لغت نامه سیستم
- ② برای مدل کردن همکاری های ساده
- ③ مدل کردن شمای منطقی پایگاه داده ها

مدل کردن لغت نامه سیستم :

To model the vocabulary of a system

منظور فصل چهارم که در رابطه با کلاس ها می باشد برای توضیحات بیشتر به این فصل مراجعه کنید .

مثال : مدل کردن یک همکاری ساده



برای مدل کردن بانک اطلاعاتی:

① دانه به دانه کلاس ها را بررسی می کنیم. از این جهت که کدام کلاس عمر داده هایی که در درون آن است بیش از عمر برنامه است .

② یک کلاس دیاگرام را رسم می کنیم که فقط طول عمر داده هایش بیش از طول عمر برنامه هایش باشد. هر کدام از این کلاس ها را با یک Tagged value مشخص می کنیم.
Tagged value را به عنوان یک Persistent بررسی می کنیم.

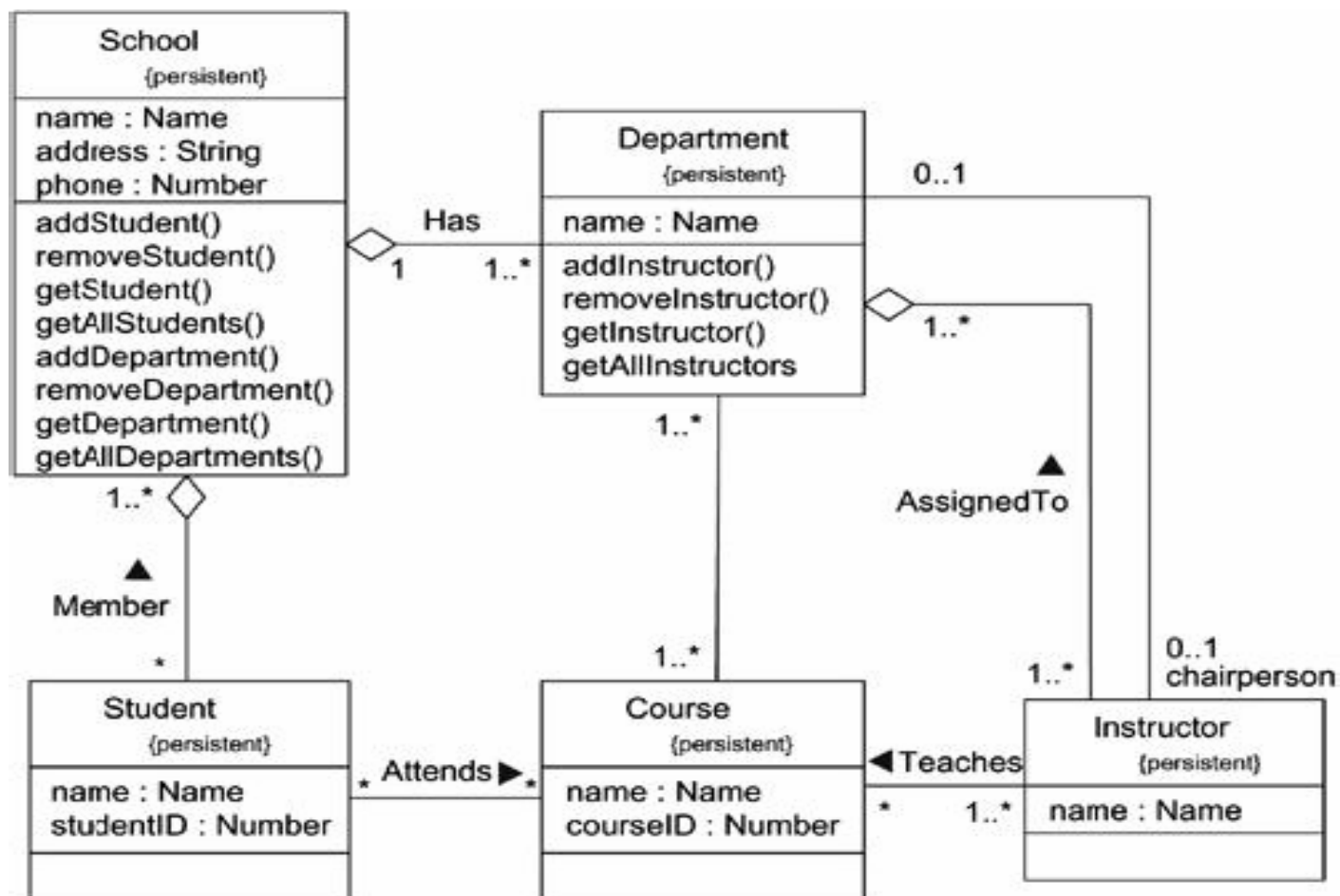
③ Attribute ها را دقیقاً مشخص می کنیم Type. و اسم آن ها را مشخص می کنیم و می گوییم آیا این مشخصات ذخیره می شوند یا نه؟

④ به وضعیت رابطه های یک به یک و یک به چند و چند به چند رسیدگی می کنیم. اگر لازم باشد یک سری عناصر را مثلاً در رابطه چند به چند اضافه می کنیم.

⑤ روی هر کلاس دقت شود که چه کارهای اطلاعاتی انجام شود. عملیات لازم برای هر کلاس را در صورت لازم توسعه می دهیم.

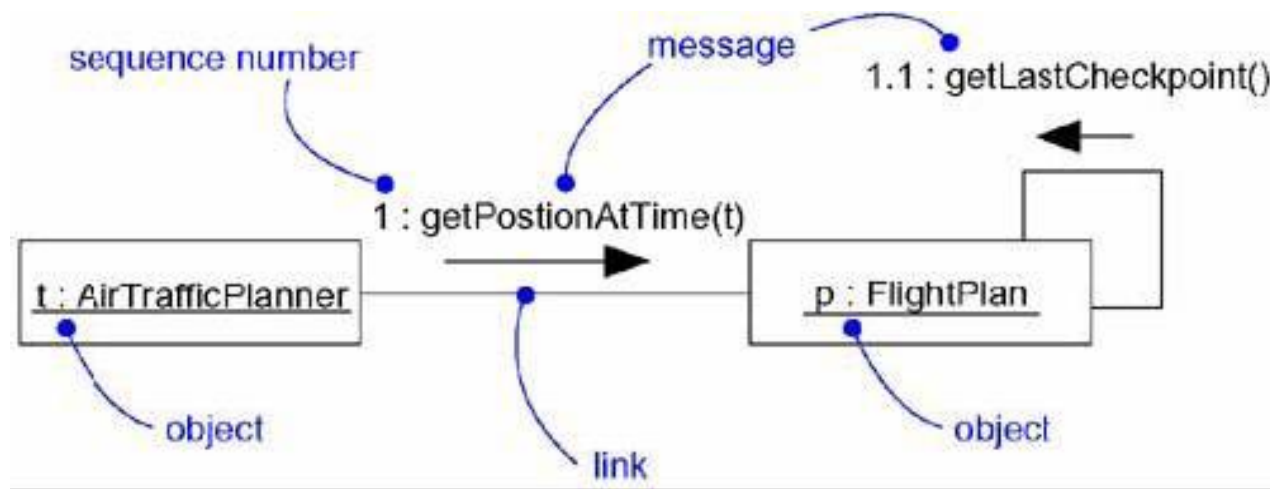
⑥ در این مرحله از یک ابزار برای تولید خودکار شمای بانک استفاده می کنیم.

مثال : مدلسازی یک شما



persistent

مثال : پیغام ها ، لینک و توالی



Interaction چیست؟

نمایش دهنده يك تعامل است که تشکیل شده از مجموعه اي از اشیا و ارتباط بین آن ها و همین طور پیغام هایی که بین اشیا رد و بدل می شود .

Interaction

از دو دیاگرام Sequence و Collobration تشکیل شده است .

تعریف Interaction : یک رفتار که تشکیل شده از مجموعه ایی از تبادل پیام ها میان مجموعه ای اشیا در یک زمینه خاص برای انجام یک هدف می باشد .

استفاده از interaction برای مدل کردن جنبه دینامیک از همکاریها ، به نمایندگی مجمعی از اشیایی که نقش ایفا می کنند و همه با هم کار می کنند برای اجرای برخی از رفتارها .

Sequence diagram : يك interaction دیاگرامی است که بر توالی زمانبندی پیغام ها تاکید دارد.

Collaberation diagram : يك interaction دیاگرامی است که بر ساختار و سازماندهی انتقال پیغام ها بین اشیا تاکید دارد.

این دیاگرام برای دو هدف استفاده می شود:

- 1 برای مشخص کردن سیستم .
- 2 برای این استفاده می شود که می خواهیم از روی آن کد بنویسیم.

: Sequence diagram

آن را به صورت يك جدول نمايش مي دهيم.

- ♦ در محور X ها اشيائي كه در ان تعامل شركت دارند رسم مي شوند.
- ♦ معمولاً ترتيب قرار گرفتن اشيا از چپ به راست بر حسب تعامل هايي كه در آن شركت مي كنند مي باشد.

: Collaboration diagram

در اين قسمت فقط بين اشيا يي كه با هم تعامل پيغام ي دارند مشخص مي شود.

در حالت كلي سه چيز در تعامل وجود دارد:

- ① اشيا Object
- ② پيوند ها Link
- ③ پيغام ها Message

Sequence diagram نسبت به collaboration diagram دارای دو چیز اضافه تری هستند:

① Life line : نقطه ایجاد شی تا نقطه از بین رفتن شی را مشخص می کند.

② Focus of control : نقطه ای که در آن کنترل وجود دارد. آن را با یک مستطیل ضخیم نمایش می دهند.

در sequence diagram می توان دو چیز را اضافه کرد:

① امکان شرط گذاشتن : در صورتی که آن پیغام اجرا شود آن شرط برقرار می شود.

② actor ها : که شروع کننده یک فعالیت و یا تعامل هستند.

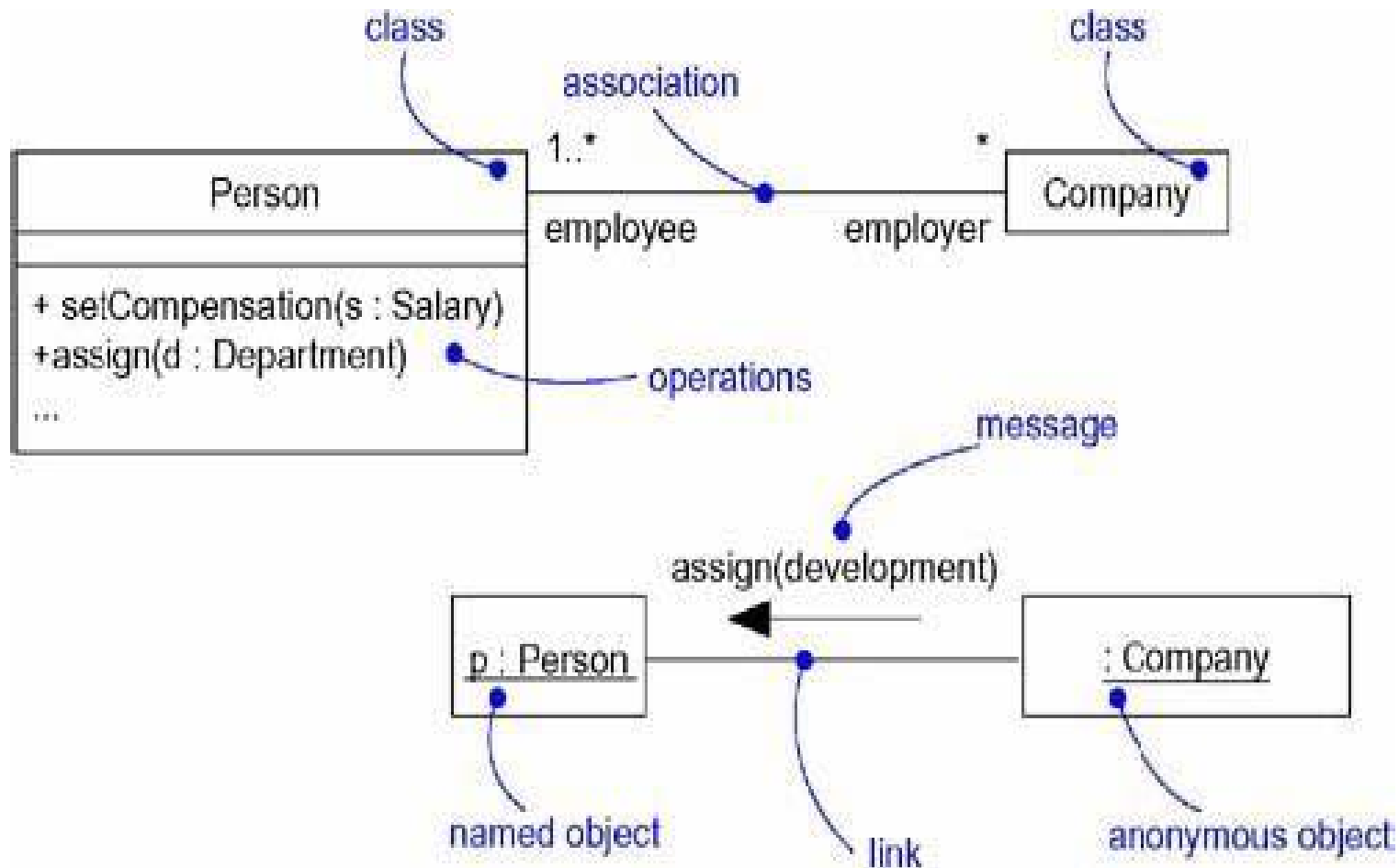
در collaboration diagram مي توان دو چيز را اضافه كرد:

- 1 بحث شماره گذاري
- 2 بحث شرط گذاري روي پيغام ها

از sequence diagram در موارد زير استفاده مي شود:

- 1 براي كنترل جريان برنامه
- 2 براي زمان بندي

مثال : Link و همکاری



پیغام Message : پیغام هایی که بین اشیا وجود دارد به یکی اشکال زیر می باشد :

Call ♦

Return ♦

Send ♦

Destory ♦

Create ♦

Call : صدا زدن یک متد از یک شی

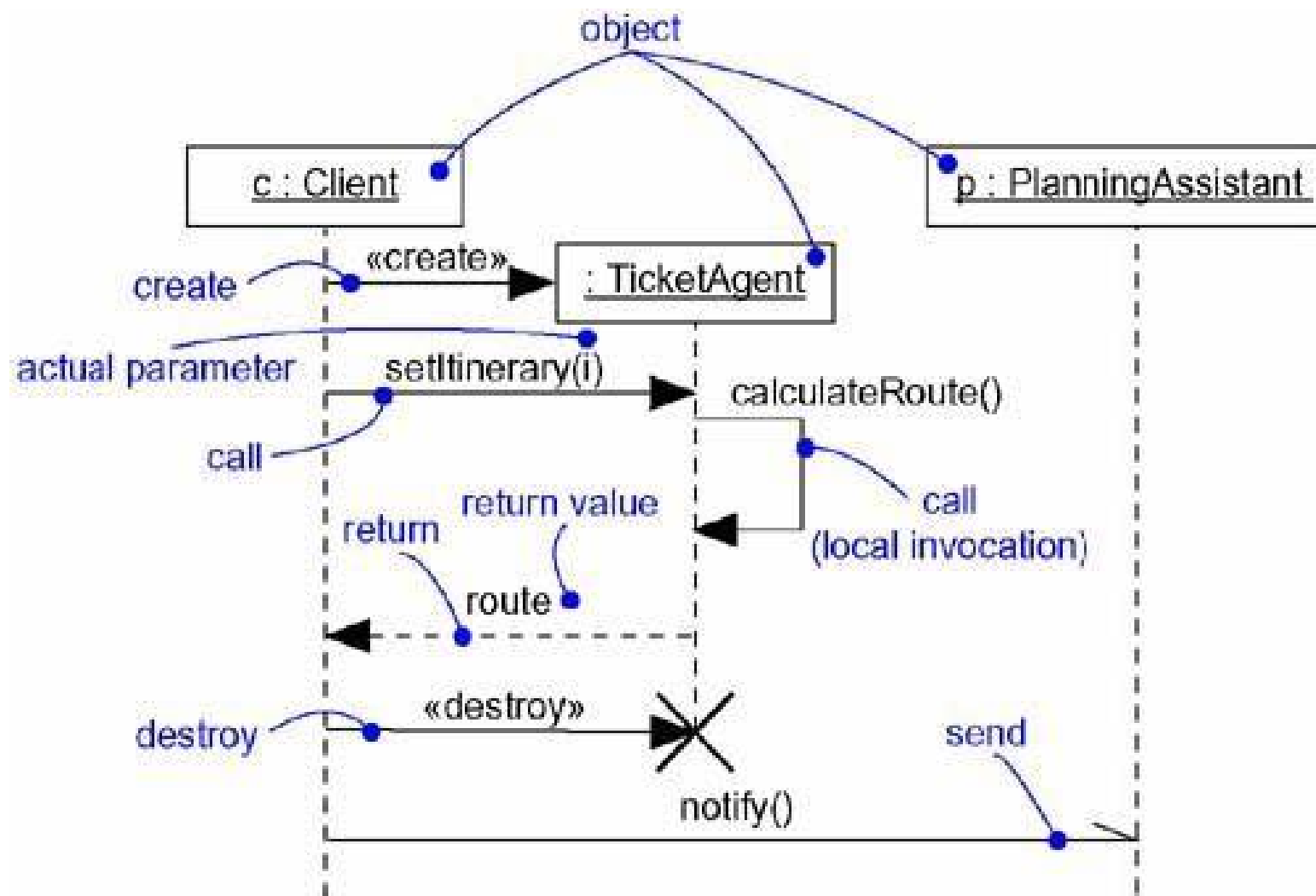
Return : تمام شدن کار یک متد

Send : سیگنال فرستادن بین اشیا مانند : wait و notify

Create : بوجود آوردن یک شی

Destory : از بین بردن اشیا

Message : مثال



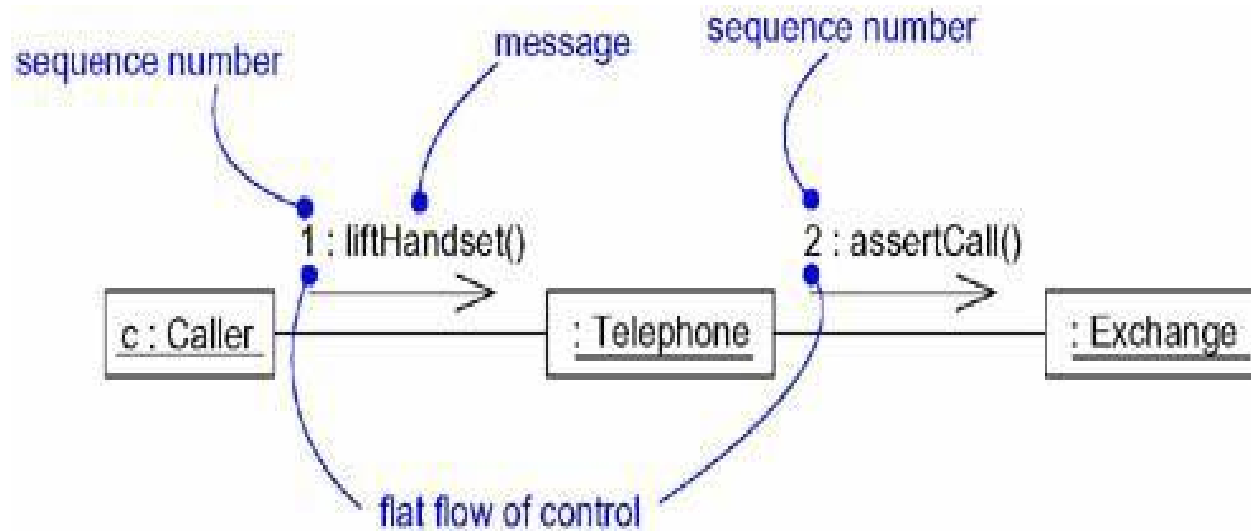
Sequencing : توالی یا عملیات پشت سر هم

دو نوع توالی وجود دارد :

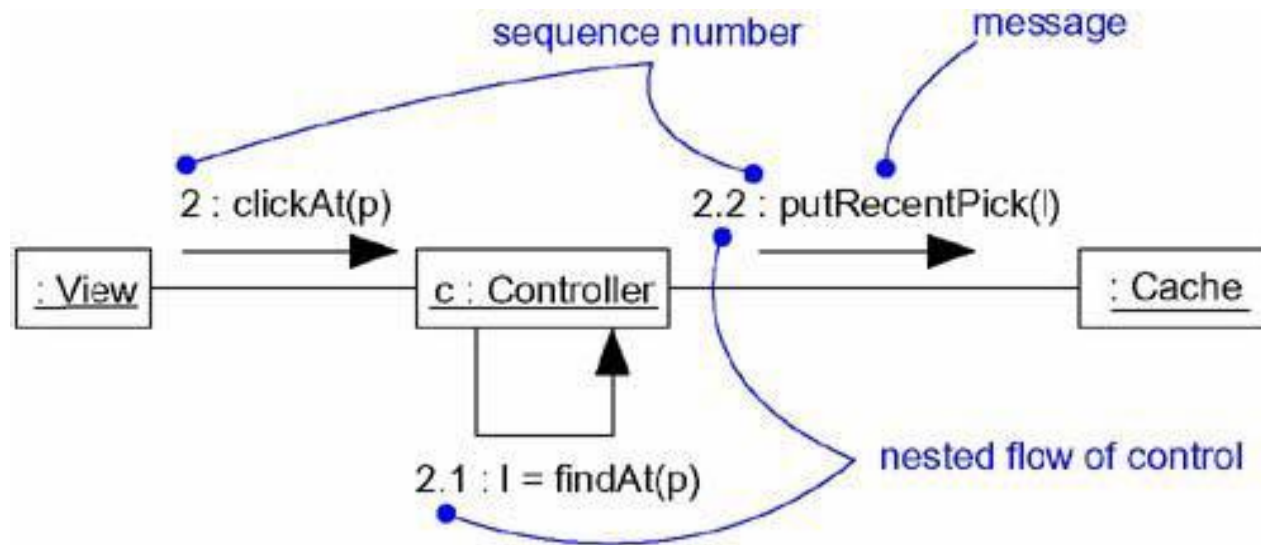
1 توالی همسان Flat : شماره گذاری به ترتیب می باشد از ابتدا تا انتها

2 توالی رویه ای Procedural : شماره گذاری رویه ای می باشد به یک متد شماره ای می دهیم اگر همین متد ، متد بعدی را صدا زد شماره بالاتری به متد بالایی می دهیم به همین ترتیب تا انتها

Flat Sequence : مثال



Procedural Sequence : مثال



D5 : ejectHatch(3)

از D که نقطه شروع می باشد ۵ مرحله به جلو حرکت می کنیم .

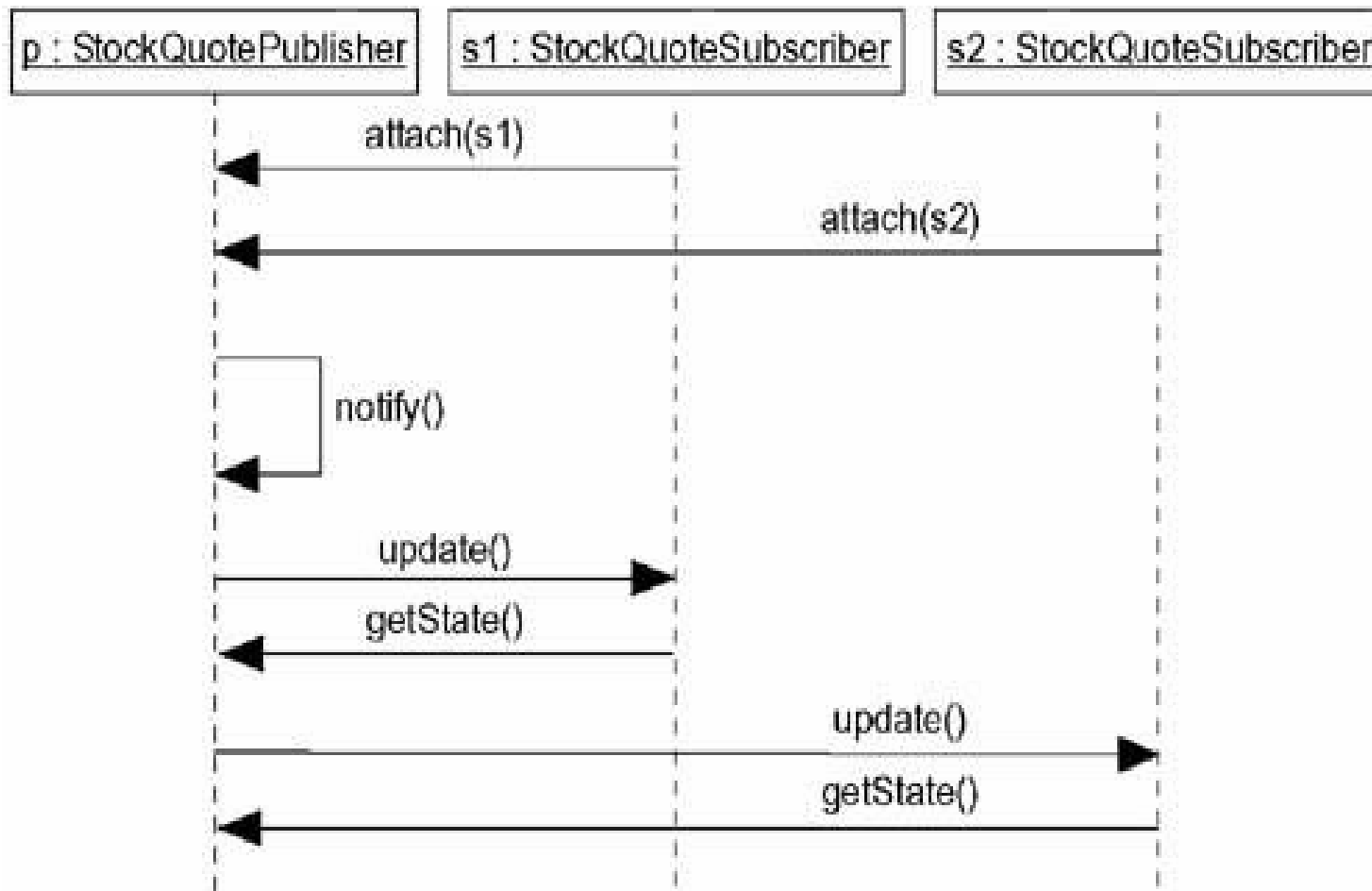
1.3.2 : p := find("Rachelle")

از نقطه شروع که ۱ باشد به قسمت ۳ آن و سپس به قسمت ۲ آن حرکت می کنیم .

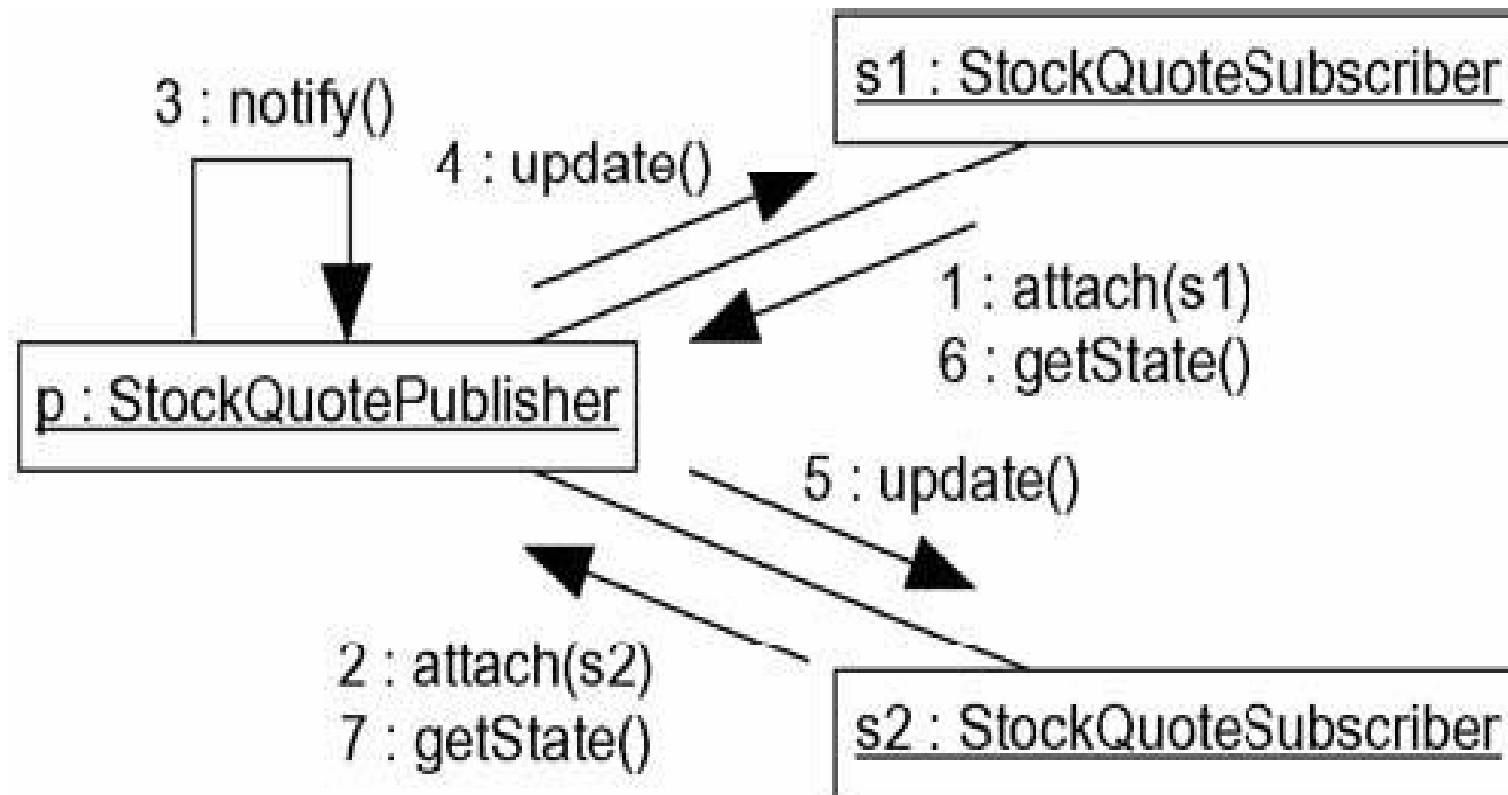
براي مدل سازي كنترل جريان برنامه :

- 1 براي چه Interaction مي خواهيم Sequence رسم كنيم . مثلا در سطح سيستم يا در سطح زير سيستم و غيره.
- 2 اشيايي كه در تعامل نقش دارند را مشخص مي كنيم. اشيا را از بالا به پايين و از چپ به راست بر حسب نقش و زمان اجرا مشخص مي كنيم.
- 3 براي همه اشيا خط زندگي ان ها را مشخص مي كنيم.
- 4 با پيغام شروع كننده Interaction شروع مي كنيم. الگوريتم و برنامه را Trace کرده و قدم به قدم پيش مي رويم و پيغام ها را به دياگراممان اضافه مي كنيم . در اين قسمت اگر لازم باشد براي برنامه شرط يا پيغام مي گذاريم.
- 5 پيغام ها را شماره گذاري مي كنيم.

Flow of Control by Time : مثال



Flow of Control by Organization : مثال



برای مدل کردن جریان کنترل با استفاده از ساختار یا سازماندهی پیغام ها:

① **object** هایی که در این تعامل نقش دارند را مشخص می کنیم. اشیایی که انتقال پیغام زیادی به آن ها انجام می شود را در وسط صفحه قرار داده. اگر **attribute** های آن شی به نحوی تغییر کرده باشد که در یک شی دیگر قرار بگیرد هر دو شی را ذکر می کنیم و با رابطه های زیر ارتباط بین آن ها را ذکر می کنیم:

✓ رابطه **Become** تبدیل شدن

✓ رابطه **Copy** از شی مورد نظر اصلی مان کپی می گیریم تا شی دیگر تولید شود.

② ارتباط بین اشیا را مشخص می کنیم.

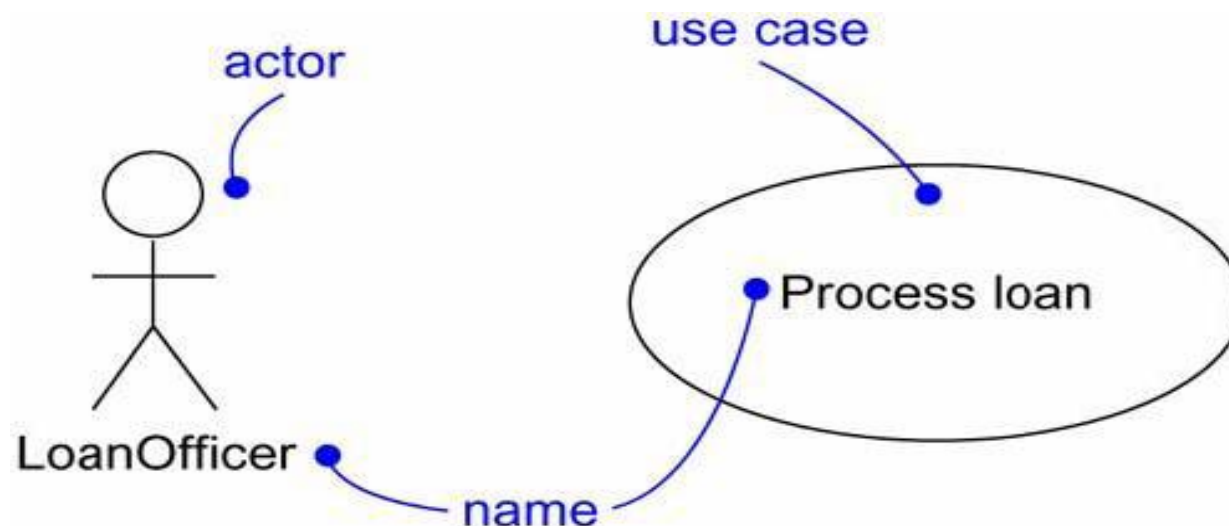
نکته: برای چهار دیاگرام **class diagram** و **sequence diagram** و **collaboration diagram** و **state chart diagram** می توان مهندسی مستقیم و معکوس استفاده کرد.

از شرط و چرخش زیاد در **collaboration diagram** و **sequence diagram** استفاده نکنیم بلکه بهتر است در **activity diagram** قرار دهیم زیرا **activity** دیاگرام حالت الگوریتم را دارد.

Use Case : مشخص کننده رفتار یک سیستم یا قسمتی از یک سیستم می باشد در حقیقت با توالی از عملیات تشکیل شده که سیستم انجام می دهد تا یک نتیجه را به Actor یا عامل برساند .
و مجموعه ای از interaction هاست که در یک طرف اشیا هستند که خارج از سیستم هستند .
سیستمی موفق است که که کلیه انتظارات Actor بر آورده کند .

Actor (عامل) : یک عامل نمایش دهنده یک مجموعه همسان از نقش هاست که کاربر use case در تعامل با use case بازی خواهد کرد .

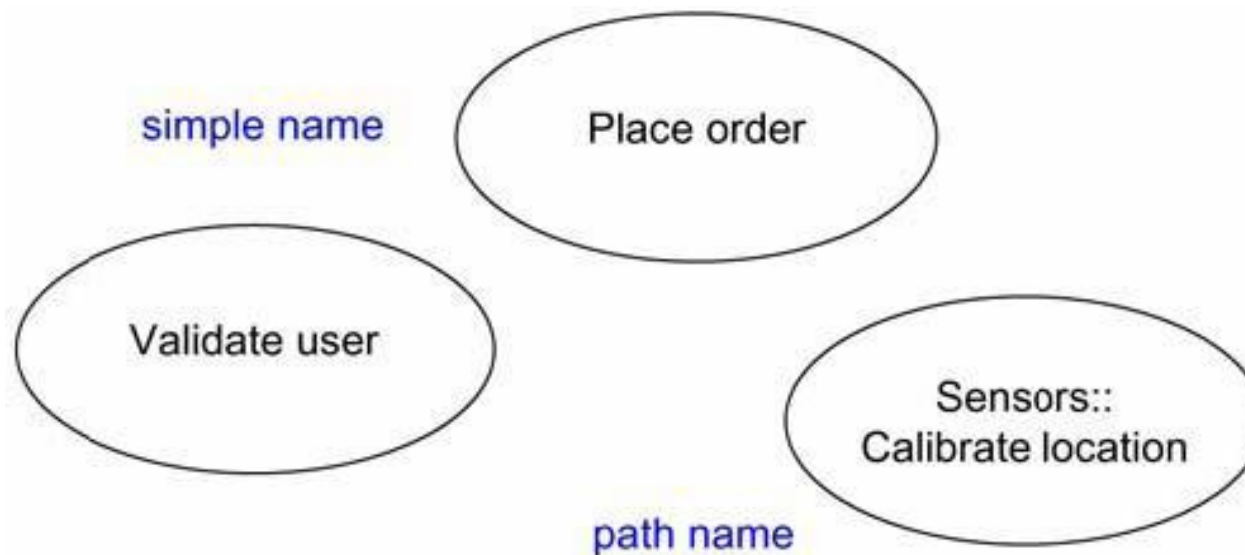
انواع Actor : انسان ها ، سیستم های خود کار دیگر بیرون از سیستم و دستگاه سخت افزاری



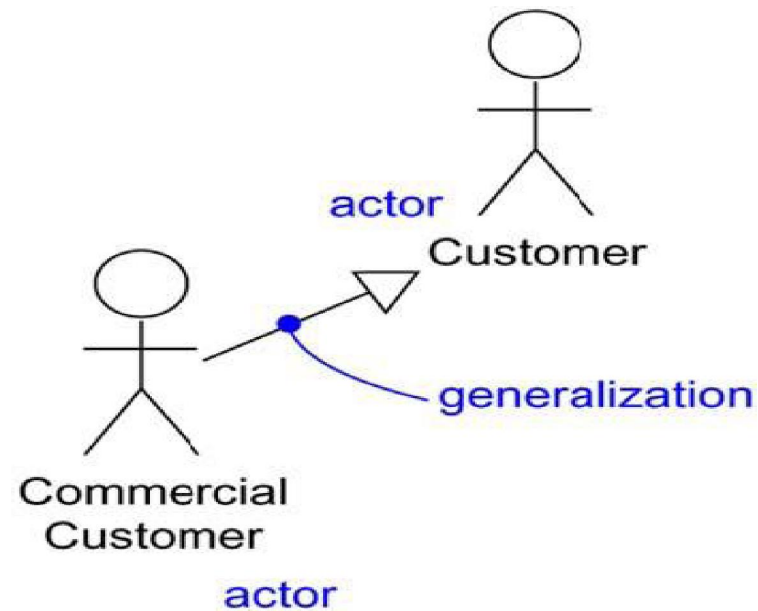
Names (نام) :

نام یک use case در یک package می بایست منحصر به فرد باشد .

می توان نام آن را ساده یا با مسیر مشخص نمود مانند زیر :



♦ رابطه بین Actor ها **generalization** می باشد .



برای نمایش عامل ها به شکل های مختلف از **stereotype** استفاده می شود .

کارهایی که در use case انجام می شود :

جریان رویداد ها در use case

روی کاری که درون سیستم انجام می شود در مورد چگونگی انجام کار در این قسمت بحث نمی شود بلکه کارهایی use case باید انجام دهد مورد بررسی قرار می گیرد .

انواع روش برای نمایش انجام کار use case :

✓ استفاده از متن ساده

✓ Activity Diagram

نوشتن جریان رویدادها در متن ساده :

❶ جریان اصلی رویداد روی سیستم (رفتار های نرمال عامل)

مانند زمانیکه شخص پسورد درست وارد کند .

❷ جریان استثنا در سیستم

مثلا زمانیکه فرضا شخص سه مرتبه پسورد درست وارد نکند) .

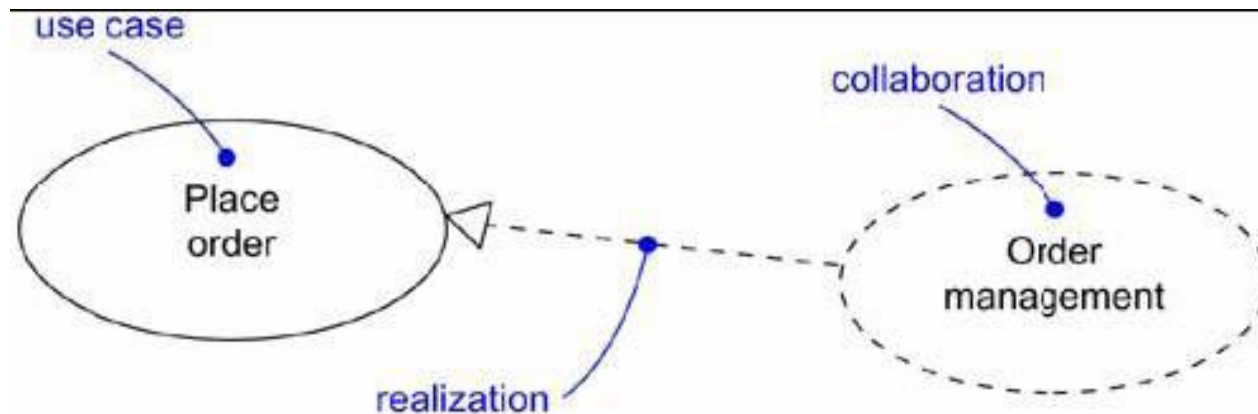
سناریو و use case

رابطه سناریو با use case مانند رابطه ی شی با کلاس می باشد
پس برای use case ممکن است سناریو های مختلفی ایجاد شود .

همکاری ها و use case

رابطه بین collaboration و use case ، realization

می باشد .



سازماندهی use case ها :

ما می توانیم use case های سیستم را در گرو هایی درون بسته تقسیم بندی و سازماندهی کنیم با سه مکانیزم سازماندهی که عبارتند از :

◆ Generalization (تعمیم)

◆ include (شمولیت)

◆ Extend (توسعه)

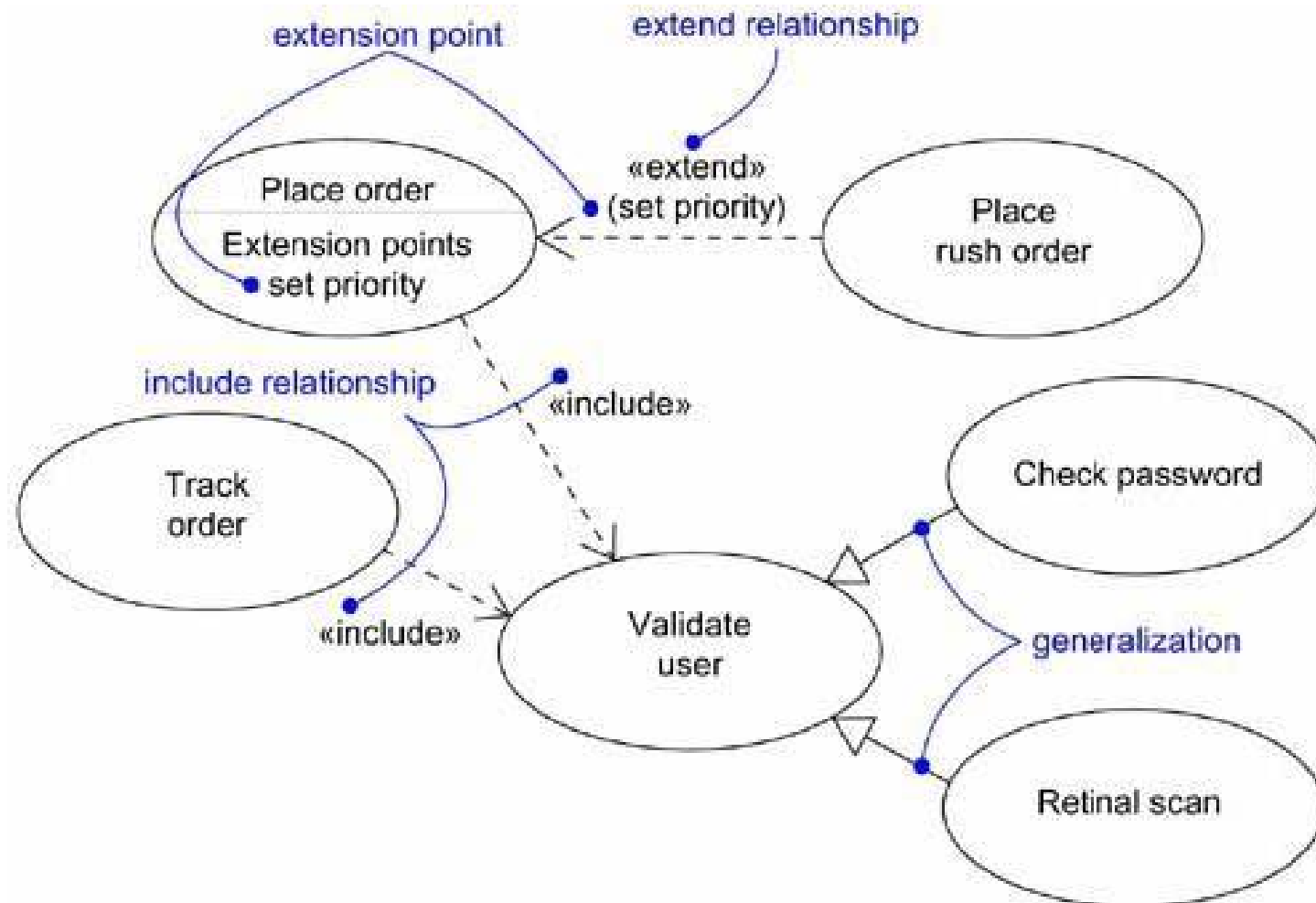
رابطه تعمیم بین use case ها خیلی شبیه رابطه تعمیم بین کلاس هاست .

در use case دیاگرام زمانی که می خواهیم use case ای را که حتما انجام شود تا نیاز Actor برآورده شود آن را با include نشان می دهیم .

Extend : use case ای که حالت خاصی از use case ای که include شده را توسعه داده و Actor می تواند آن use case را انجام داده یا از آن صرف نظر کند .

برای توضیح بیشتر و ایجاد یک use case سازماندهی شده در طی یک مثال تحت عنوان سفارش کتاب شرح خواهیم داد .

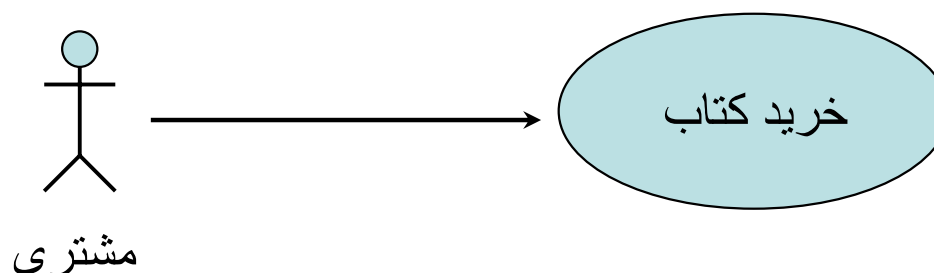
Generalization, Include, and Extend



مثال : رسم use case خرید یا سفارش کتاب .

برای رسم این نمودار ابتدا نیاز عامل را بررسی کرده سپس تحت عنوان یک use case در آورده حال اگر این کاربرد حجیم و پیچیده باشد همانطور که قبلا ذکر کردیم می بایست به کاربرد های کوچکتری تجزیه کنیم سپس اعمال مکانیزم های سازماندهی به این دیاگرام انجام خواهیم داد .

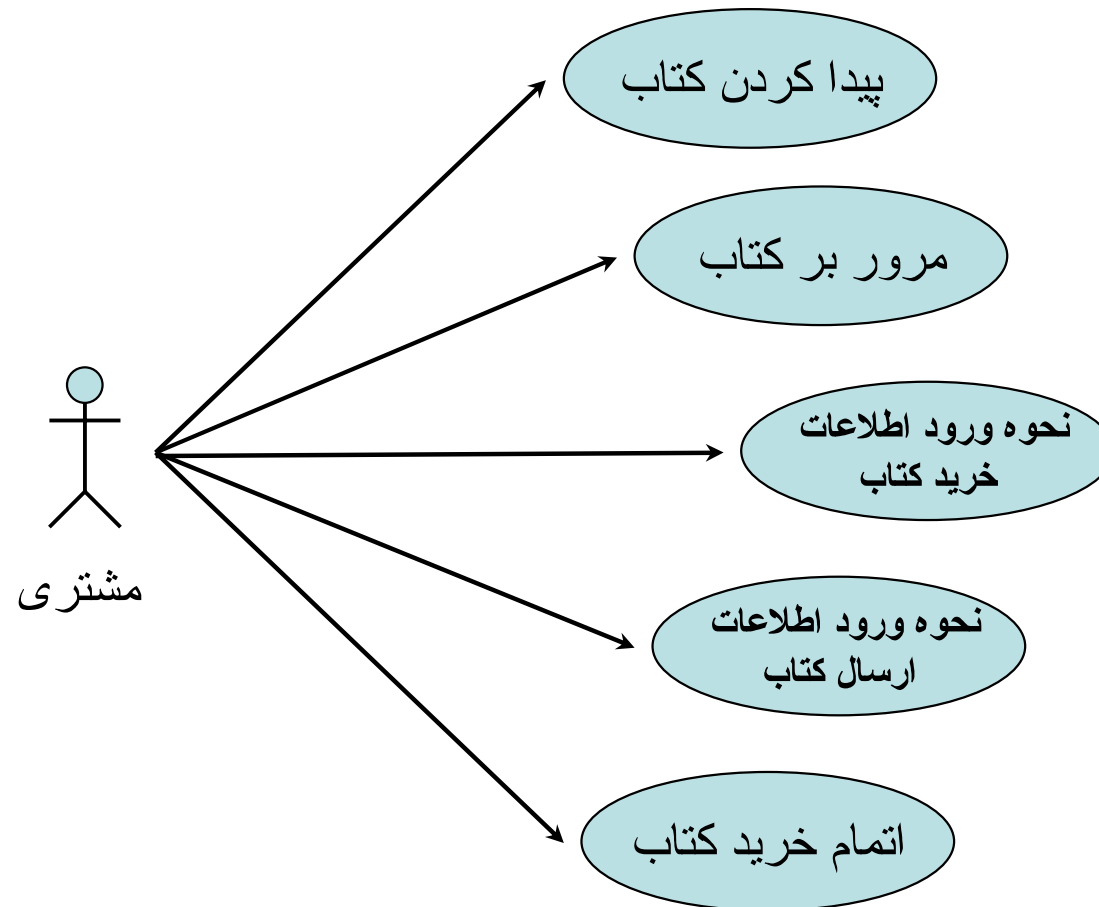
داریم :



همان طور که مشاهده می کنید این کاربرد چندان چیزی را بیان نمی کند و مبهم خواهد بود پس نیاز داریم این کاربرد را تفکیک کنیم .

تفکیک use case خرید کتاب

داریم :



اعمال روابط include و extend

رابط **include** این امکان را به ما می دهد که یک کاربرد را همواره درون یک کاربرد دیگر قرار داد به این شکل که کاربرد اصلی باید به کاربرد درج شده وابسته گردد چرا که مجبور به استفاده از عملیات اوست . این رابطه دو مزیت دارد :

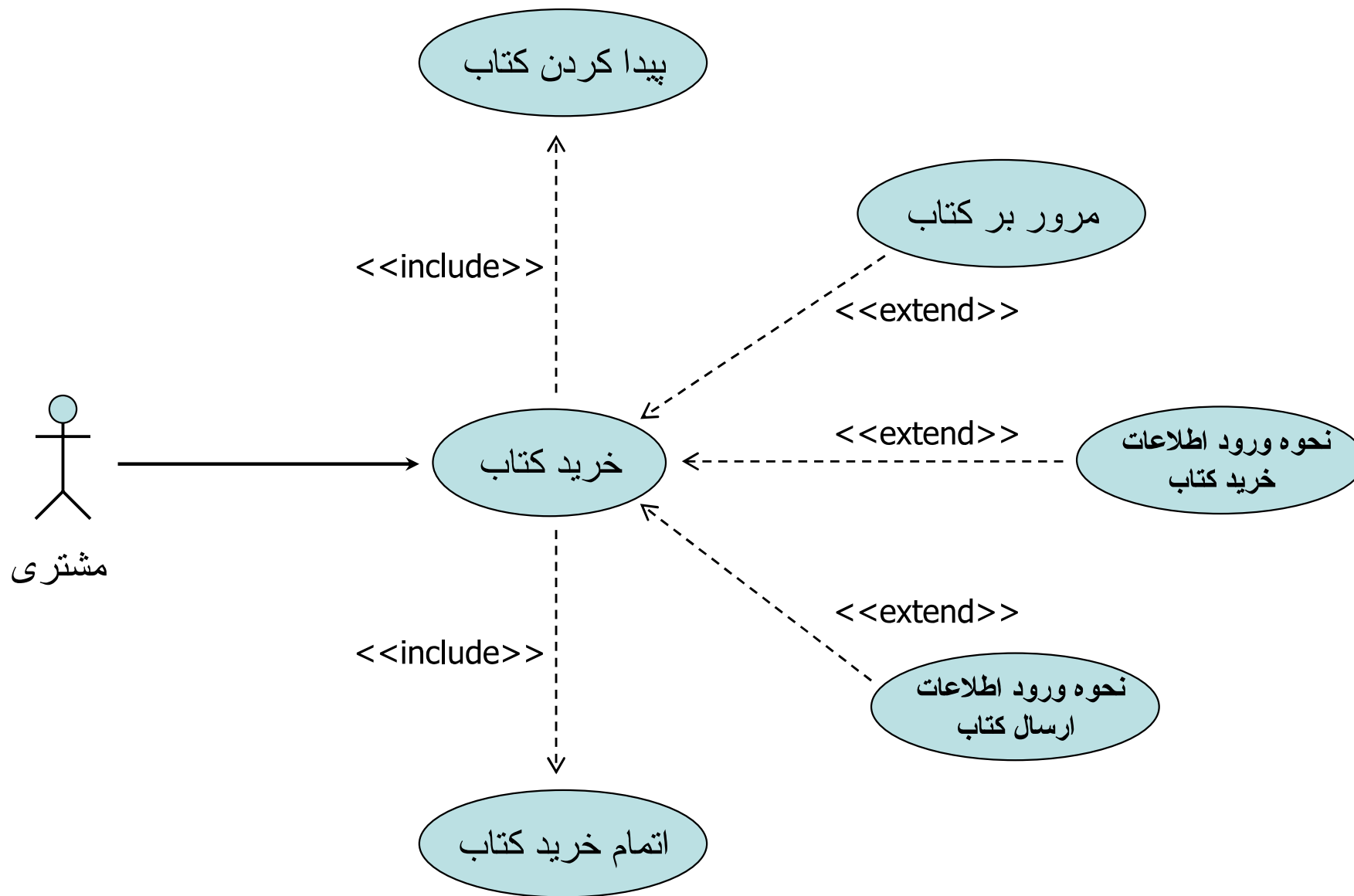
◆ Use case اصلی ساده شده

◆ Use case درج شده را می توان در بیش از یک use case اصلی استفاده کرد .

رابط **extend** این امکان را فراهم می کند تا یک کاربرد را به صورت یک گزینه داخل کاربرد دیگر قرار داد به این شکل که کاربرد اصلی همواره کاربرد توسعه را در بر نمی گیرد بلکه بسته به وجود شرایطی که کاربرد اصلی تعیین می کند ممکن است جریان رویدادهای کاربرد توسعه به جریان رویداد های کاربرد اصلی اضافه گردند . وابستگی از سمت کاربرد توسعه به سمت کاربرد اصلی است .

اکنون این مکانیزم ها را اعمال خواهیم کرد .

کاربرد خرید کتاب به همراه رابط های **include** و **extend**

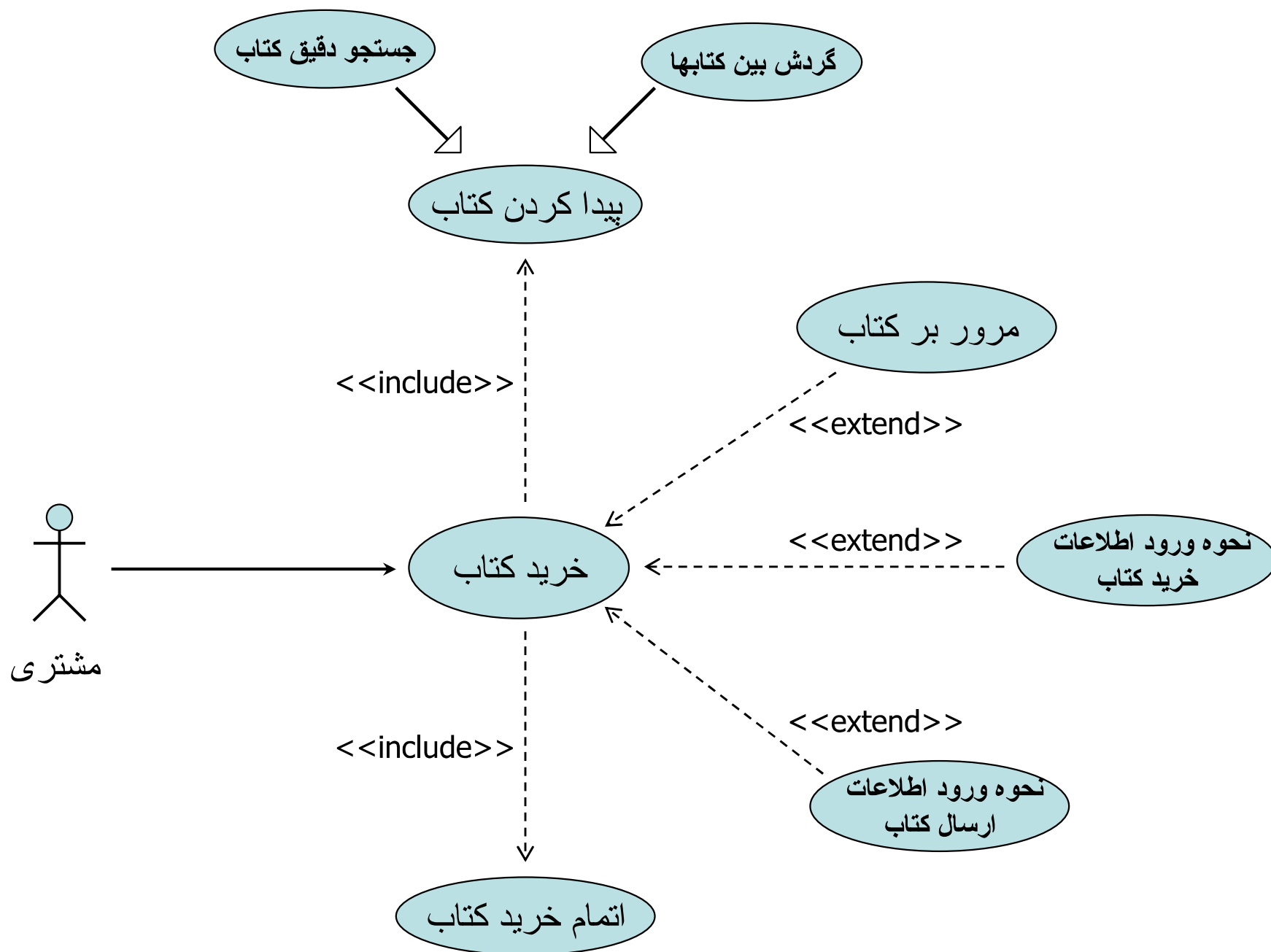


تعمیم use case

در بعضی موارد ممکن است که در یک کاربرد چندین مسیر جداگانه داشته باشیم . مثلا ممکن است برای پیدا کردن کتاب دو راه وجود داشته باشد . مشتری که درک روشنی از کتاب دارد و مشتری که در رابطه با کاتب مورد نظر خود اطلاعات چندان ندارد .

در این قبیل موارد از رابطه **تعمیم** استفاده می کنیم البته بیاد داشته باشید که صرفا براساس میزان پیچیدگی یک کاربرد ، به تعمیم آن مبادرت نورزید .

کاربرد خرید کتاب به همراه تعمیم



روشهای مدل کردن use case

برای مدل کردن use case می توانیم آن را در سطوح مختلف اعمال نمود ، داریم :

- ① مدل کردن یک سیستم
- ② مدل کردن یک subSystem
- ③ مدل کردن یک رفتار سیستم

مزایای استفاده از use case

- ① مدل کردن رفتار یک عنصر با استفاده از دید کاربرد باعث انتقال جزئیات و مفاهیم سیستم به کسانی که بیرون از سیستم هستند .
- ② با این وسیله می توان آن عناصر را سند کرد و برای کاربران آن مولفه
- ③ تست کردن

مزایا در حالت کلی :

- ◆ راحتتر ساخته می شود
- ◆ راحتتر مستند می شود
- ◆ راحتتر تست می شود .

مدل کردن رفتار یک عنصر (قدمها)

1 مشخص کردن Actor های موجود در سیستم

2 عامل های پیشنهادی شامل آن مجموعه ای می شوند که نیازمند رفتار خاصی از آن عنصر هستند برای انجام کارهایشان یا آن عامل هایی که به صورت مستقیم یا غیر مستقیم از کارکردهای سیستم استفاده می کنند .

3 سازمان دهی Actor ها در یک ساختار سلسله مراتبی (تعمیم)

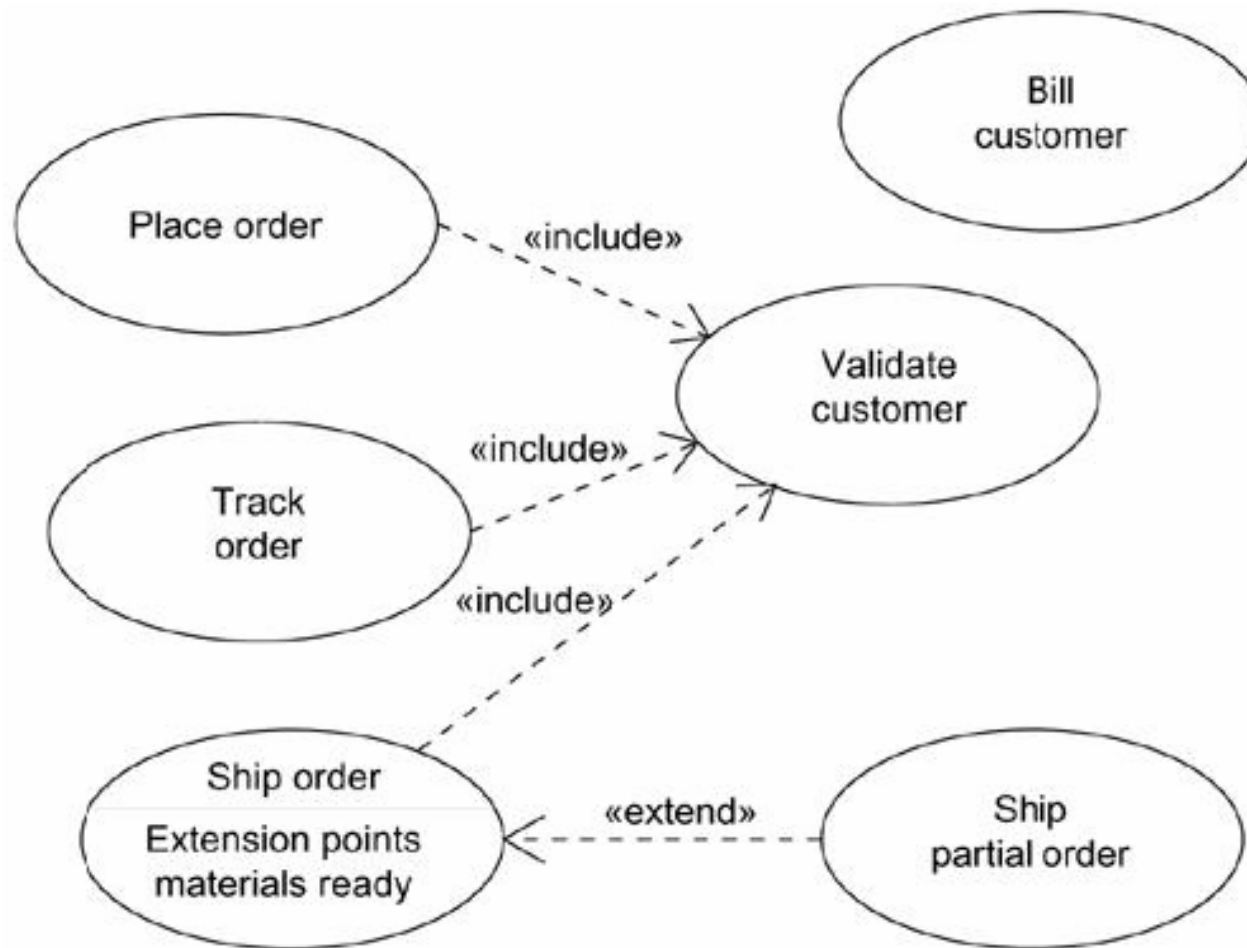
4 برای هر عامل سیستم آن و راه های اساسی که Actor با سیستم در ارتباط و تعامل است مشخص می کنیم .

باید به تعاملاتی که موجب تغییر در عنصر می شود دقت خاصی شود .

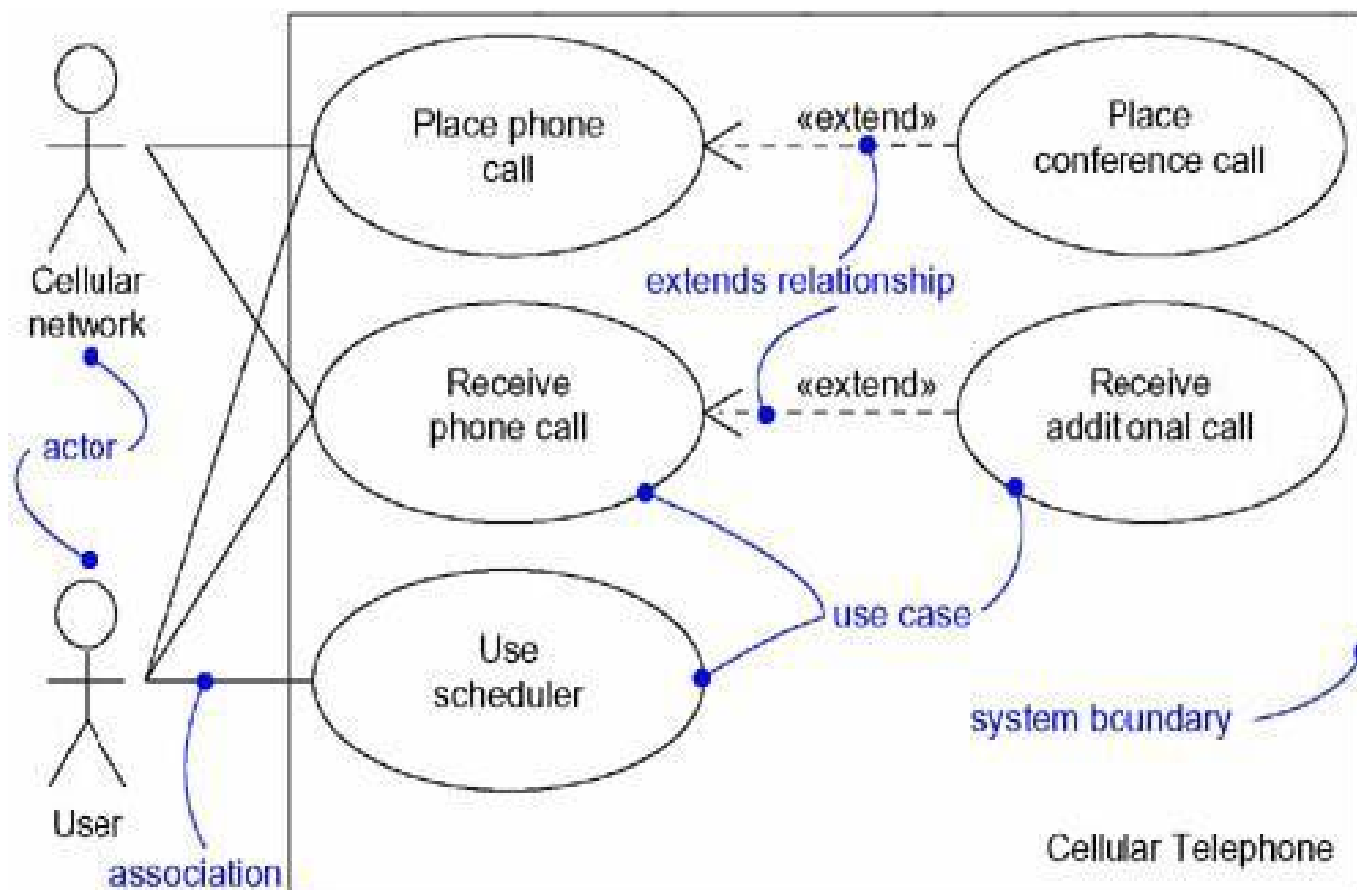
5 حالات استثنا هر Actor با سیستم تعامل می کند مشخص شود .

6 دسته بندی یا به عبارتی سازماندهی use case ها توسط مکانیزم های ذکر شده .

Modeling the Behavior of an Element



A Use Case Diagram : مثال



Use case Diagram

نموداری است برای نمایش مجموعه ای از کاربرد ها و عامل ها و روابط بین آنها

انتظارات از use case Diagram

- 1 مرزبندی سیستم
- 2 مدل کردن نیازمندیهای یک سیستم

در use case view دو بحث را مطرح می کنیم:

- 1 use case diagram
- 2 activity diagram

در use case دیاگرام سه قسمت وجود دارد که شامل:

- 1 actorها
- 2 use caseها
- 3 روابط بین آن ها

در این قسمت دو چیز قابل اضافه کردن است:

① مرز بندی سیستم

② package برای دسته بندی use case می توان به آن اضافه کرد.

برای مدل کردن (زمینه) Context سیستم:

① actor های پیرامون سیستم را باید مشخص کنیم. گروه هایی را باید در نظر بگیریم که به این سیستم نیاز دارند.

② چه گروه هایی نیاز دارند که کارکردهای سیستم را اجرا کنند.

③ چه کسانی با سخت افزار و نرم افزار ها ارتباط دارند

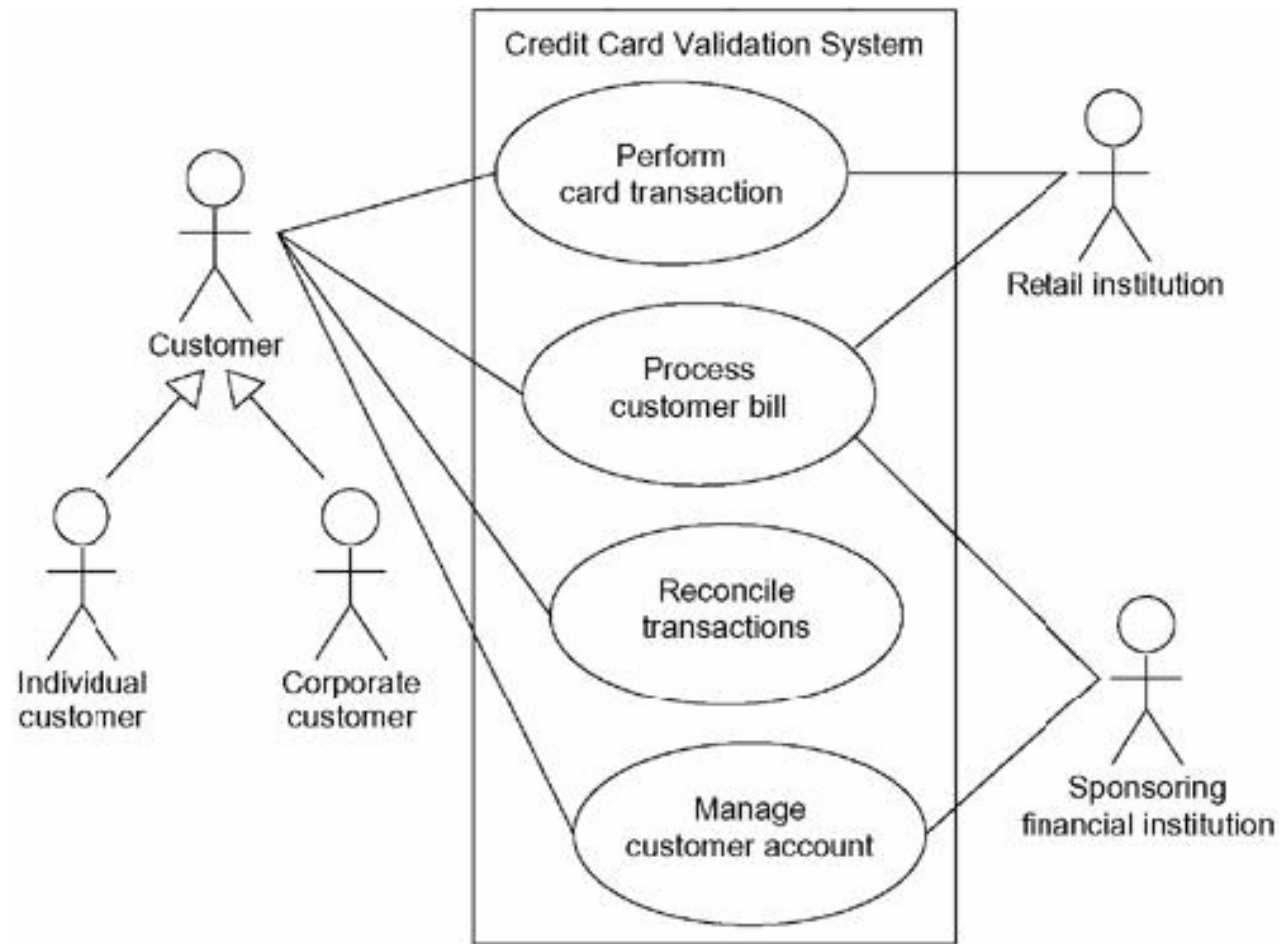
④ چه کسانی مدیریت را روی سیستم ها انجام خواهند داد.

⑤ acotr های سیستم را دسته بندی و سازماندهی کرده.

⑥ اگر به فهم مطالب کمک می کند از stereotype استفاده کنیم.

⑦ use case های کلی که actor ها از آن ها استفاده می کنند را اعمال کنیم .

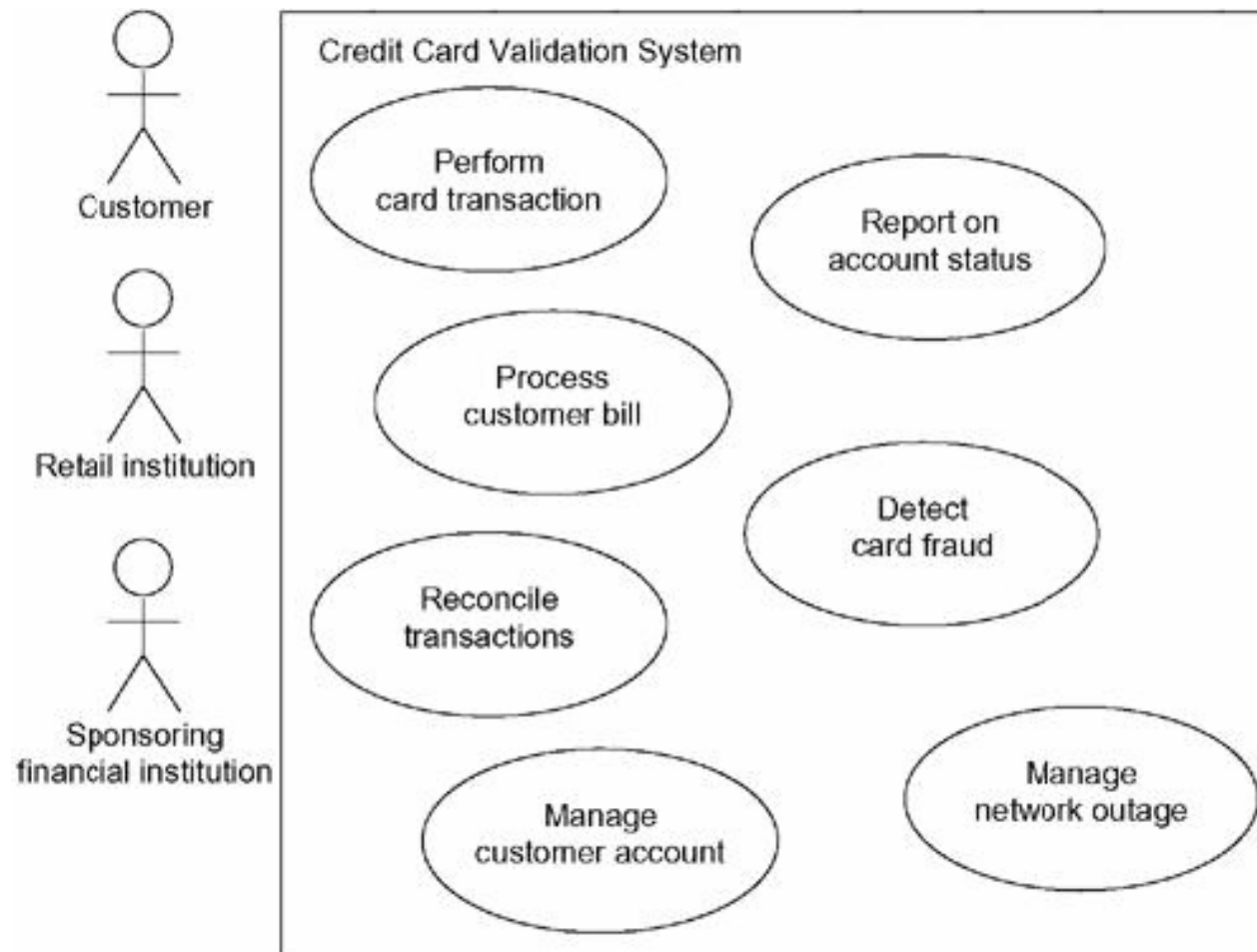
Modeling the Context of a System



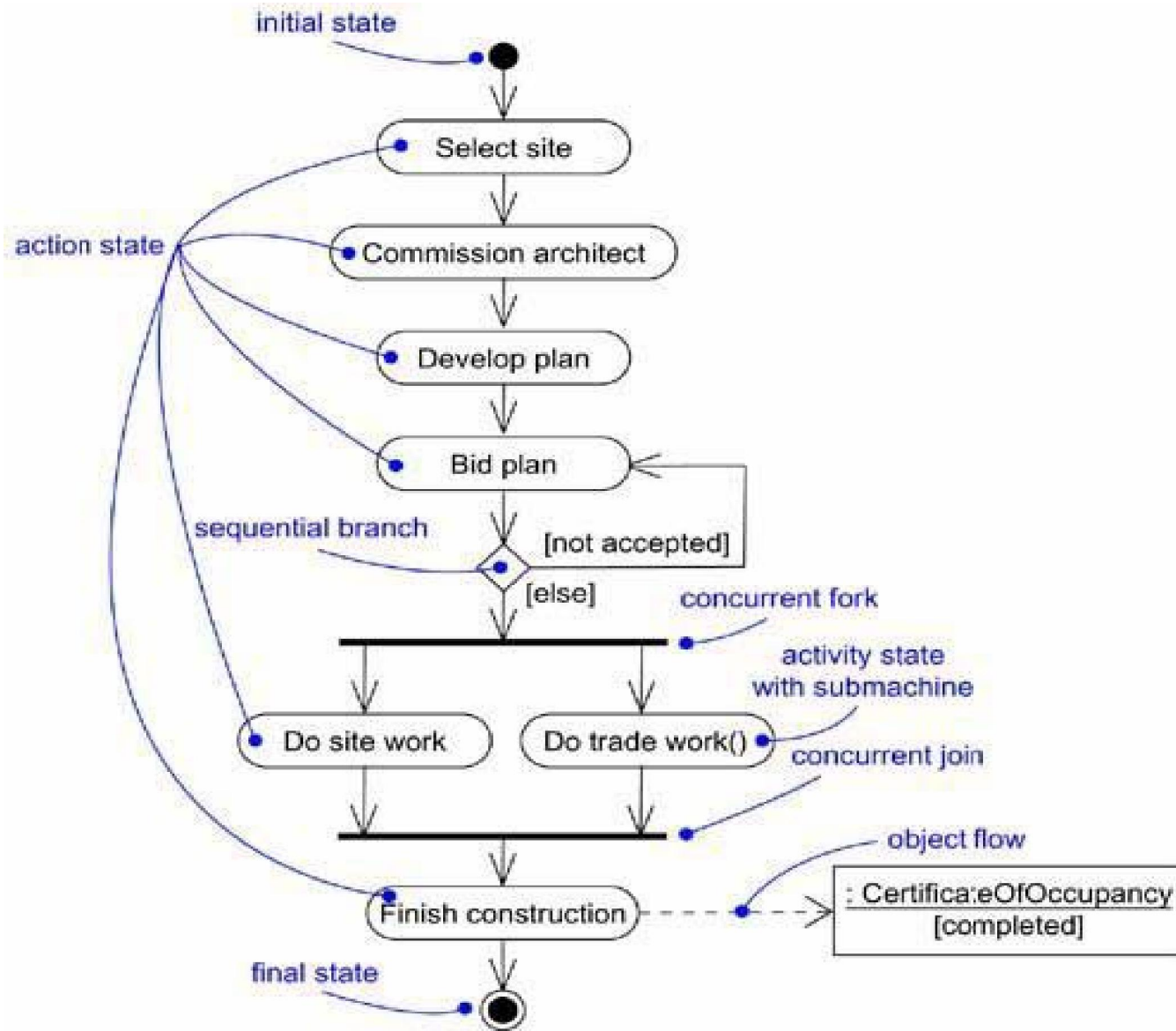
برای مدل کردن نیازمندی ها:

- 1 context سیستم یا الگوریتم سیستم را درآوریم.
- 2 برای هر actor رفتارهایی را که انتظار داریم سیستم برای آن انجام دهد را مشخص کنیم.
- 3 این رفتار های مشترك را تحت نام use case دسته بندی کنیم.
- 4 با استفاده از سه رابطه Extend و Generalization و Include آن use case را سازماندهی می کنیم.
- 5 در صورت لزوم برای هر کدام از use case ها جریان رویدادها را مشخص می کنیم.

Modeling the Requirements of a System



Activity Diagrams

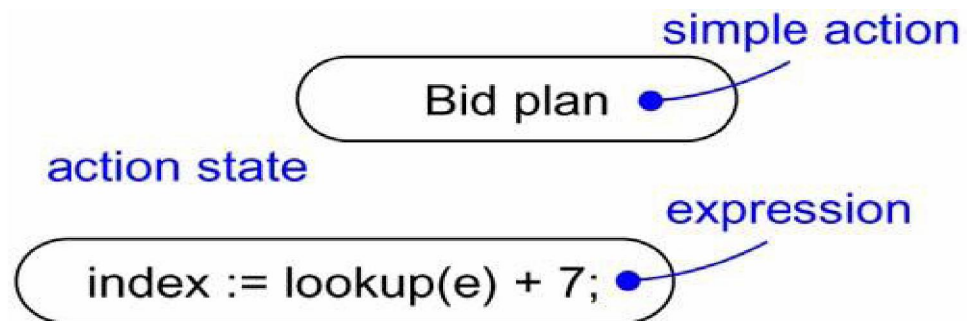


این دیاگرام نمایش دهنده جریان فعالیت ها از يك فعالیت به يك فعالیت دیگری است.
يك فعالیت در حالت انجام غیر اتمیک است.

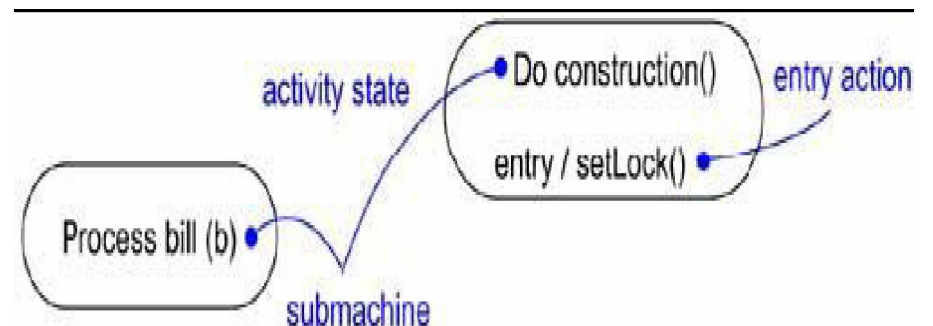
فرق activity با action در چیست؟

در activity يك کاری داریم که قابل تجزیه است یعنی کارهایمان مشخص است.
اما در action يك کار غیر قابل تجزیه است. بعضی از عملیات ها قابل تجزیه نیستند یعنی
ان قدر کوچکند که دیگر تجزیه نمی شوند.

Action States



Activity States



سه قطعه اصلي در activity diagram عبارتند از:

① activity state and action state

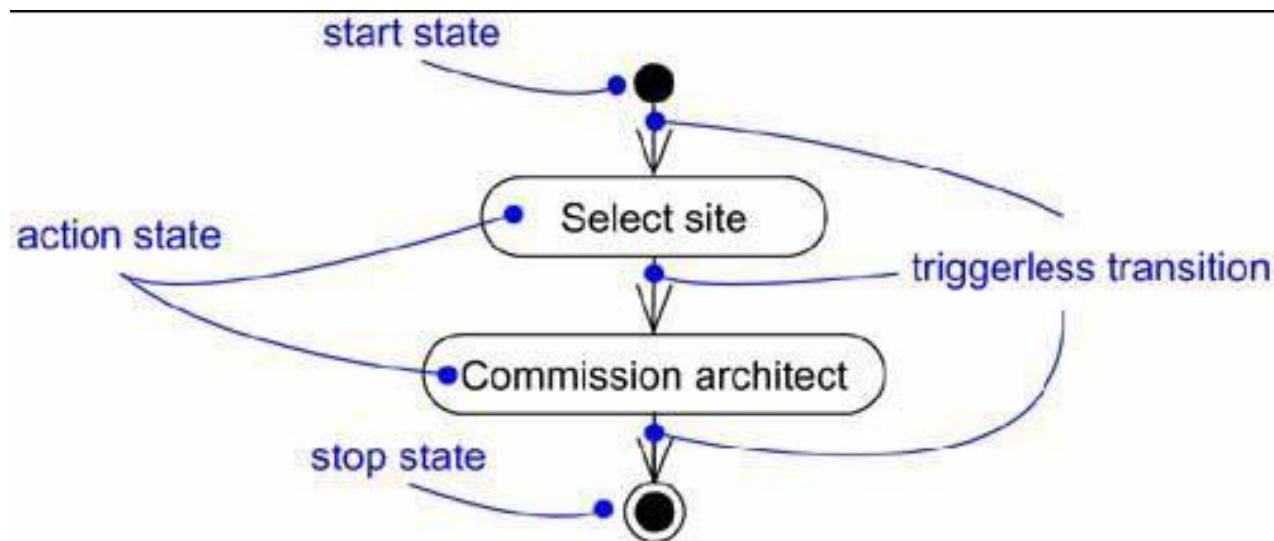
② Transition

③ Object

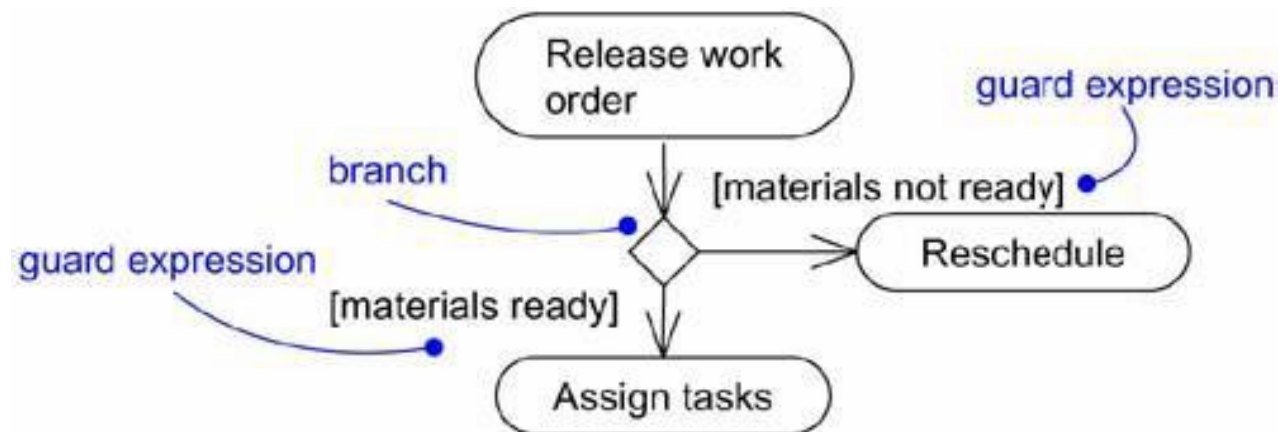
: Transition

مشخص کننده پایان يك Activity و شروع يك Action است.
مشخص مي كند يك Activity يا يك Action تمام شده و چه Activity و يا Action هايي بايد شروع شود.

Triggerless Transitions



Branching



در عملیات join این قضیه را داریم که در آن چند انشعاب و شاخه در یک جا به هم متصل می شوند و یک شاخه را ایجاد می کند که این بحث fork & join را می سازند.

شی ای اجازه fork & join را دارد که:

① استفاده از Thrad

② fork & join باید balance یا تعادل داشته باشد یعنی اگر چهار تا join داشتیم باید چهار تا هم fork داشته باشیم.

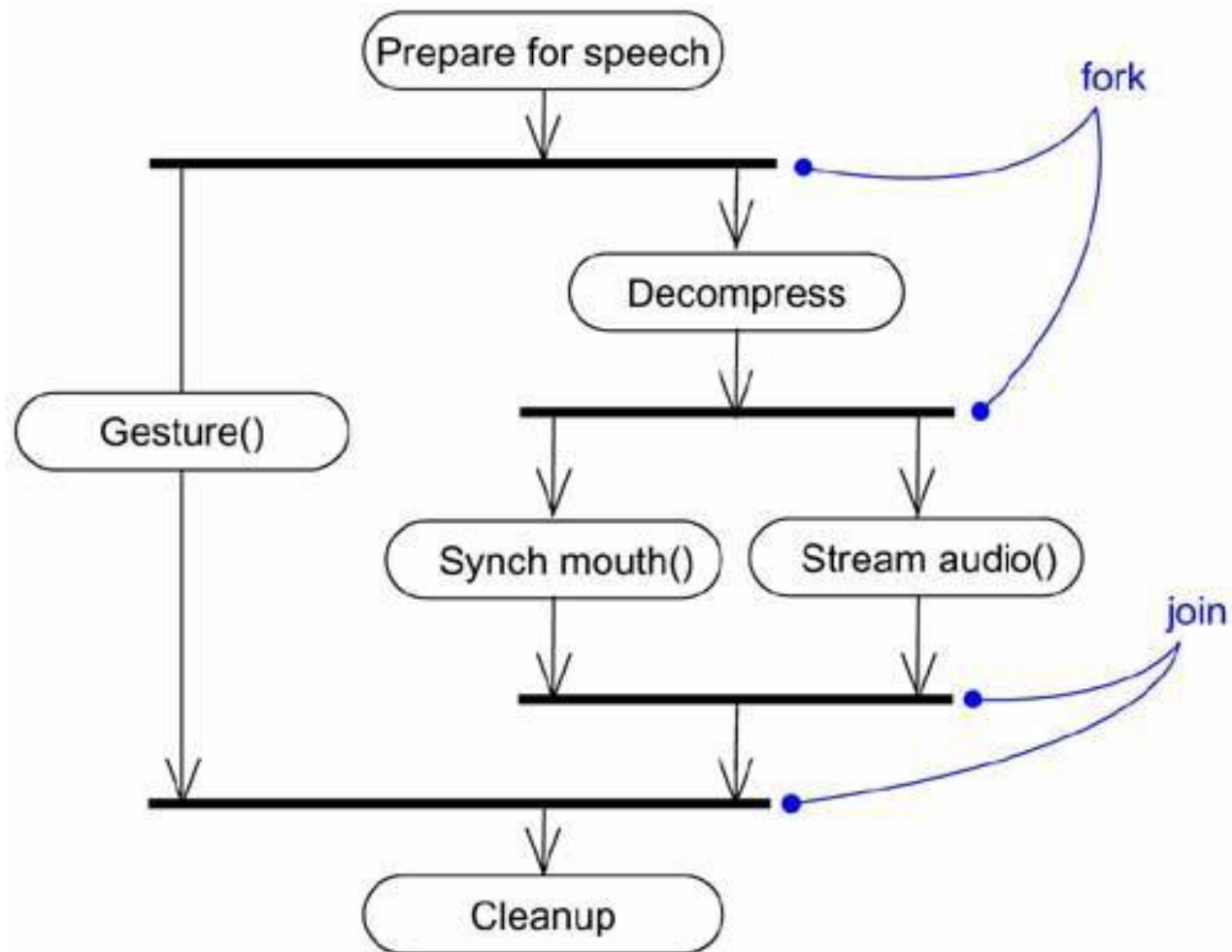
اگر fork & join با هم برابر نباشد يك شاخه اي ازاد باقیمانده كه اين از ايراد كار ماست.

نکته: از swimlane براي گروه بندي activity ها در يك ACTIVITY DIAGRAM استفاده مي شود.

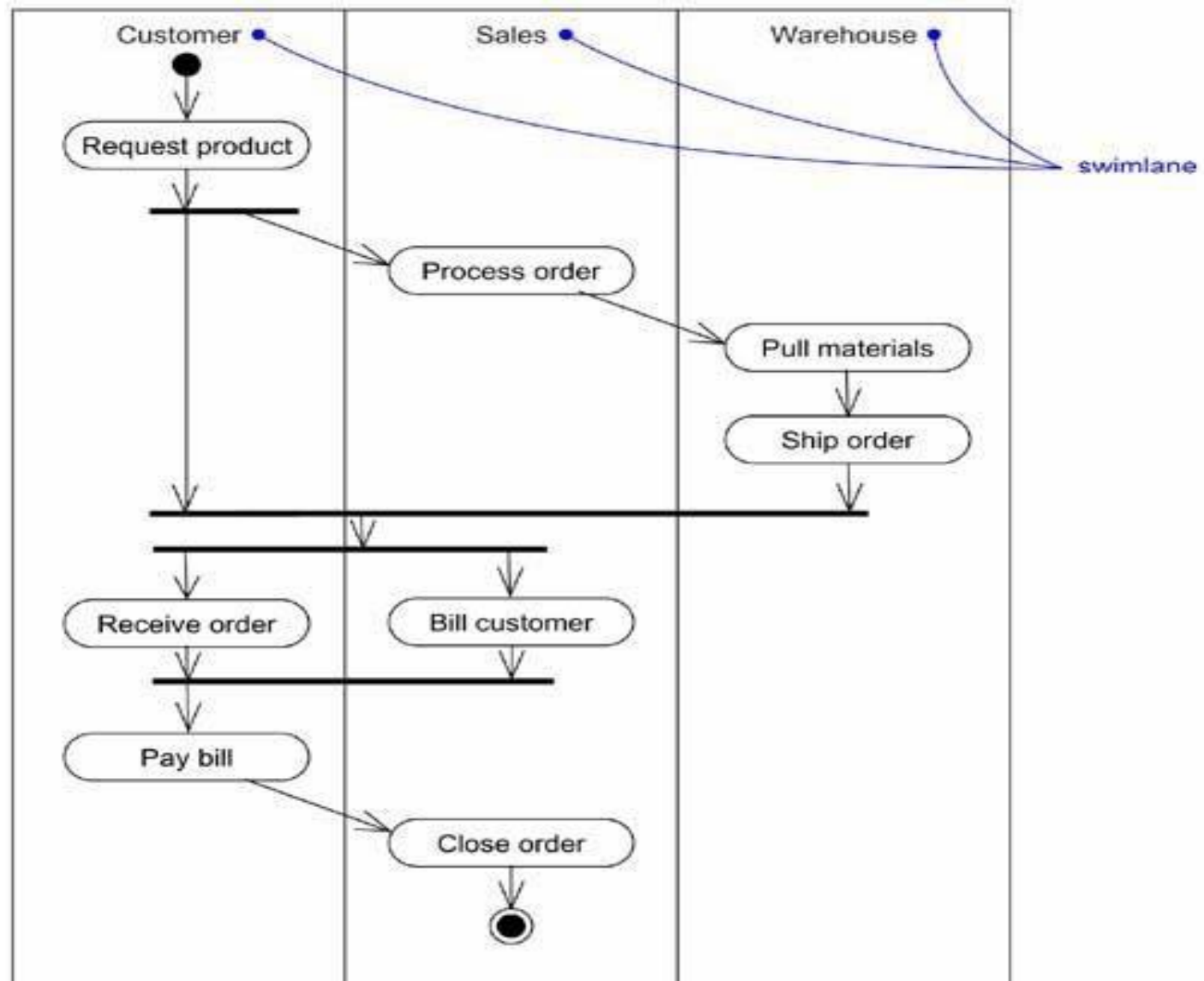
از bissnes logic براي تعيين قواعد يا قوانين حاكم در ان سازمان يا مجموعه مورد استفاده قرار مي گيرد.

bissnes object اشيايي است كه در ان منطق سيستم و سازمان نقش دارد.

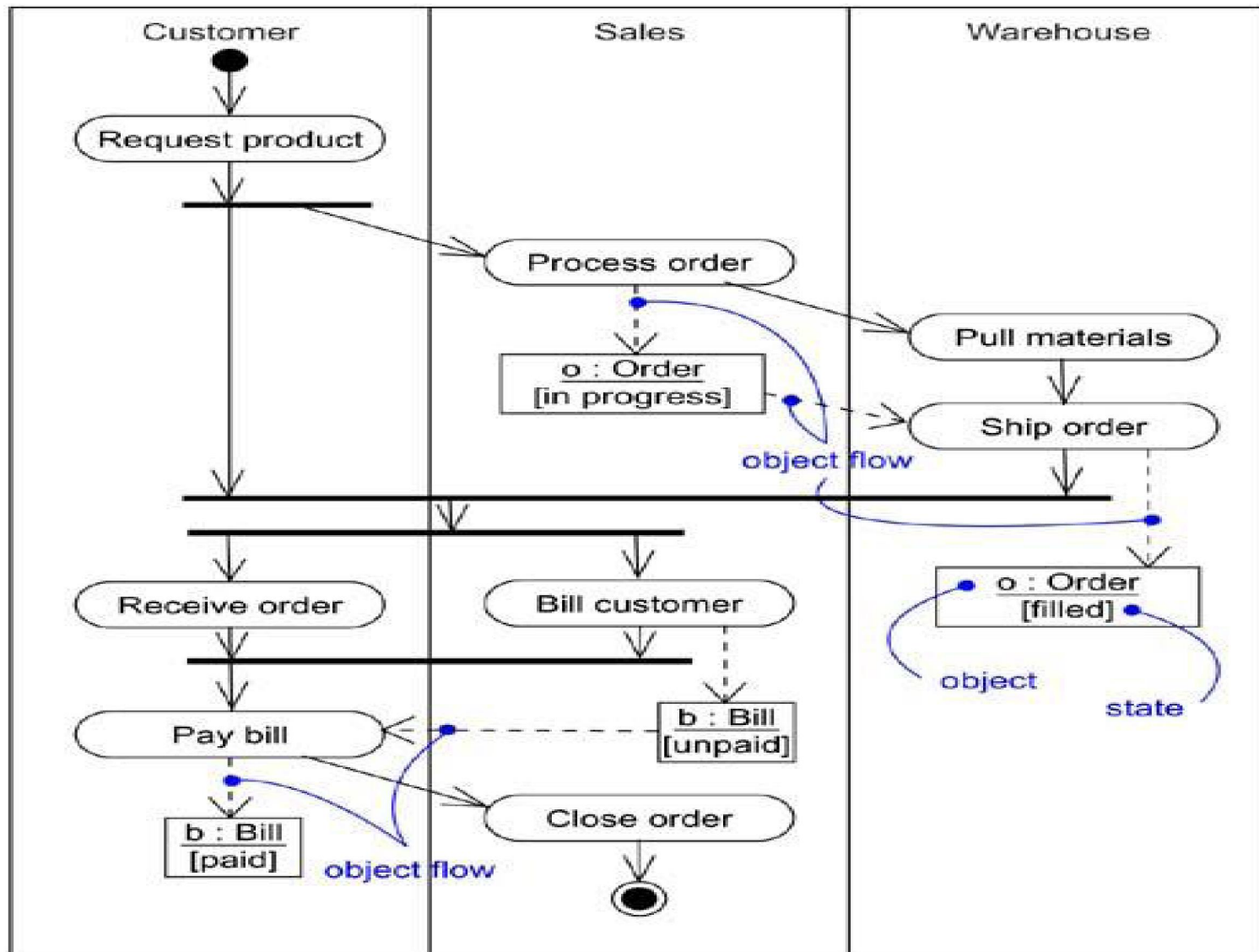
Forking and Joining



Swimlanes



Object Flow



برای مدل کردن یک جریان کاری چه اعمالی را باید انجام داد:

① مشخص کنیم Work flow چه چیز می باشد. بهترین انتخاب آن است که برای هر use case یک activity دیاگرام استفاده کرد.

② آن business object هایی که در بالاترین سطح در این جریان کاری دخالت دارند و مسئولیت دارند را مشخص کنیم. برای هر کدام از اشیا یک swimlane رسم کنیم.

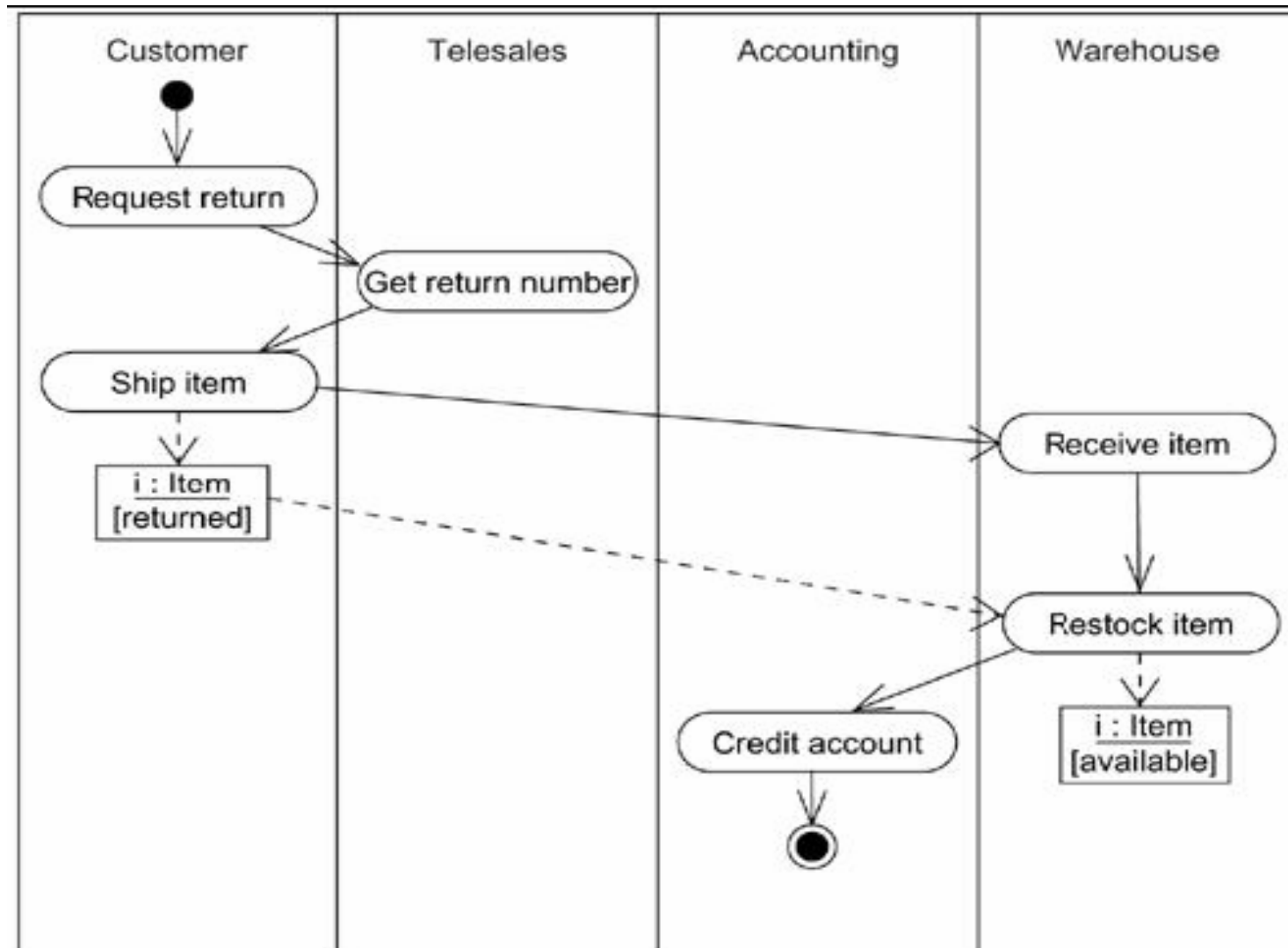
③ باید مشخص کنیم قبل از اینکه این کار انجام شود آیا مقدماتی لازم است یا نه؟ اگر لازم باشد آنها را مشخص کنیم و وضعیت سیستم را در پایان این کارمان مشخص کنیم.

④ با یک دایره توپر آن را نشان داده و شروع می کنیم و activity ها و یا action هایی که رخ می دهد را به شکل مان اضافه می کنیم. اگر مجموعه ای از عملیات پیچیده است آن مجموعه را با یک activity diagram واضح تر می کنیم.

⑤ خطوط انتقال را وصل می کنیم. ابتدا خطوط ساده و بعد خطوط شرطی و بعد خطوط موازی را می کشیم.

⑥ اگر اشیایی وجود دارد که با ارسال آن ها در مراحل مختلف ما می توانیم خوانایی برنامه را بالا ببریم آن اشیا را به سیستم اضافه می کنیم.

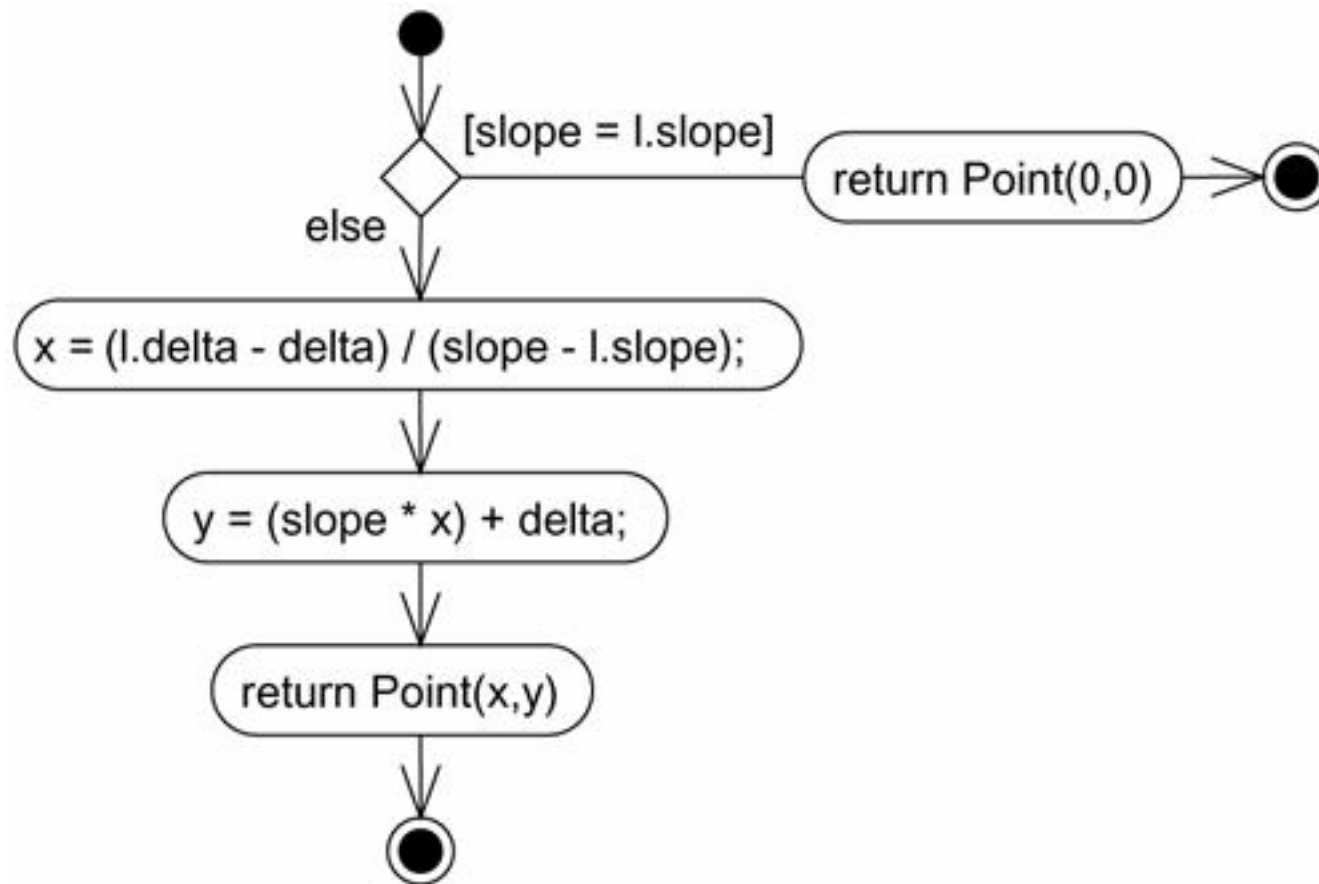
Modeling a Workflow



برای مدل کردن Operation ها:

- ① خود operation را تعریف می کنیم.
- ② پیش شرط ها و وضعیت های نهایی و نیز وضعیتی که برایش اتفاق می افتد را مشخص می کنیم.
- ③ با شک دایره توپر شروع می کنیم و در ابتدا عملیاتی که در آن Opeartion انجام می شود را مشخص می کنیم. در این مرحله شرط ها و حلقه های تکرار را اضافه می کنیم.
- ④ نهایتاً اگر operation مربوط به تگ Object یا Active object است باید از آن استفاده کنیم.

Modeling an Operation



رویداد ها : آن چیزهای که اتفاق می افتند به آنها رویداد گویند و هر کدام از این رویداد ، رویداد یا جریانی هستند که می توان از نظر زمان و مکان مشخص کرد .

☑ در **State machine** ما از رویداد ها استفاده می کنیم .

☑ این رویدادهایی که در سیستم مدل می کنیم محرک های هستند که باعث انتقال در سیستم می شوند در واقع یک شی می تواند توسط این رویدادها از یک حالت به حالت دیگر برود.

چیزهای که یک رویداد شامل آنها می تواند باشد :

۱- سیگنال

۲- فراخوانی **call**

۳- گذشت زمان **passing a time**

۴- تغییر در **state**

انواع رویداد ها عبارتند از :

- 1 رویدادهایی که بین Actor و سیستم می باشد که آن ها را External می نامیم.
- 2 رویدادهایی که بین اجزای درونی سیستم می باشد که آن ها را Internal می نامیم.

سیگنال: نوع خاصی از رویداد است که به صورت غیر همزمان کار می کند یعنی يك محرك را می فرستد و منتظر جواب نمی ماند و کار خود را ادامه می دهد. يك سيگنال يك شي نامدار است که در سیستم توسط يك شي به صورت غيرهمزمان رها می شود و يك شي ديگر ان را دریافت می کند.

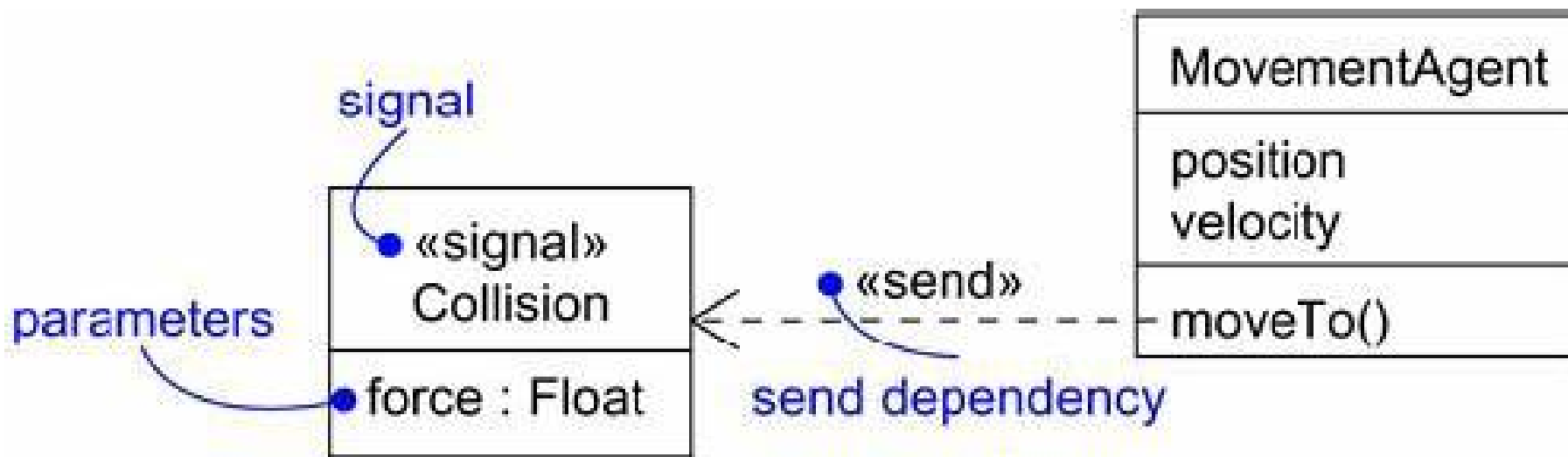
✓ برای نمایش دادن سیگنال از رابطه dependency استفاده می کنیم.

مثال : فرستادن Exception ها

✓ خود سیگنال دارای Attribute و operation میباشد .

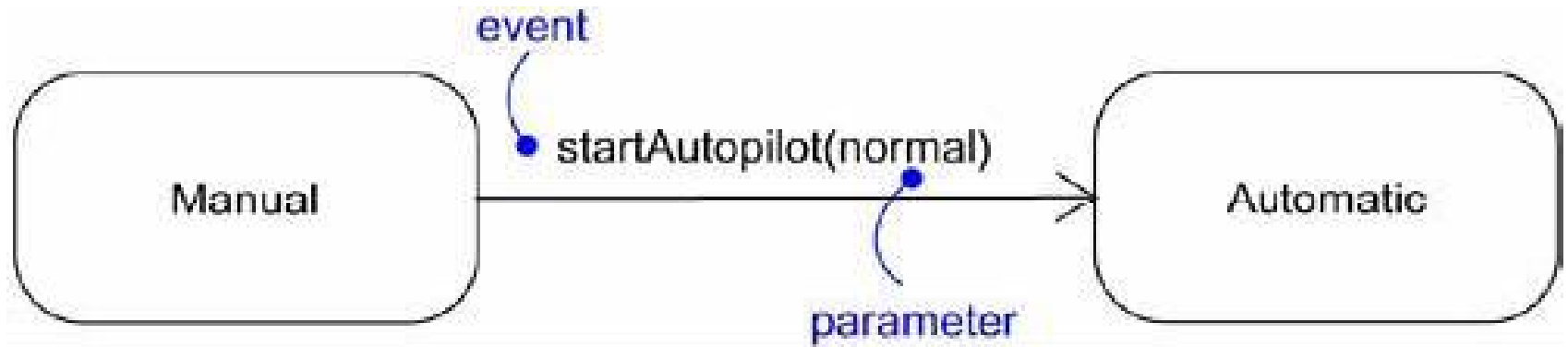
✓ برای نمایش دادن سیگنال از رابطه وابستگی و Stereotype « send » استفاده می کنیم .

مثال :



فرا خواني (call) : يعني **call event** با صدا زدن يك متد و تمام شدن ان متد از يك
وضعيت به وضعيت ديگري مي رود همچنين نوعی رویداد بوده که
حالت همزمانی دارد .

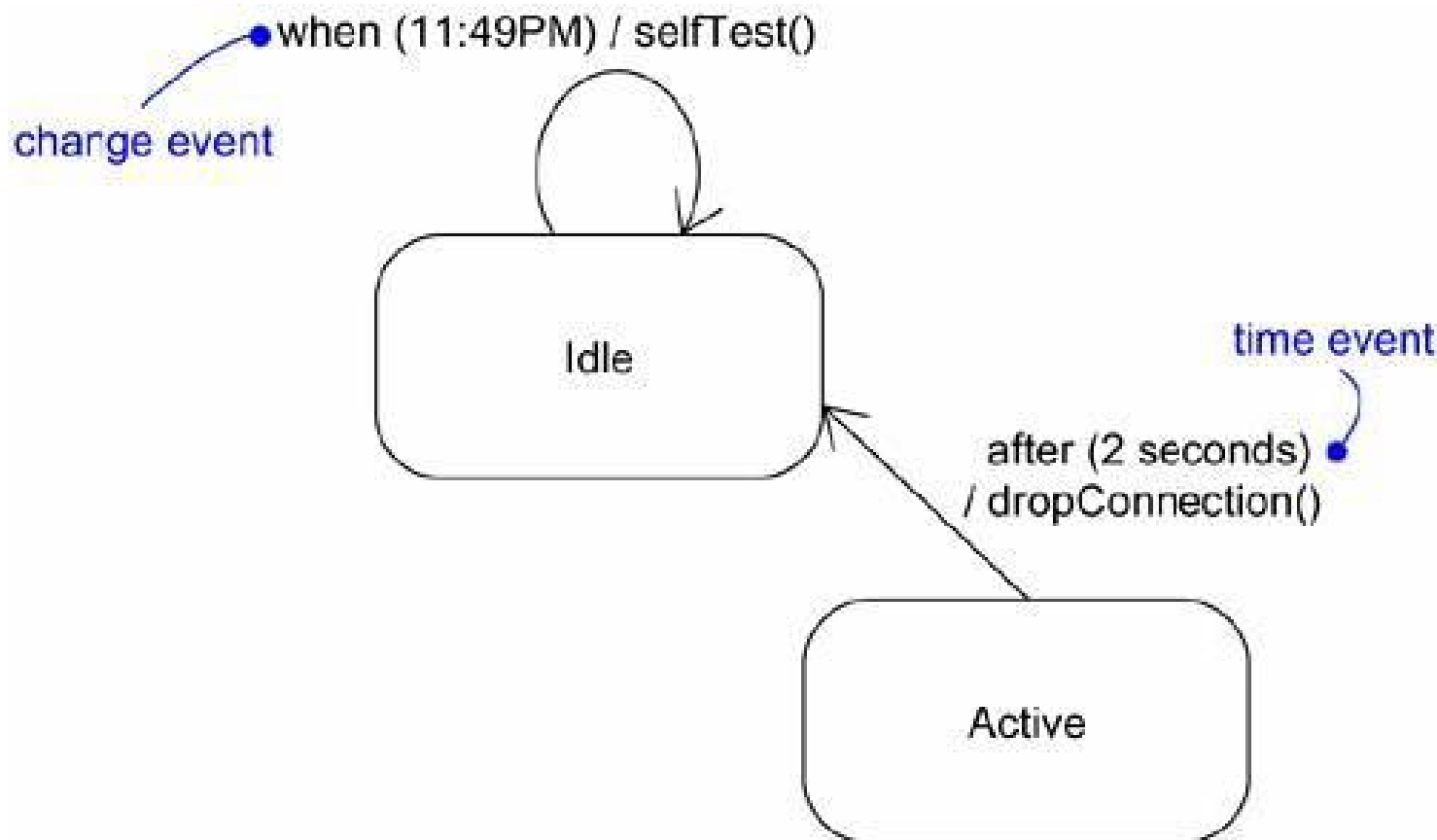
✓مانند مثال زیر که در بازی با استفاده از یک دکمه ماشین از حالت دستی به
اتوماتیک می بریم .



گذشت زمان : Time and Change Events

با کلمه **after** می باشد. روی هر انتقال دو اطلاع مهم می توانیم داشته باشیم یکی زمان و دیگری اتفاقی که می افتد.

مثال : با فرض کانکت بودن مودم :



✓ روی یک انتقال ما دو اطلاع مهم داریم که با اسلش (/) جدا می شود .

۱- زمان مورد که باید بگذرد .

۲- **Action** و کارهای که بعد از این زمان باید انجام دهد .

تغییر وضعیت : با کلمه **when** می آید و بعد از اسلش قرار می گیرد.

✓ روی یک انتقال ما دو اطلاع مانند فوق در تغییر زمان داریم :

۱- ابتدا پس از **when** یک شرط و سپس بعد از اسلش (/)

۲- کار های که باید انجام شود .

✓ **نکته :** در در سیگنال و فراخوانی دو **object** داریم ولی در تغییر زمان و تغییر وضعیت ممکن است يك **object** داشته باشیم .

تفاوت سیگنال و فراخوانی (تفاوت Signal & Call)
در يك فراخوانی و قتیتمدی را صدا می زنیم صدا زننده متوقف می شود تا عملا کار متد صدا زننده شده تمام شود که به این کار سنکرون می گوییم .

در سیگنال برای **Run** کردن يك **thread** متد **Start** ان صدا زده می شود . یعنی به طور غیر همزمان است . پس تفاوت **Call** و **Signal** در این است که **Call** همزمان یا سنکرون است ولی **Signal** غیر همزمان می باشد .

مانند : تلفن و پست

برای مدل کردن Exception ها تنها دو کار را انجام می دهیم:

① برای هر کلاس و اینترفیس تمام متدهای آن exception ها را در می آوریم.

② Exception ها را به ترتیب از super به sub مرتب می کنیم .

③ برای هر Operation باید مشخص کنیم چه استثنایی بوجود می آید .

دلیل مدل کردن رویداد ها:

در state machine رویدادهایی که مدل می کنیم يك محرك هایی هستند که موجب تغییر وضعیت می شوند در ماشین وضعیت.

ماشین وضعیت چیست؟

یک ماشین وضعیت مدل کننده چرخه زندگی یک شی می باشد. این شی می تواند نمونه ای از یک کلاس یک **use case** یا حتی کل سیستم باشد.

مثال : یک **Thread** یک نمونه ماشین وضعیت است.

✓ نکته : با رخ دادن یک **Event** ما می توانیم در ماشین وضعیت حرکت کنیم

مثال : سیستم تنظیم دما

✓ نکته : **Activity** دیاگرام نوع خاصی از **stateMachine** است که فقط یک رویداد دارد.

تفاوت بین State machine و Activity diagram ؟

در **state machine** چیزی که برای ما مهم است حالت **event** در **event** است که وجود دارد. انجام فعالیت ها و عملیاتی که رخ می دهد به چه صورت است. اما در **Activity diagram** نوع خاصی از **state machine** که تنها **event** ی که در درونی است وجود دارد پایان یک عملیات است.

تعريف StateMachine :

يك **state machine** يك رفتار است كه مشخص كننده دنباله اي از حالت هاست كه يك شي در زندگي خودش خواهد رفت در واكنش به يك سري رويداد ها به اضافه پاسخي كه به آن رويداد ها خواهد داد.

تعريف State :

يك حالت يا وضعيت يا حالي است كه يك شي در حيات خود خواهد داشت و اين حالت ها مطابق با يكسري شرط ها و معيارها خواهد بود يا اينكه انجام يك عمليات خاص خواهد بود يا انتظار براي يك رويداد خاص خواهد بود .

☑ يك state حالي است كه يك شي در حيات خودش خواهد داشت كه :

① اين حالت ها مطابق با يك سري شرط ها و معيار ها خواهد بود.

② يا اينكه انجام يك عمليات خاصي خواهد بود.

③ يا اينكه انتظار براي يك رويداد خاص خواهد بود.

✓ نکته : يك Object براي يك مقدار محدودي در يك state قرار مي گيرد. يعني يك Object زماني كه در يك state به سر مي برد بايد محدود باشد.

✓ نکته : يك شي در يك state بايد در زمان محدود بسر ببرد .

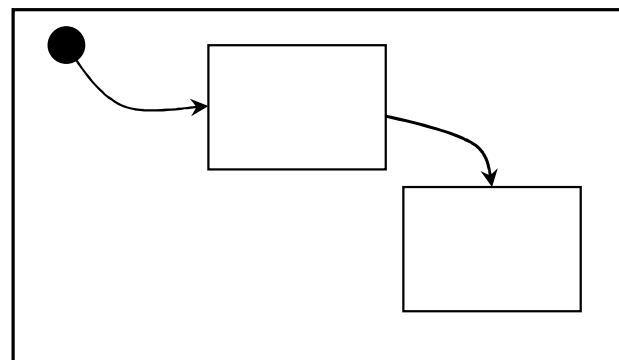
✓ قسمت های State :

① نام : name

② یک Action ورود / یک Action خروج (Entry and Exit Actions)

③ Internal Transition آن انتقال های که می تواند انجام شود درون State بدون تغییر State

④ sub state (زیر وضعیت) : یک وضعیت می تواند یک زیر وضعیت داشته باشد
مانند :



⑤ Deferred events : لیست رویداد هایی است که ان state ان ها را پس می دهد.

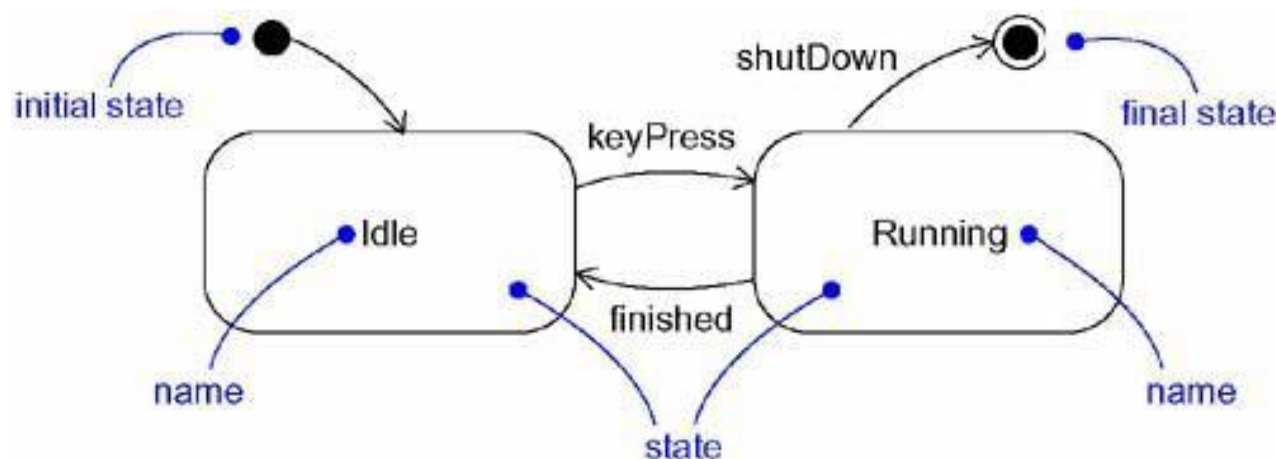
Transition یا انتقال : (خطوط ارتباطی State ها)

يك رابطه اي است میان دو حالت یا وضعیت که مشخص مي کند که شي در وضعیت اول با رخ دادن يك رویداد با انجام يك پاسخ وارد وضعیت دوم مي شود .

Transition

Internal Transition : انتقال داخلی : وضعیت آن تغییر نمی کند ولی وضعیت

مثال :



✓ برای نام گذاری State ها :

- ۱- اسمی که برای يك **state** انتخاب می شود یا در لغت نامه سیستم باید تعریف شده باشد یا اینکه خودمان آن را به لغت نامه اضافه کنیم .
- ۲- اسم گذاری در این قسمت مانند اسم گذاری در **Class** ها می باشد یعنی حروف اول را باید با حروف بزرگ نوشته شود.

✓ در يك **transition** یا انتقال می تواند پنج جز قرار بگیرد که آن ها عبارتند از :

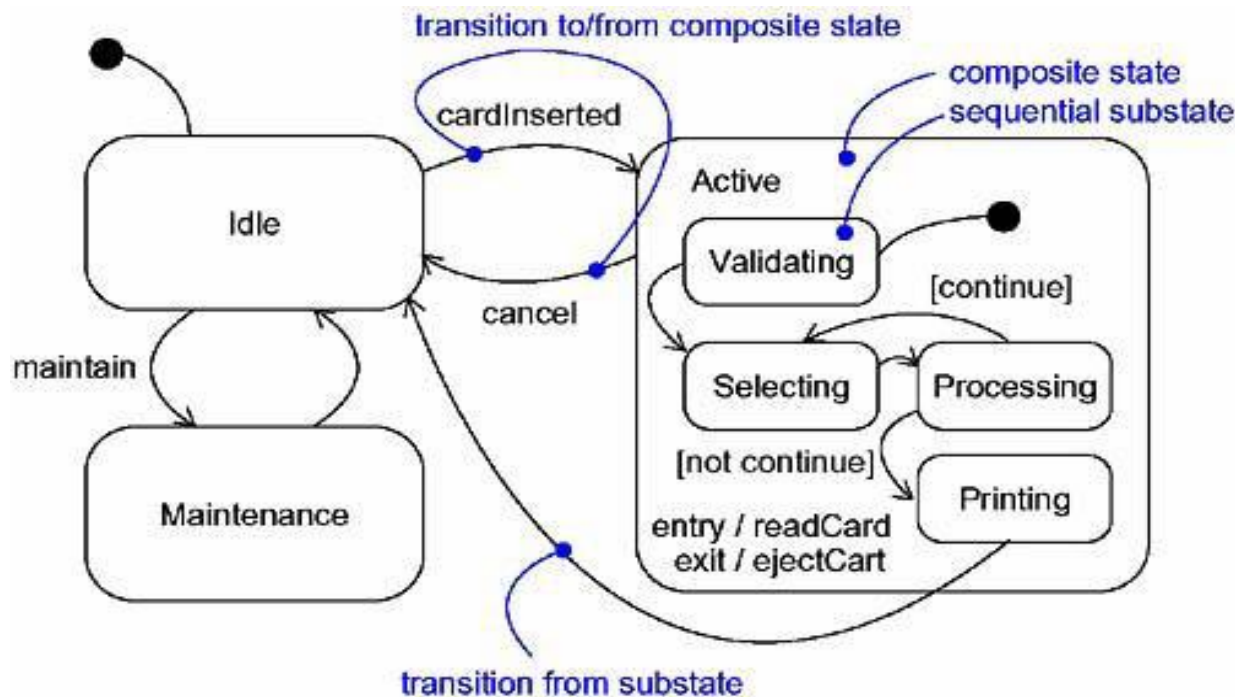
- ① **state** شروع کننده : شروع اش از کجاست.
- ② **event** یا آن رویدادی که موجب این انتقال می شود.
- ③ **state** ی که به آن منتقل می شویم.
- ④ پاسخ و عملی که همزمان به آن انتقال انجام می شود که غیر قابل تجزیه می باشد.
- ⑤ شرط محافظ: شرطی که روی رویداد قرار می گیرد یعنی انتقال وقتی اتفاق می افتد که شرط رویداد برقرار باشد.

✓ فرق self transition با internal transition ؟

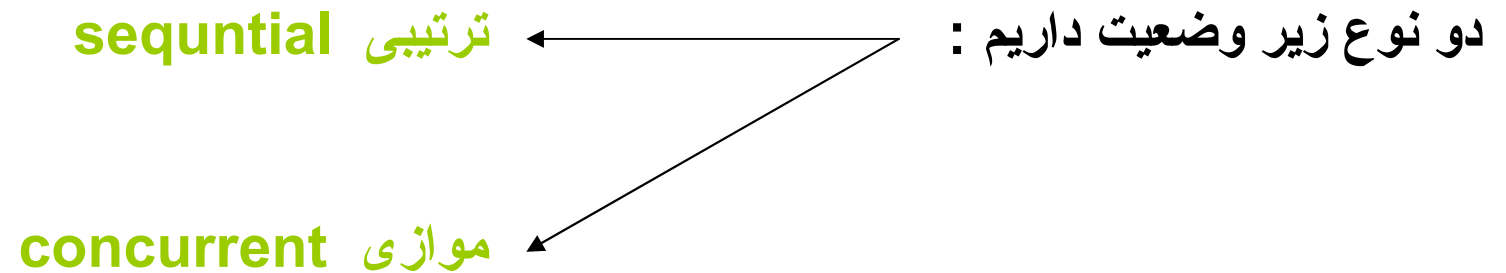
در self transition دو عملیات ورود و خروج state انجام می شود اما در انتقال درونی آن دو عملیات مربوط به ورود و خروج از یک state انجام نمی شود.

✓ **نکته :** یک sub state زمانی اتفاق می افتد که یک سیستم در یک وضعیت باشد حال آن که در همان وضعیت می تواند یک state دیگری را انجام دهد .

Sub State زیر وضعیت : در یک سیستم ممکن است یک State زیر حالت های مختلفی در همان حالت به خود بگیرد .



مثال : یک زیر وضعیت



زیر حالت ترتیبی sequential نقطه مقابل زیر حالت موازی concurrent است .

✓ **نکته :** در زیر حالتی که در آن زیر حالت و وضعیت آن موازی بود باید همه زیر حالت در حالت نهایی ◎ و اتمام قرار گیرند تا از این حالت خارج شویم .

History States (سابقه ی حالت) :

فرضا ممکن است در انجام کاری در انجام یک حالت فرضا نسخه ی پشتیبان گرفتن ما سه مرحله یا حالت داریم ممکن است قسمتی از کار انجام داده باشیم ولی ناگهان برق برود و دیگر از صفحه شروع نمی کنیم بلکه از H کارهای که انجام شده

✓ زیر وضعیت موازی یا همان Concurrent sub state مانند عملیات Fork& join در Java می باشد :

۱- در يك state که از چند Sub state به شکل موازي تشکيل شده زماني از ان وضعیت خارج مي شود که همه sub state موازي خاتمه يافته باشند و اگر يکي از ان ها تمام نشده باشد ان state خارج نمي شود .

۲- در مکانيزم Sequence به ترتيب پس از اجراي همه State که به صورت مرتب است از ان state خارج مي شويم .

۳- در مکانيزم Backing up که با استفاده از History مي باشد. اين history مشخص مي کند کدام sub state بايد انجام شود. يعني بر حسب اينکه قبلا کدام sub state انجام شده مشخص مي کند به کدام sub state برود .

✓ مراحل مدل سازی يك State machine :

1 باید مشخص کنیم که يك کلاس يا **use case** يا سیستم. اگر کلاس است باید به جمع اوري اطلاعات راجع به کلاس ها و کلاس هاي مجاور اطلاعات جمع کنیم. در مورد سیستم بزرگ باید سیستم را از دید يك رفتار خاص مدل کنیم .

2 **Initial state** و **Final state** را رسم کنیم .

3 رویدادهایی را مشخص کنیم که **Object** باید پاسخ دهد. اگر از قبل مشخص شده باشد این رویداد ها را مي توان در **Interface** که صاحب ان **Object** مي باشد مشخص کنیم . اگر مشخص نشده باشد باید به دنبال تکامل ان شي با دیگر اشیا گشت .

4 از **state** اول شروع به رسم کردن وضعیت های سطح بالایی که شی می تواند داشته باشد می کنیم و همزمان آن ها را مشخص می کنیم . در مرحله بعد انتقال را رسم می کنیم البته همراهِ با رویداد اولی که موجب این انتقال می شود.

5 پاسخ را باید در کنار آن بنویسیم .

6 در صورت لزوم هر کدام از **state** ها که لازم است با اضافه کردن **sub state** توسعه می دهیم.

7 در این مرحله باید مدل را چک کنیم که آیا مدل های نوشته شده با **Interface** تطابق دارند یا نه؟

8 با يك ابزار یا يك لیست بین قسمت های مختلف حرکت کنید ببینیم آیا رفتارهای سیستم درست است یا نه؟ آیا نقاط کور وجود دارد یا نه؟